Physics of Deep Learning

Aleksander Czejdo
Tudor Dimofte
Brianna Grado-White
Charles Renshaw-Whitman

October 15, 2026

An introduction to physics-inspired analysis and inquiry in deep learning, particularly focusing on techniques from statistical mechanics, statistical field theory, and (a bit of) dynamical systems.

Contents

-1 Prerequisites **3**

0 A one-day teaching plan **4**
0.1 Schedule . . . 5
0.2 Main references . . . 5

1 Intro reading: learning mechanics **6**

2 Statistical mechanics of learning **7**
2.1 The Bayesian model of learning . . . 7
2.2 Why the exponential of a loss is a likelihood . . . 9
2.3 Partition functions, free energy, and cumulants . . . 10
2.4 A physics example: the Ising model . . . 12
2.5 Phases and phase transitions . . . 15
2.6 The Ising perceptron: a first-order transition in learning . . . 19
2.7 Comparison with QFT . . . 22

3 Large width at initialization: the Gaussian process limit **23**
3.1 Notation . . . 23
3.2 Normalizing the network . . . 24
3.3 Gaussian processes . . . 25
3.4 The NNGP correspondence . . . 28
3.5 NNGP via functional integrals . . . 30
3.6 L/N . . . 32
3.7 Edge of chaos . . . 34

4 The NTK and large width during training **35**
4.1 Gradient flow and the NTK . . . 36
4.1.1 Geometry of gradient flows . . . 36
4.2 Features . . . 38
4.3 Constant NTK at large width . . . 39
4.4 Lazy learning . . . 43
4.5 Summary . . . 43
4.6 Bayesian learning at large width: kernel learning . . . 45

5 Mean-field scaling **47**
5.1 Mean-field theories in physics . . . 47
5.1.1 Warm-up: the Curie–Weiss magnet . . . 48
5.1.2 The general shape of a mean-field model . . . 49
5.2 Mean-field scaling for one-hidden-layer networks . . . 50
5.2.1 Initialization . . . 50
5.2.2 The loss is a mean-field energy . . . 51
5.2.3 Equations of motion and the mean field . . . 51
5.2.4 From particles to measures: the continuity equation . . . 52
5.2.5 The McKean–Vlasov equation and self-consistency . . . 53
5.3 The NTK is no longer constant: feature learning . . . 56
5.4 The scaling dial and Maximal Update Parameterization . . . 56
5.5 What is this framework good for? . . . 57
5.6 Grokking as a lazy-to-rich transition: a reading exercise . . . 58

5.7 Bayesian grokking: a first-order transition (alternative reading) . . . 60

6 Non-equilibrium physics, MSRJD, and DMFT 62
6.1 From stochastic dynamics to path integrals: the MSRJD formalism . . . 63
6.2 Double descent as a dynamical phase transition . . . 66
6.2.1 Model definition and formalism . . . 66
6.2.2 Overview of results via training and generalization error . . . 68
6.3 Dynamical mean-field theory (DMFT) . . . 70
6.3.1 Self-consistency . . . 71
6.3.2 Recent applications in deep learning . . . 72

7 Scaling laws 73
7.1 The empirical laws . . . 74
7.2 Joint laws, bottlenecks, and the Chinchilla revision . . . 75
7.3 Where do the laws come from? . . . 76
7.4 A survey of theoretical proposals . . . 78
7.5 The quantization model: exponents from the statistics of skills . . . 79
7.6 Emergence: smooth laws from staggered cliffs . . . 81
7.7 Exponents from geometry and from spectra . . . 82
7.7.1 Geometry: the data-manifold argument . . . 82
7.7.2 Spectra: scaling laws from kernel eigenvalues . . . 83
7.8 A solvable large-N field theory . . . 83
7.9 All the knobs at once: a dynamical theory . . . 84
7.10 Beyond pretraining: post-training and test-time scaling . . . 85

A The replica method 86
A.1 What the annealed approximation gets wrong . . . 86
A.2 Replicas . . . 87
A.3 A variant: the spherical perceptron . . . 89

B From linear regression to kernel regression 91
B.1 Linear regression . . . 92
B.2 Linear regression with nonlinear features . . . 94
B.3 Kernel regression . . . 97
B.4 Kernel ridge regression . . . 99
B.5 Linearized networks are kernel ridge regressors . . . 102

C Solutions to exercises 103

-1 Prerequisites

We don't assume any physics background, though reading some of the suggested reference material may be helpful. We do assume:

- a basic understanding of probability theory and Bayesian inference;
- familiarity with simple neural networks (MLP's);
- undergraduate-level calculus and linear algebra;

- and, in a few places, differential geometry.

What you'll learn

By the end of these notes, you should be able to:

- Use the statistical-mechanics dictionary for Bayesian learning — energy, entropy, partition function, free energy, cumulants, order parameters, quenched averages — and explain why the exponential of a loss function is a likelihood.
- Solve the Ising model in the mean-field approximation; explain what a phase transition is, why finite systems show smooth sigmoids and growing susceptibility peaks instead of true kinks, and how the Ising perceptron realizes a first-order transition to perfect generalization, metastable lag included.
- Derive the Gaussian-process description of a wide network at initialization, and explain the roles of the fan-in normalization, the depth-to-width ratio, and the edge of chaos.
- Derive the NTK, and explain why it freezes at large width — so that training reduces to kernel regression, with no feature learning.
- Derive, heuristically, the mean-field limit of one-hidden-layer networks — neurons as interacting particles, self-consistency, the McKean–Vlasov equation — and explain how its *moving* kernel encodes feature learning; place the NTK and mean-field limits on the scaling dial, and connect it to μP and hyperparameter transfer.
- Compare the two theoretical accounts of grokking — an equilibrium first-order transition vs. delayed lazy-to-rich dynamics — and propose an experiment that would distinguish them.
- Recognize the MSRJD/DMFT path-integral formulation of training dynamics, the role of correlation and response functions, and double descent as a dynamical phase transition.
- Derive the three scaling laws of the quantization model, explain "emergence" as the aggregation of staggered cliffs, and survey where scaling exponents come from: skill statistics, data geometry, and kernel spectra.

0 A one-day teaching plan

These notes contain far more than can be taught in a day. This section records one selection of material that was taught in a single day at the Iliad Intensive (September 2026, day B.6), as a worked example of how the notes can be used. The day alternates

three lectures, given from the slide deck that accompanies these notes, with exercise sessions in which participants pick one exercise from the notes and work on it, alone or in small groups. It closes with a short reading and a discussion.

The deck is the file `slides.tex` in the same folder as these notes. The website build compiles it and offers it in the "Slides" row at the top of this chapter's web page; it can also be opened directly at iliad-team.github.io/iliad-intensive/downloads/qft/qft-slides.pdf. Its three parts are the three lectures below: frames 1–20 (Part I), 21–41 (Part II), and 42–52 (Part III). The lectures deliberately leave derivations to the exercise sessions, so the deck is sparse by design; the corresponding sections of these notes fill in what the slides state.

0.1 Schedule

time	session	where in these notes
10:00–11:00	Lecture, Part I: Bayesian learning and phase transitions	Sections 2.1 to 2.5
11:00–11:15	Break	
11:15–12:00	Exercises, Part I (choose one)	Exercises 2.1 to 2.3
12:00–12:30	Lecture, Part IIa: large width at initialization; the NTK	Sections 3.1 to 3.4, Section 3.6, Sections 4.1 to 4.2
12:30–1:30	Lunch	
1:30–2:00	Lecture, Part IIb: large width during training; mean-field scaling	Sections 4.3 to 4.4, Sections 5.2 to 5.4
2:00–3:00	Exercises or lab, Part II (choose one)	Exercises 3.3, 4.4, 4.5 and 5.3
3:00–3:45	Lecture, Part III: neural scaling laws, in theory and practice	Sections 7.1 to 7.5
3:45–4:15	Break	
4:15–5:00	Exercise, Part III	Exercise 7.1
5:00–5:30	Reading on "learning mechanics"	Section 1
5:30–5:50	Discussion in small groups	the prompts of Section 1
5:50–6:00	Feedback	

0.2 Main references

The day is based on the following references, all of which appear elsewhere in these notes. For Part I: Tong's lectures on statistical field theory and on kinetic theory [89, 88], the physics behind the lecture; Engel and Van den Broeck [26] for the perceptron tradition; and Györgyi [35] for the Ising perceptron transition. For Part II: Hanin's lectures [37] and Roberts, Yaida, and Hanin [71] on large-width limits and the L/N expansion; Ringel et al. [69] on statistical field theory in deep learning; Spiliopoulos, Sowers, and Sirignano [84] for rigorous NTK and mean-field limits; Lin [54] for experiments identifying features through the empirical NTK; and the two accounts of grokking [49, 73], which were not lectured but are the natural follow-up reading (Sections 5.6 and 5.7). For Part III: Kaplan et al. [44] and Henighan et al. [39] for the

empirical laws; Hoffmann et al. [41] and Porian et al. [66] for Chinchilla and the reconciliation with Kaplan; Michaud et al. [57] for the quantization model; and Maloney, Roberts, and Sully [55] and Bordelon, Atanasov, and Pehlevan [9] for the solvable field theories.

A complementary introduction to the physics of learning, taught at the August 2026 Intensive by Renshaw-Whitman, can be found in [68]; Appendix B is adapted from it.

1 Intro reading: learning mechanics

The review [80] argues that a scientific theory of deep learning — a "mechanics of learning" — is already emerging, and it organizes the evidence into five converging strands: analytically solvable settings, insightful limits, simple empirical laws, disentangled hyperparameters, and universal phenomena. The body of the paper develops the first two strands in detail and samples the remaining three. These notes will do much the same, so it is worth keeping the five strands in mind as a map of everything that follows.

The paper's starting observation deserves to be savored, because it sets the tone for the whole subject. A deep learning system is completely specified by four explicit ingredients: an architecture (a composition of simple linear and nonlinear maps), a dataset, an objective, and a learning rule (gradient-based updates from a random initialization). Nothing is hidden: every parameter, every gradient, every intermediate activation can be measured, at every step, to whatever precision we like. In the language of the reading, deep learning directly exposes its own equations of motion — *the central challenge is not opacity but complexity.* This places the field in very different territory from the genuinely opaque sciences: in neuroscience or cell biology the microscopic equations of motion are unknown and the state is largely unmeasurable, while in high-energy physics we must infer the microscopic laws indirectly, from scattering amplitudes. The better analogy is fluid dynamics: there, too, we possess the microscopic law — for deep learning, the update equation is five lines of code — and what is missing is the *effective* description, the analogue of Navier–Stokes.

Read **Section 2** (only) of [80], and think about the following:

1. *Rank the physics analogies in Table 1 of the reading* (solvable models ↔ hydrogen atom; limits ↔ thermodynamic limit; empirical laws ↔ Kepler and Planck; hyperparameters ↔ Reynolds number; universality ↔ critical phenomena). Which are the most compelling? Which do you think are most useful in understanding deep learning?

2. *Discretization and other approximations.* The reading's Discretization Hypothesis proposes that practical networks are best understood as noisy, finite approximations to models of infinite size — like a finite-difference discretization of a PDE — so that finite-size effects typically *cost* performance rather than enable it. Given what you've seen so far, does this seem accurate? The hypothesis is falsifiable: it would be refuted by a finite-size effect that delivers a general benefit unachievable in any limit. What are some possible experiments/setups that could falsify it?

3. *What actually matters for alignment?* The reading wants falsifiable, quantitative, *average-case* predictions rather than worst-case bounds — physics rather than classical learning theory. What is lost? E.g. for alignment, does one want worst-case guarantees or forecasting of typical capabilities? Which of the five lines of evidence is most relevant for alignment?

You may want to revisit these questions at later stages as you go through these notes, and learn more about physics-inspired approaches in deep learning.

2 Statistical mechanics of learning

There is a large and well-developed body of work applying physics to learning theory. Much of it consists of *statistical mechanics* applied to *Bayesian learning*. The classic textbook is Engel and Van den Broeck [26]; modern reviews include [98, 6]; and an excellent reference for the physics itself is David Tong's lecture course on statistical field theory [89], which we follow closely in Sections 2.4 to 2.5.

In this section we focus on the Bayesian model of learning. We first recall what it is, why it is a reasonable idealization of how networks are actually trained, and why the exponential of a loss function deserves to be called a likelihood (Sections 2.1 to 2.2). We then switch to physics. We introduce partition functions, free energies, and cumulants (Section 2.3), and work through the Ising model in the mean-field approximation, building up to first- and second-order phase transitions (Sections 2.4 to 2.5). The payoff is a learning problem that is the Ising magnet's twin, the Ising perceptron, which shows a first-order transition to perfect generalization; it is left as a self-contained walk-through exercise (Section 2.6). Section 2.7 is a translation guide for readers arriving from quantum field theory. The ideas introduced here — order parameters, free energies, saddle points, quenched averages, phase transitions — will keep returning throughout the notes.

2.1 The Bayesian model of learning

Consider a network f_θ with parameters $\theta \in \mathbb{R}^P$, trained on n samples $(x^\mu, y^\mu)_{\mu=1}^n$ to minimize a loss

$$\mathcal{L}(\theta) = \sum_{\mu=1}^{n} \ell\big(f_\theta(x^\mu), y^\mu\big)\,, \tag{2.1}$$

a sum over samples of a per-sample loss ℓ. In practice, the network is initialized at random, $\theta_0 \sim \pi(\theta)$, and trained with some version of gradient descent,

$$\Delta\theta \approx -\eta\, \nabla_\theta \mathcal{L}(\theta)\,. \tag{2.2}$$

A common idealization of this process, and the one this section is about, is *Bayesian learning*. It says that after training, the parameters should be regarded as drawn from the posterior distribution

$$p_\beta(\theta) = \frac{1}{Z}\, e^{-\beta\, \mathcal{L}(\theta)}\, \pi(\theta)\,, \qquad Z = \int d\theta\, e^{-\beta\, \mathcal{L}(\theta)}\, \pi(\theta)\,. \tag{2.3}$$

Here the initialization distribution $\pi(\theta)$ plays the role of the Bayesian *prior*, and $e^{-\beta\mathcal{L}(\theta)}$ is interpreted as the *likelihood* of the training data given the parameters θ (Section 2.2 explains why this is reasonable). The positive constant β, the *inverse temperature*, controls how strongly the data constrain the parameters. We will see in a moment that it can be read as a rough measure of how much training has been done.

An interpretation of β. It is convenient to write the loss as n times an average over samples,

$$\mathcal{L}(\theta) = n\, L_n(\theta)\,, \qquad L_n(\theta) := \frac{1}{n}\sum_{\mu=1}^{n} \ell\big(f_\theta(x^\mu), y^\mu\big)\,. \tag{2.4}$$

The *empirical loss* L_n stays finite as $n \to \infty$, converging to the *population loss* $L(\theta) := \mathbb{E}_{(x,y)}\, \ell(f_\theta(x), y)$, the expectation over the true data distribution. Thus for large n the posterior is

$$p_\beta(\theta) \;\propto\; e^{-\,n\beta\, L(\theta)}\, \pi(\theta)\,, \tag{2.5}$$

and the sample size n always accompanies the inverse temperature, in the combination $n\beta$. Increasing the number of samples is the same as lowering the temperature. If, moreover, samples are drawn fresh at each training step, then n is also a rough measure of training time. Altogether: *more data = more training = larger $n\beta$ = colder*. We will often write $e^{-\beta L}$ for short, but it is worth remembering that the effective β grows with n.

Bayes vs. gradient descent. We do not train networks by sampling from (2.3); we train them by gradient descent. Why, then, is the Bayesian model a useful idealization? There are two connections.

The first is heuristic: the analogy just described, $n \to \infty$ (more samples, longer training) $\leftrightarrow$ $\beta \to \infty$ (lower temperature). As we will see, sweeping a parameter like β can take a system through phase transitions, and the analogy predicts that training can do the same. It is only an analogy. In particular, when a system is driven *dynamically* through a first-order transition, the transition is delayed by *hysteresis* (Section 2.5), so timing predictions from the Bayesian model should be treated with care. A genuinely dynamical formalism for training, the MSRJD path integral, appears in Section 6.

The second connection is exact, and is what makes the Bayesian model testable in practice. Replace the batch noise of SGD by constant, isotropic Gaussian noise; the result is called *Langevin dynamics*:

discrete optimizer	continuous (S)DE
SGD : $\Delta\theta = -\eta(\nabla_\theta\mathcal{L} + \text{batch noise})$	$\dot\theta = -\eta\nabla_\theta\mathcal{L} + \frac{\eta}{\sqrt{B}}\Sigma(\theta)^{\frac{1}{2}}d\xi$
GD : $\Delta\theta = -\eta\,\nabla_\theta\mathcal{L}$	$\dot\theta = -\eta\nabla_\theta\mathcal{L}$
LD : $\Delta\theta = -\eta\,\nabla_\theta\mathcal{L} + \sqrt{2\eta T}\,\xi$	$\dot\theta = -\eta\nabla_\theta\mathcal{L} + \sqrt{2\eta T}\,d\xi$

(2.6)

Here ξ is Gaussian noise with mean 0 and covariance I, η is the learning rate, and T is a parameter called the *temperature* (or diffusion constant). Each discrete algorithm on the left has a continuous-time limit on the right, obtained by shrinking the step size and scaling up the number of steps: gradient descent becomes *gradient flow*, and Langevin dynamics becomes the *Langevin equation*, a stochastic differential equation. The key

fact is that if one starts from any reasonable distribution of parameters and evolves with the Langevin equation (for a sufficiently nice $\mathcal{L}$), the distribution converges at late times to the Gibbs distribution

$$p(\theta) = \frac{1}{Z}\, e^{-\beta \mathcal{L}(\theta)}\,, \qquad \beta = 1/T\,. \tag{2.7}$$

(See [88, Chapter 3] for a physics-focused discussion of this convergence.) A prior can be included by training on the modified loss $\mathcal{L}(\theta) - \beta^{-1} \log \pi(\theta)$, which reproduces (2.3). Thus one can *engineer* the Bayesian posterior by Langevin training, and this is how theoretical predictions derived from (2.3) are usually tested. Note that the Langevin route gives a different Bayes-to-gradient connection from the first: here training time $t \to \infty$ at fixed T gives the posterior at $\beta = 1/T$, while $T \to 0$ at fixed t gives gradient flow.

Remark

SGD itself is more complicated than Langevin dynamics. The covariance $\Sigma(\theta)$ of its effective noise depends on the position in the loss landscape, and at late times SGD need not converge to any stationary distribution. Despite many qualitative and empirical similarities between Bayesian learning and SGD/Adam training, the precise relation between them remains an open problem. We will compare the two repeatedly: for wide networks in Section 4 (gradient flow) versus Section 4.6 (Bayesian); for toy models of grokking in Section 5 [73]; and, for toy models of superposition [25] through the lens of singular learning theory, in [18].

2.2 Why the exponential of a loss is a likelihood

The Bayesian model treats $e^{-\beta \mathcal{L}(\theta)}$ as the probability of the training data given the parameters. For the loss functions actually used in deep learning, this is not a metaphor: the losses were *designed* as negative log-likelihoods, so that minimizing the loss is maximum-likelihood estimation. Two examples cover most of practice.

Mean squared error. Suppose the labels are the network's output plus Gaussian noise, $y = f_\theta(x) + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Then the probability density of a label given the input and the parameters is

$$p(y \,|\, x, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}}\, \exp\Big[-\frac{1}{2\sigma^2} \big(y - f_\theta(x)\big)^2 \Big]\,, \tag{2.8}$$

and for n independent samples,

$$\prod_{\mu=1}^{n} p(y^\mu \,|\, x^\mu, \theta) \;\propto\; e^{-\beta\, \mathcal{L}(\theta)}\,, \qquad \mathcal{L}(\theta) = \frac{1}{2} \sum_{\mu=1}^{n} \big(f_\theta(x^\mu) - y^\mu\big)^2\,, \quad \beta = \frac{1}{\sigma^2}\,. \tag{2.9}$$

The MSE loss is the negative log-likelihood of a Gaussian noise model, and the inverse temperature is the inverse noise variance. Noisier labels mean a hotter posterior, which constrains the parameters less.

Cross-entropy. For classification, the network outputs a probability distribution $p_\theta(y \mid x)$ over K classes, typically the softmax of its K output values. The cross-entropy loss is $\ell = -\log p_\theta(y \mid x)$, so that

$$\prod_{\mu=1}^{n} p_\theta(y^\mu \mid x^\mu) \;=\; e^{-\mathcal{L}(\theta)} \tag{2.10}$$

exactly, with $\beta = 1$: the exponential of the cross-entropy loss *is* the likelihood, with no noise model to choose. The same is true of the next-token loss of a language model, which is exactly the log-probability the model assigns to the training corpus.

The general pattern. Whenever a loss is a negative log-likelihood, $e^{-\mathcal{L}}$ is the likelihood and (2.3) at $\beta = 1$ is the textbook Bayesian posterior. A value $\beta \neq 1$ gives a *tempered* posterior. It arises naturally when the noise level is not known (as in (2.9)), and it is also a useful theoretical knob: as the interpretation of $n\beta$ above shows, temperature and sample size are interchangeable in the Bayesian model. There is also a useful limiting case. Some losses are hard constraints, $\ell = 0$ if the prediction is right and $\ell = \infty$ if it is wrong. Then $e^{-\ell}$ is a step function, the posterior is uniform on the parameters that fit every training sample, and no β appears at all. This is the *zero-temperature* limit, and it is the setting of the Ising perceptron in Section 2.6.

2.3 Partition functions, free energy, and cumulants

The Bayesian posterior (2.3) looks exactly like a *thermal* system in equilibrium with a heat bath, the basic object of statistical mechanics. The dictionary is:

Bayesian learning	**statistical mechanics**
parameters $\theta \in \mathbb{R}^P$	microscopic state of a system
	(*e.g.* the spins of P electrons)
loss $\mathcal{L}(\theta)$	energy $E(\theta)$, or Euclidean action $S(\theta)$
posterior $p(\theta) \propto e^{-\beta\mathcal{L}(\theta)}$	Boltzmann distribution at temperature $T = 1/\beta$
number of samples n, or training time	inverse temperature β
#parameters P	#degrees of freedom N

To include the prior $\pi(\theta)$, simply redefine the energy as $E(\theta) = \mathcal{L}(\theta) - \beta^{-1}\log\pi(\theta)$, so that $e^{-\beta E} = e^{-\beta\mathcal{L}}\pi$. In what follows we write $E(\theta)$ for whatever sits in the exponent, and N for the number of degrees of freedom, as physicists do; for a network, $N = P$.

The thermodynamic limit. Statistical mechanics is about systems with very many degrees of freedom, and one studies their behavior as $N \to \infty$: the *thermodynamic limit*. In this limit one distinguishes *extensive* quantities, which grow in proportion to N (the total energy of a gas), from *intensive* ones, which stay finite (the temperature, or the energy per particle). Both notions transfer directly to learning: with P parameters and n samples, the loss (2.1) is extensive in n and the empirical loss L_n is intensive. Many of the approximations in these notes are thermodynamic limits of some kind, sending the number of parameters, the width, or the number of samples to infinity.

Partition function and free energy. The normalization constant in (2.3),

$$Z(\beta) = \int d\theta\, e^{-\beta E(\theta)}\,, \tag{2.11}$$

is called the *partition function*. In Bayesian terms it is the marginal likelihood of the data. It depends on β and on any other hyperparameters of the problem. Its logarithm defines the *free energy*[1] and the free energy *density*,

$$F(\beta) := -\log Z(\beta)\,, \qquad \varphi := \frac{F}{N}\,. \tag{2.12}$$

There are three reasons to care about F rather than Z. First, F is extensive and φ intensive: the free energy density has a limit as $N \to \infty$, while Z itself is exponentially large or small. Second, "F is minimized": at large N the integral (2.11) is dominated by the regions of parameter space with the smallest local free energy, so the system's macroscopic state is found by minimizing F. Section 2.4 makes this precise. Third, derivatives of F compute expectation values, as follows.

Sources and cumulants. Suppose we care about the expectation values of some *observables* $\mathcal{O}_i(\theta)$, functions of the parameters. Add *source terms* J^i to the exponent,

$$Z(\beta, J) = \int d\theta\; e^{-\beta E(\theta) + \sum_i J^i\, \mathcal{O}_i(\theta)}\,, \qquad F(\beta, J) = -\log Z(\beta, J)\,. \tag{2.13}$$

Differentiating under the integral,

$$\mathbb{E}(\mathcal{O}_i) = -\frac{\partial F}{\partial J^i}\bigg|_{J=0}\,, \qquad \mathrm{Cov}(\mathcal{O}_i, \mathcal{O}_j) = -\frac{\partial^2 F}{\partial J^i \partial J^j}\bigg|_{J=0}\,, \tag{2.14}$$

and in general the p-th derivative is the p-th *cumulant*,

$$\kappa_p(\mathcal{O}_{i_1}, \ldots, \mathcal{O}_{i_p}) = -\frac{\partial^p F}{\partial J^{i_1} \cdots \partial J^{i_p}}\bigg|_{J=0}\,: \tag{2.15}$$

the mean, the covariance, the third central moment, and from $p = 4$ on, combinations of moments that vanish for Gaussian variables. The sourced free energy is the generating function of cumulants. Cumulants will return in field-theory clothing in Section 3.3, where they measure how far a network at initialization is from a Gaussian process.

If the only observable we care about is the energy itself, no source is needed: β already couples to E, and

$$\mathbb{E}(E) = \frac{\partial F}{\partial \beta}\,, \qquad \mathrm{Var}(E) = -\frac{\partial^2 F}{\partial \beta^2}\,, \qquad \kappa_3(E) = \frac{\partial^3 F}{\partial \beta^3}\,, \ \ldots \tag{2.16}$$

We will meet these derivatives again shortly: discontinuities in them are how phase transitions are defined.

[1] A physicist's free energy is $-\beta^{-1} \log Z$; we absorb the factor of β throughout, so that F is dimensionless.

Exercise 2.1 (Partition functions, sources, and cumulants). (★) Take a Boltzmann distribution $p_\beta(\theta) \propto e^{-\beta E(\theta)}$, an observable $\mathcal{O}(\theta)$, and the sourced partition function $Z(\beta, J) = \int d\theta\, e^{-\beta E(\theta) + J\mathcal{O}(\theta)}$, with $F = -\log Z$. All expectations are in p_β, that is, at $J = 0$.

(a) Show that $\mathbb{E}(\mathcal{O}) = -\partial_J F|_{J=0}$ and $\mathrm{Var}(\mathcal{O}) = -\partial_J^2 F|_{J=0}$.

(b) The third cumulant is defined by continuing the pattern, $\kappa_3(\mathcal{O}) := -\partial_J^3 F|_{J=0}$. Compute it in terms of the moments $\mathbb{E}(\mathcal{O}^3)$, $\mathbb{E}(\mathcal{O}^2)$, $\mathbb{E}(\mathcal{O})$, and show that it equals the third *centered* moment $\mathbb{E}[(\mathcal{O} - \mathbb{E}\mathcal{O})^3]$.

(c) Show that for a Gaussian $\mathcal{O}$, $\log Z(J)$ is exactly quadratic in J, so that all cumulants with $p \geq 3$ vanish. Compute κ_4 in terms of centered moments and check that it is *not* the fourth centered moment. (Moral: cumulants measure the failure of higher moments to be determined by lower ones in the Gaussian pattern.)

(d) Verify (2.16).

Quenched averages. In a learning problem there is a second source of randomness besides the posterior: the training set itself. Quantities like the free energy depend on which n samples were drawn, and at large n one expects them to concentrate around their average over draws. Physicists call randomness that is fixed before the system equilibrates *quenched disorder*, by analogy with impurities frozen into an alloy when it is quenched in water; the training data are quenched disorder for the posterior. The *quenched free energy* is

$$\mathbb{E}_{\text{data}}\big[F\big] = -\,\mathbb{E}_{\text{data}}\big[\log Z\big]\,, \tag{2.17}$$

and it is the free energy, not the partition function, that should be averaged, because it is the free energy that is extensive and whose derivatives give expectation values. Averaging $\log Z$ is usually hard, and the *annealed approximation* swaps the order,

$$\mathbb{E}_{\text{data}} \log Z \;\approx\; \log \mathbb{E}_{\text{data}} Z\,, \tag{2.18}$$

which is an upper bound by Jensen's inequality and often a good approximation. We will use it in Section 2.6. The exact treatment goes through the *replica method*, sketched in Appendix A; the classic reference is [26].

2.4 A physics example: the Ising model

The analogy with thermal physics opens up a bag of well-developed approximation methods. We illustrate two of them, the *mean-field approximation* and the *saddle-point approximation at large N*, on the most famous model in statistical mechanics. For the physics of this example, well beyond what we cover here, see [89, Chapter 1].

The model. Let Λ be a graph with N vertices and undirected edges $\langle ij \rangle$; typically Λ is a regular lattice in D dimensions, and every vertex has the same number d of

neighbors ($d = 2D$ for a hypercubic lattice). At each vertex i sits a *spin* $s_i \in \{\pm 1\}$. A state of the system is a choice of all N spins, so there are 2^N states, and the energy of a state is

$$E(s) = -J \sum_{\text{edges } \langle ij \rangle} s_i s_j \;-\; B \sum_{i=1}^{N} s_i \,. \tag{2.19}$$

The *coupling* $J > 0$ rewards neighboring spins for aligning with each other, and the *external magnetic field* B rewards spins for aligning with it. At inverse temperature β the Boltzmann distribution is $p_\beta(s) = e^{-\beta E(s)}/Z$ with $Z(\beta, J, B) = \sum_{s \in \{\pm 1\}^N} e^{-\beta E(s)}$. Dimensionally, only the combinations βJ and βB matter.

The simplest macroscopic quantity describing a state is its *magnetization*, the average spin,

$$m(s) = \frac{1}{N} \sum_{i=1}^{N} s_i \;\in\; [-1, 1] \,. \tag{2.20}$$

It is our first example of an *order parameter*: a single number that summarizes an N-dimensional microscopic state for the purpose of the questions we want to ask. The question here is whether the spins align ($|m|$ close to 1, an *ordered* phase) or not (m near 0, a *disordered* phase), and how this depends on β, J, and B.

Mean-field approximation. We would like to push the distribution $p_\beta(s)$ over 2^N states forward to a distribution over the single variable m. The interaction term stands in the way, since it couples each spin to its particular neighbors. The mean-field approximation replaces the neighbors of each spin by their average: suppose that each spin feels only the *average* field of all the others, so that in the neighbor sum we may replace each s_j by m,

$$\sum_{\langle ij \rangle} s_i s_j = \frac{1}{2} \sum_i s_i \underbrace{\sum_{j \sim i} s_j}_{\approx d\, m} \;\approx\; \frac{d\, m}{2} \sum_i s_i = \frac{N d}{2}\, m^2 \,. \tag{2.21}$$

(The factor $\frac{1}{2}$ compensates for counting each edge twice.) The energy becomes a function of m alone,

$$E(s) \;\approx\; -N\Big(\frac{dJ}{2}\, m^2 + B\, m\Big) \,. \tag{2.22}$$

This is an uncontrolled approximation for a lattice in low dimension, where a spin's few neighbors fluctuate along with it, but it turns out to be qualitatively right for $D \geq 2$ and quantitatively right for $D \geq 4$ [89, Chapter 1]. It becomes exact for a model in which every spin couples weakly to every other; that model, the Curie–Weiss magnet, returns in Section 5.1 as the prototype of mean-field limits of neural networks.

Entropy, and the reduction to one variable. With the energy depending only on m, the sum over states can be organized by the value of m:

$$Z \;\approx\; \sum_m \Omega(m)\, e^{N\beta\left(\frac{dJ}{2} m^2 + Bm\right)} \,, \qquad \Omega(m) := \#\{s : m(s) = m\} = \binom{N}{\frac{1+m}{2} N} \,, \tag{2.23}$$

since a state with magnetization m has exactly $\frac{1+m}{2}N$ spins up. Stirling's formula gives

$$\frac{1}{N}\log\Omega(m) \xrightarrow{N\to\infty} h(m)\,, \qquad h(m) := \log 2 - \frac{1-m}{2}\log(1-m) - \frac{1+m}{2}\log(1+m)\,, \tag{2.24}$$

the *entropy* per site at magnetization m: the log of the number of microscopic states compatible with the macroscopic value m. It is largest at $m=0$, where $h=\log 2$ and almost all of the 2^N states live, and it vanishes at $m=\pm 1$, where exactly one state lives. Replacing the sum over m by an integral (the spacing is $2/N$), the partition function becomes a one-dimensional integral,

$$Z \;\approx\; \int_{-1}^{1} dm\, e^{-N\beta\, s(m)}\,, \qquad s(m) := -\frac{dJ}{2}m^2 - Bm - \beta^{-1}h(m)\,. \tag{2.25}$$

The function $s(m)$ is called the effective *action*, or the free-energy density at fixed m: energy minus temperature times entropy, per site. *We have reduced the problem from N variables to one.* This is the general shape of every calculation in this section: push the microscopic distribution forward to an order parameter, at the price of an entropy term that counts how many microstates sit at each value.

Saddle points. At large N the integrand $e^{-N\beta s(m)}$ is exponentially peaked at the *minima* of s, and the integral is dominated by a window of width $\sim 1/\sqrt{N}$ around the lowest one. This is the saddle-point approximation (also called Laplace's method), and it gives

$$\varphi(\beta,J,B) = -\frac{1}{N}\log Z \xrightarrow{N\to\infty} \beta\,\min_m s(m)\,. \tag{2.26}$$

The free energy is minimized, in exactly the sense promised in Section 2.3. The minimizer $\bar m$ is the magnetization the system actually displays. Setting $\partial s/\partial m = 0$ and using $h'(m) = -\tanh^{-1}(m)$,

$$dJ\,\bar m + B = \beta^{-1}\tanh^{-1}(\bar m)\,, \qquad \text{that is,} \qquad \boxed{\bar m = \tanh\big(\beta\,(dJ\,\bar m + B)\big)}\,. \tag{2.27}$$

This transcendental equation is best solved graphically (Figure 1). Depending on β, J, and B it has one solution, or three; when there are three, the outer two are minima of s and the middle one is a maximum.

Self-consistency (an aside). There is another route to (2.27), which is the form in which mean-field theory usually appears and which we will use in Section 5. Look at the effective field felt by a single spin s_i:

$$B_i^{\text{eff}} = -\frac{\partial E}{\partial s_i} = J\sum_{j\sim i} s_j + B \;\approx\; dJ\,m + B\,. \tag{2.28}$$

A single spin in this field has partition function $Z_i = \sum_{s_i=\pm1} e^{\beta B^{\text{eff}} s_i} = 2\cosh\beta(dJm+B)$ and mean $\mathbb{E}(s_i) = \beta^{-1}\partial_B \log Z_i = \tanh\beta(dJm+B)$. If the approximation is to be consistent, the average that comes out must equal the average we put in, $m = \mathbb{E}(s_i)$, and that is (2.27). Solve the one-body problem in the average environment, then demand that the average be reproduced: this is the prototype of every mean-field theory.

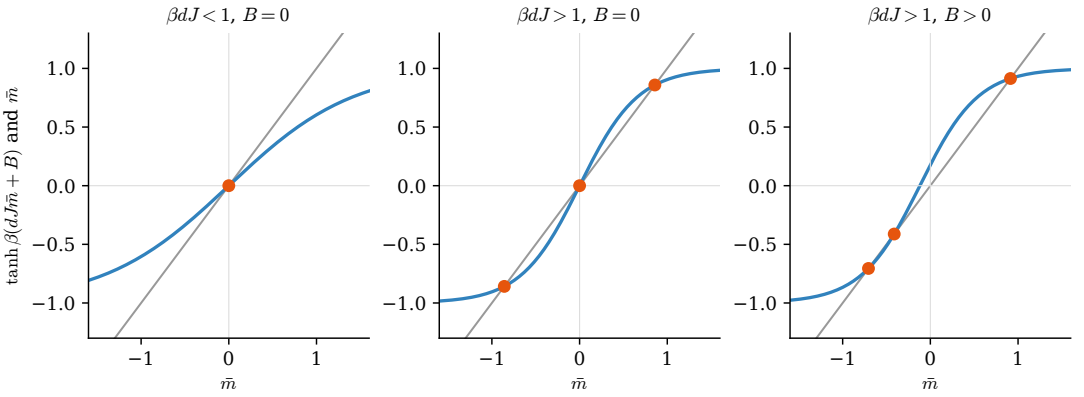


Figure 1: Graphical solution of the mean-field equation (2.27): intersections of $\tanh \beta(dJ\bar{m} + B)$ with the diagonal. Left: high temperature, one solution at $\bar{m} = 0$. Middle: low temperature at $B = 0$, three solutions, of which $\bar{m} = 0$ is a maximum of s and $\pm\bar{m}$ are degenerate minima. Right: low temperature with $B > 0$, three solutions, with the positive one the global minimum.

2.5 Phases and phase transitions

Now we analyze the structure of the minima of $s(m)$, which control where the probability concentrates, as β, J, and B are varied. Taylor-expanding the entropy, $h(m) = \log 2 - \frac{m^2}{2} - \frac{m^4}{12} - \dots$, the action is

$$s(m) \;\approx\; \text{const} - B\,m + \frac{1}{2}\big(\beta^{-1} - dJ\big)\,m^2 + \frac{\beta^{-1}}{12}\,m^4 + \dots \tag{2.29}$$

This expansion of a free energy in powers of an order parameter is called a *Landau theory*, and the qualitative analysis below depends only on its form, not on the microscopic model behind it.

The ordered phase and a first-order transition. Take $dJ > \beta^{-1}$: the coupling beats the temperature, the quadratic coefficient in (2.29) is negative, and $s(m)$ is a double well (Figure 2). The system wants to be ordered, with neighboring spins aligned; but aligned which way? The field decides. For $B < 0$ the global minimum is near $m = -1$, for $B > 0$ near $m = +1$, and at $B = 0$ the two minima are exactly degenerate. As B is swept through $B_c = 0$, the magnetization the system displays jumps from $\bar{m} \approx -1$ to $\bar{m} \approx +1$. This is a *first-order phase transition*.

The structure of this transition is universal, in the sense that it follows from two isolated minima crossing and not from any detail of the model. Consider the expected magnetization, which by (2.14) is a derivative of the free energy with respect to the field:

$$\mathbb{E}(m) = \frac{1}{N}\sum_i \mathbb{E}(s_i) = -\frac{1}{\beta}\frac{\partial \varphi}{\partial B}\,. \tag{2.30}$$

At finite N, the two minima of s are two competing *phases*, and $\mathbb{E}(m)$ is a probability-weighted average of their magnetizations. As $N \to \infty$ the saddle-point approximation

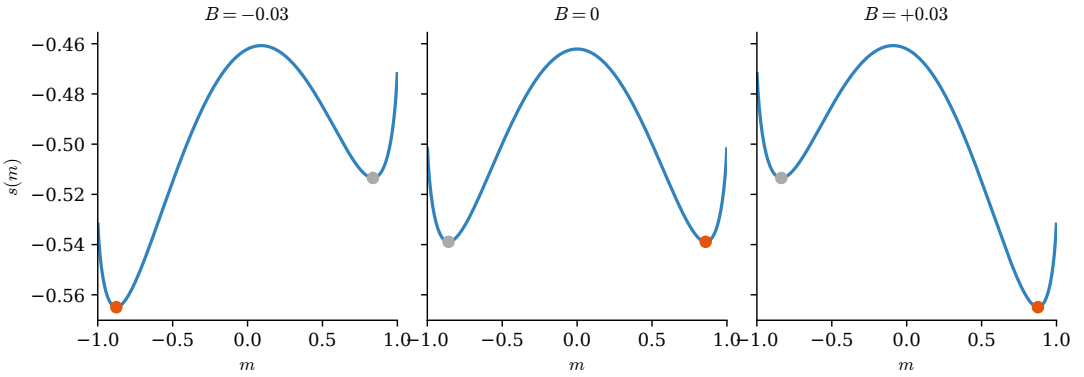


Figure 2: The action $s(m)$ of (2.25) in the ordered phase ($\beta dJ = 1.5$), for a negative, zero, and positive external field. The two minima tilt with B; the global minimum (orange) switches sides at $B = 0$. The suppressed minimum (grey) survives as a metastable state.

puts all the weight on the lower minimum, so $\mathbb{E}(m)$ develops a discontinuity at B_c, and φ itself develops a kink (Figure 3, left). *A discontinuity in a first derivative of the free-energy density, in the thermodynamic limit, is the definition of a first-order transition.* Its second derivative, the *susceptibility*

$$\chi := \frac{\partial\, \mathbb{E}(m)}{\partial B} = -\frac{1}{\beta}\frac{\partial^2 \varphi}{\partial B^2} = N\beta\, \mathrm{Var}(m)\,, \tag{2.31}$$

is a peak whose height grows like N and whose width shrinks like $1/N$: a nascent delta function (Figure 3, right). Genuine non-analyticity can occur only at $N = \infty$: at finite N the partition function is a finite sum of analytic functions of B, so everything is smooth, and what one sees is a rapid *crossover* that sharpens as N grows. Watching the peak in χ grow with N, rather than saturate, is the standard numerical diagnostic that separates a true transition from a crossover. Exercise 2.2 works out the shapes of these curves for a general first-order transition; they are worth remembering whenever "emergent capabilities" in models of growing size are under discussion (Section 7.6).

Metastability and hysteresis. The suppressed minimum in Figure 2 does not disappear when it stops being the global one. A system that starts there and evolves by local dynamics (flipping one spin at a time) must climb over the barrier to reach the true minimum, which takes a time exponential in N. In practice the system stays in the *metastable* phase until the field is pushed well past B_c, and the transition observed dynamically *lags* the thermodynamic one. This is *hysteresis*, familiar from real magnets. It is the reason the "$\beta \leftrightarrow$ training time" analogy of Section 2.1 must be handled with care at first-order transitions, and we will meet it again in the guise of delayed generalization (grokking) in Section 5.7.

A second-order transition. An *n-th order* phase transition is a discontinuity in the n-th derivative of the free-energy density as $N \to \infty$. The Ising model has a second-order transition too, at fixed $B = 0$, as the temperature $T = \beta^{-1}$ crosses $T_c = dJ$

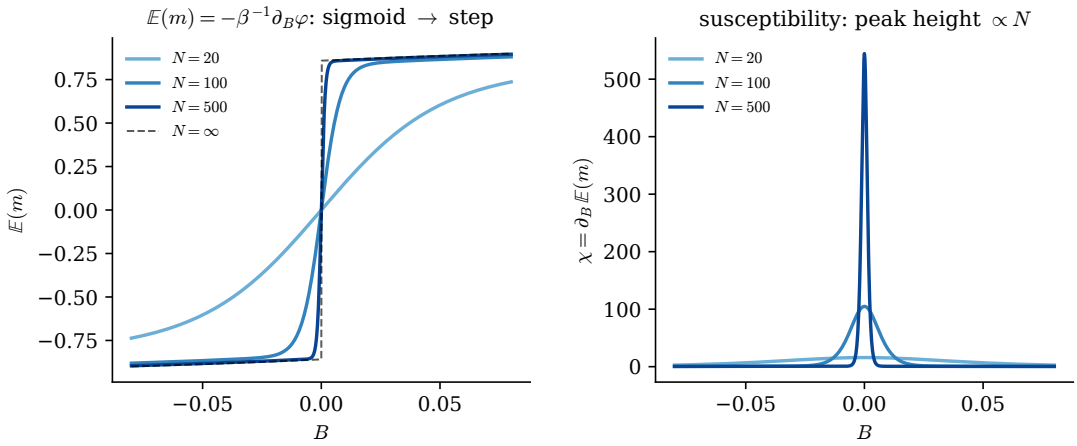


Figure 3: Fingerprints of a first-order transition, in the mean-field Ising model at $\beta dJ = 1.5$, computed from the one-dimensional integral (2.25) at finite N. Left: the expected magnetization as a function of the field is a sigmoid of width $\propto 1/N$, sharpening into a step. Right: the susceptibility $\chi = \partial_B \mathbb{E}(m)$ is a peak of height $\propto N$ and width $\propto 1/N$.

(Figure 4). Here nothing crosses anything: the single basin at $m = 0$ flattens and splits into two symmetric ones. The quadratic coefficient in (2.29) changes sign at T_c, and the minima sit at

$$\bar{m}^2 = 3\,(\beta dJ - 1) \;\approx\; 3\left(1 - \frac{T}{T_c}\right) \qquad (T \lesssim T_c)\,, \tag{2.32}$$

so the order parameter $\bar{m} \propto (T_c - T)^{1/2}$ moves *continuously* but not smoothly; φ and $\partial_T \varphi$ are continuous, and the discontinuity sits in $\partial_T^2 \varphi$ (the heat capacity). The point $(T, B) = (T_c, 0)$ is a *critical point*. Below it the two ordered states $\pm\bar{m}$ are exact mirror images, and the system must pick one: this is *spontaneous symmetry breaking*, the $\mathbb{Z}_2$ symmetry $s \to -s$ of the energy at $B = 0$ being broken by the state. The exponent $\frac{1}{2}$ in (2.32) is an example of a *critical exponent*. Mean-field theory predicts it in every dimension, and it is wrong in low dimension: the exact value is $\frac{1}{8}$ in $D = 2$ (Onsager), and there is no transition at all in $D = 1$. Mean-field exponents become exact for $D \geq 4$. Understanding why is the subject of the renormalization group, for which see [89].

Back to learning. In the Bayesian model of Section 2.1, the natural control parameter is $n\beta$, the amount of data or training. The Ising analogue is sweeping β at fixed $B = 0$, which sees the second-order transition. First-order transitions in learning arise when two qualitatively different families of solutions compete, one entropically favored and one favored by the data; the Ising perceptron of Section 2.6 is the cleanest example. Before turning to it, the following exercise establishes what *every* first-order transition looks like at finite N.

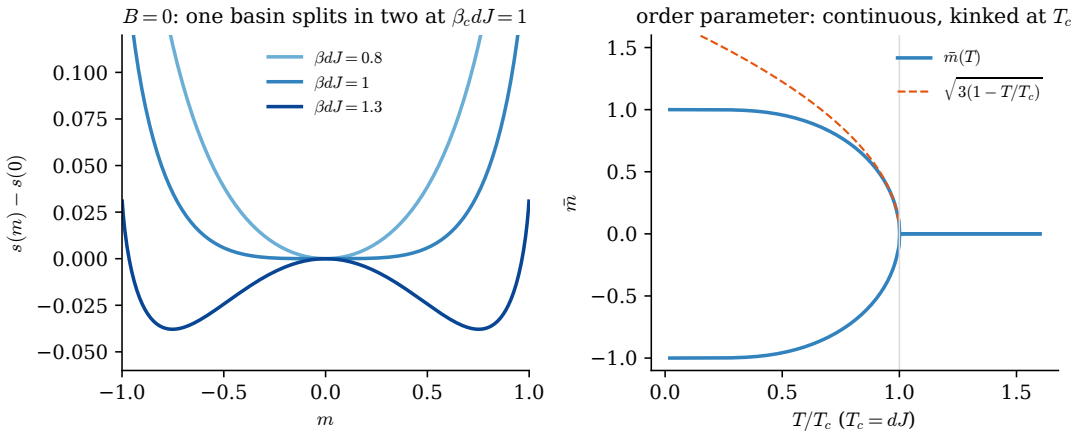


Figure 4: The second-order transition of the mean-field Ising model at $B=0$. Left: the action $s(m)$ for temperatures above, at, and below $T_c = dJ$; a single basin flattens and splits into two symmetric ones. Right: the resulting magnetization $\bar{m}(T)$, continuous at T_c but with the square-root singularity (2.32) (dashed).

Exercise 2.2 (Anatomy of a first-order transition). **(★)** Consider any system whose partition function has the form $Z(B) = \int d\theta\, e^{-NE(\theta,B)}$, depending on *some* hyperparameter B. Here B could be the inverse temperature, an external field, or the load α of Section 2.6; we have absorbed β into E to keep the setup general. Suppose that $E(\theta, B)$ has exactly two isolated minima, at θ_1 and θ_2. Then at large N the integral is dominated by two competing phases. Letting U_1, U_2 be small neighborhoods of θ_1, θ_2,

$$Z(B) \;\approx\; \int_{U_1} d\theta\, e^{-NE(\theta,B)} + \int_{U_2} d\theta\, e^{-NE(\theta,B)} \;=\; e^{-N\varphi_1(B)} + e^{-N\varphi_2(B)}\,, \tag{2.33}$$

with $\varphi_i(B) = -\frac{1}{N}\log\int_{U_i} d\theta\, e^{-NE(\theta,B)} = E(\theta_i, B) + o(1)$ the free-energy densities of the two basins. Suppose further that as B crosses a critical value B_c the two minimum values cross: $\varphi_1(B_c) = \varphi_2(B_c)$ with $\varphi_1'(B_c) \neq \varphi_2'(B_c)$, where a prime denotes ∂_B.

(a) *Occupation.* Show that the probability of finding the system in basin i is

$$p_i(B) = \frac{e^{-N\varphi_i(B)}}{e^{-N\varphi_1(B)} + e^{-N\varphi_2(B)}}\,, \tag{2.34}$$

and that, expanding to first order in $\Delta B := B - B_c$,

$$p_2(B) \approx \frac{1}{1 + e^{-N\,\Delta\varphi'\,\Delta B}}\,, \qquad \Delta\varphi' := \varphi_1'(B_c) - \varphi_2'(B_c)\,, \tag{2.35}$$

a logistic function of $N\,\Delta\varphi'\,\Delta B$. Over what window of B does the system switch allegiance?

(b) *First derivative.* Show from (2.33) that

$$\frac{\partial \varphi}{\partial B} \;\approx\; p_1(B)\,\varphi_1'(B_c) + p_2(B)\,\varphi_2'(B_c)\,, \tag{2.36}$$

the probability-weighted average of the branch slopes. Sketch it at finite N and at $N=\infty$: a sigmoid of height $\Delta\varphi'$ sharpening into a jump. The kink in φ itself is the defining discontinuity of a first-order transition. Show also, directly from the integral, that $\partial_B \varphi = \mathbb{E}\big[\partial_B E(\theta, B)\big]$: the observable that jumps is the one conjugate to the control parameter. When $B=\beta$ it is the energy, and the jump is the *latent heat*; for the Ising model with B the field, it is the magnetization.

(c) *Second derivative.* Using $\partial_B p_1 = -\partial_B p_2 = -N p_1 p_2\,\Delta\varphi'$ (verify this), show that

$$-\frac{\partial^2 \varphi}{\partial B^2} \;=\; N\,p_1 p_2\,(\Delta\varphi')^2 \;-\; p_1\varphi_1'' - p_2\varphi_2''\,. \tag{2.37}$$

Sketch this curve. Near B_c, where $p_1 p_2 \approx \tfrac{1}{4}$, the first term is a peak of height $\sim N(\Delta\varphi')^2/4$ and width $\sim 1/(N\Delta\varphi')$, which converges to a delta function as $N\to\infty$. Show also that when $\mathcal{O}(\theta) = \partial_B E(\theta, B)$ does not depend on B, $-\partial_B^2 \varphi = N\,\mathrm{Var}(\mathcal{O}) \geq 0$. (For $B=\beta$ this is the heat capacity; for the Ising field it is the susceptibility.)

(d) *Plot it.* Take the minimal model $\varphi_1(B) = a\,(B - B_c)$, $\varphi_2 = 0$ and plot φ, $\partial_B \varphi$, and $-\partial_B^2 \varphi$ for a few values of N. (Use a numerically stable $\log(e^x + e^y)$.) Watching the peak grow with N rather than saturate is the standard diagnostic separating a true transition from a smooth crossover.

2.6 The Ising perceptron: a first-order transition in learning

We close the physics part of this section with a complete statistical-mechanics analysis of a learning problem, built to run in parallel with the Ising magnet. It is the simplest model we know in which *more data* produces a genuine phase transition to *perfect generalization*, and it goes back to Györgyi [35]; see [26, Chapter 7]. It is presented as a walk-through exercise; solutions are in Appendix C, but try each part before reading them.

Setup. A *teacher* is a fixed vector of binary weights $\theta^* \in \{\pm 1\}^N$. Inputs are Gaussian, $x \sim \mathcal{N}(0, I_N)$, and the teacher labels them by

$$y(x) = \mathrm{sign}(\theta^* \cdot x)\,. \tag{2.38}$$

A *student* is another binary vector $\theta \in \{\pm 1\}^N$, guessing labels the same way, $f_\theta(x) = \mathrm{sign}(\theta \cdot x)$. This model is called the *Ising perceptron*, because its weights take the values of Ising spins. We observe n examples (x^μ, y^μ) with $y^\mu = y(x^\mu)$, and do Bayesian learning at zero temperature with a uniform prior, in the sense of Section 2.2: the per-sample loss is 0 if the student classifies the example correctly and ∞ otherwise, so

the posterior is uniform on the students that classify *every* example correctly. Since teacher and student agree on x exactly when $y(x)\,(\theta\cdot x)>0$,

$$p(\theta)\;\propto\;\prod_{\mu=1}^{n}\Theta\big(y^{\mu}\,(\theta\cdot x^{\mu})\big)\,,\qquad Z=\frac{1}{2^{N}}\sum_{\theta\in\{\pm1\}^{N}}\prod_{\mu=1}^{n}\Theta\big(y^{\mu}\,(\theta\cdot x^{\mu})\big)\,, \tag{2.39}$$

with $\Theta(u)=1$ for $u>0$ and 0 otherwise. With the factor $1/2^{N}$, Z is the fraction of all students consistent with the data. Both N and n will be taken large, at fixed ratio: the *load*

$$\alpha:=\frac{n}{N}\,, \tag{2.40}$$

the number of examples per parameter, is the control parameter of the problem.

Exercise 2.3 (The Ising perceptron). (★)

(a) *The order parameter is a magnetization.* Define the *overlap* $R(\theta)=\frac{1}{N}\theta\cdot\theta^{*}\in[-1,1]$ and the alignment variables $s_i:=\theta_i\theta_i^{*}\in\{\pm1\}$. Show that $R=\frac{1}{N}\sum_i s_i$: the overlap *is* the magnetization of the spins s_i. Learning the teacher means magnetizing them to $R=1$.

(b) *Generalization error.* Show that a student at overlap R misclassifies a fresh Gaussian input with probability

$$\varepsilon(R)=\frac{\arccos R}{\pi}\,. \tag{2.41}$$

[*Hint:* project x onto the plane spanned by θ and θ^{*}; the projection is an isotropic two-dimensional Gaussian, and the two decision boundaries are diameters meeting at angle $\arccos R$. When do the signs disagree?]

(c) *Entropy: the same h as the magnet.* Show that the number of students at overlap R is $\binom{N}{\frac{1+R}{2}N}$, so that by Stirling's formula

$$\begin{aligned}\frac{1}{N}\log\#\{\theta:R(\theta)=R\}\;\to\;h(R)=\log 2&-\frac{1-R}{2}\log(1-R)\\&-\frac{1+R}{2}\log(1+R)\,,\end{aligned} \tag{2.42}$$

identical to the magnet's entropy (2.24) with $m\to R$.

(d) *Push forward to one variable.* Average Z over datasets. The examples are independent, and each is classified correctly by a student at overlap R with probability $1-\varepsilon(R)$, so

$$\mathbb{E}_x Z=\frac{1}{2^{N}}\sum_{\theta}\big(1-\varepsilon(R(\theta))\big)^{n}\,. \tag{2.43}$$

Trade the sum over θ for an integral over R using (c), and show that at fixed load α and large N,

$$\mathbb{E}_x Z \;\approx\; \int_{-1}^{1} dR\, e^{-N\, s(R)}\,, \qquad s(R) = \log 2 - h(R) - \alpha \log\big(1 - \varepsilon(R)\big)\,. \tag{2.44}$$

This is the exact analogue of the magnet's (2.25): an entropy term counting states, and a "field" term rewarding alignment. The aligning field is now the *data*, and its strength is the load α. (Averaging Z rather than $\log Z$ is the annealed approximation (2.18). It preserves all the qualitative structure here and shifts the numbers only slightly; Appendix A explains how to do the exact calculation.)

(e) Plot $s(R)$ for several values of α and locate its critical points. Do you expect a phase transition? At which α?

(f) *The teacher basin.* $R = 1$ is the perfectly aligned state, where the student is the teacher. Show that $s(1) = \log 2$ for *every* α (zero entropy, but zero data cost), and that $R = 1$ is always a local minimum: writing $\delta = 1 - R$, deviating from the teacher gains entropy $\sim \frac{\delta}{2}\log(1/\delta)$ but costs data-fit $\alpha\varepsilon \sim \alpha\sqrt{2\delta}/\pi$ (expand arccos), and $\sqrt{\delta} \gg \delta\log(1/\delta)$. This differs from the magnet, where the aligned state sat at the bottom of a smooth well and was not always a minimum.

(g) *The interior basin, the crossing, and the transition.* At $\alpha = 0$, s is minimized at $R = 0$ with $s(0) = 0 < s(1)$: the disordered cloud of typical students dominates. As α grows, this interior minimum $R^*(\alpha)$ drifts toward 1 and its value rises. Numerically, find the load α_c at which the two basins exchange dominance, $s(R^*(\alpha_c)) = s(1)$. You should find $\alpha_c \approx 1.45$, with the interior minimum still at $R^* \approx 0.70$, that is, generalization error $\varepsilon^* \approx 0.25$. Conclude that at α_c the typical student jumps from 25% error to *zero* error: a first-order transition to perfect generalization. Then connect to Exercise 2.2 with α as the control parameter: what do $\partial_\alpha\varphi$ and $-\partial_\alpha^2\varphi$ look like at finite N, and what plays the role of the latent heat? Finally, find the load at which the interior minimum disappears altogether, and explain what happens in between in the light of the metastability discussion of Section 2.5.

(h) *The dictionary.* Complete the table:

Ising magnet	**Ising perceptron**
spins $s_i \in \{\pm 1\}$	alignments $s_i = \theta_i \theta_i^* \in \{\pm 1\}$
magnetization m	overlap R
entropy $h(m)$	entropy $h(R)$ *(the same function)*
aligning field B	?
knob swept across the transition	?
ordered phase $\mathbb{E}(m) \approx \pm 1$	?
jump in $\mathbb{E}(m)$ as a function of B	?

Also articulate one structural *difference*: in the magnet the two competing minima are symmetry partners; what are they here, and which ingredient of (c) made a sharp transition possible at all? (What would change for spherical weights $\theta \in \mathbb{R}^N$, $|\theta|^2 = N$, where the slice at overlap R has volume $\propto (1-R^2)^{N/2}$? Appendix A.3 works this case out.)

2.7 Comparison with QFT

This short subsection is a translation guide, aimed mainly at readers arriving from high-energy or condensed-matter physics — if the words "action" and "path integral" mean more to you than "posterior" and "marginal likelihood," read on; everyone else may safely skip ahead.

Nearly everything in these notes lives, in the first instance, in *Euclidean equilibrium statistical mechanics*: probability distributions of Boltzmann–Gibbs form $P \propto e^{-\beta E}$, partition functions, free energies. For a field theorist, a partition function with sources,

$$Z[J] = \int \mathcal{D}\phi \; e^{-S[\phi] + \int J\,\phi}\,, \tag{2.45}$$

is a moment generating functional; its logarithm generates connected correlation functions (cumulants); the action S encodes the microscopic rules of the system, and a quadratic action means a free (Gaussian) theory, with everything else treated by perturbing around it. In these notes the role of the field ϕ is played variously by the weights θ of a network, by an order parameter such as the magnetization m or the overlap R of Sections 2.4 and 2.6, or — most usefully — by the network function $f(x)$ itself, with the input x playing the role of the spacetime coordinate. Functional integrals over the network function are developed carefully in Sections 3.3 and 3.5, and the reader coming from QFT will find many familiar ingredients there dressed up as learning theory: Wick's theorem, sources, cumulants, effective actions.

If your instincts were trained on Lorentzian signature, picture everything here as already Wick-rotated. There is no $i/\hbar$: the weights e^{-S} are real and positive, as after the standard continuation

$$Z_{\text{QFT}} = \int \mathcal{D}\phi \; e^{\frac{i}{\hbar} S[\phi]} \quad \xrightarrow{\tau = it} \quad Z_{\text{E}} = \int \mathcal{D}\phi \; e^{-S_E[\phi]/\hbar}\,. \tag{2.46}$$

The role of $\hbar$ — the parameter controlling the size of fluctuations around saddle points — is played here by temperature, or by $1/N$ at large width, or by $1/n$ at large sample size.

When we eventually study the *dynamics* of training as a field theory in Section 6, a genuinely real-time formalism does appear — a path integral over trajectories with a causal structure and a response field. That's the closest object in these notes to a Lorentzian field theory, and is precisely the Martin–Siggia–Rose construction of non-equilibrium statistical mechanics.

For readers who would like physics references alongside the learning-theory ones: David Tong's lecture notes [90, 89, 88, 87] are a friendly introduction to statistical physics, statistical field theory, kinetic theory, and QFT respectively; Lancaster and Blundell [50] is a gentle conceptual route into QFT; Altland and Simons [2] bridges toward condensed-matter field theory (and contains the Langevin/MSRJD material used in Section 6); Pathria [64] is a standard graduate statistical mechanics text; Goldenfeld [33] is the classic on phase transitions and the renormalization group; and Zinn-Justin [99] is the encyclopedic reference for the functional formalism.

3 Large width at initialization: the Gaussian process limit

The next few sections are all about different limits in neural networks, in which some parameter(s) become large. Such limits are where physical/statistical analyses really shine. Studying them teaches some basic lessons about how we should (and should not) initialize networks, and how to choose other hyperparameters. It also leads to analytically solvable models that display many properties we see in practice: lazy vs. rich learning, double descent, phase transitions, scaling laws for optimal learning. For further pedagogical references on much of this material, we recommend [71, 84, 69] and the online lectures [37].

We'll begin in this section with a classic result [62, 52, 95, 36] about increasing the width of a network at fixed depth and training-sample size.

Modern neural networks are more sophisticated than the simple MLP's discussed here, but they share many of the same features. Layer norms solve the problem of trainability at large depth, but the analysis of rich/lazy regimes, feature learning, and hyperparameter scaling is still relevant. See *e.g.* the discussion of μP in Section 5.5.

3.1 Notation

We'll use the following notation for neural networks and training data.

- $\theta \in \mathbb{R}^P$ trainable network parameters (weights, biases)
- $(x^\mu, y^\mu)_{\mu=1}^n$ training data samples, with $x^\mu \in \mathbb{R}^d$ and $y^\mu \in \mathbb{R}$ (unless otherwise stated)
- $f_\theta : \mathbb{R}^d \to \mathbb{R}$ network function

For MLP's in particular (also called FCN's = fully connected networks)

- $\{z_i^{(\ell)}\}_{i=1}^{N_\ell}$ pre-activations in layer $\ell \in \{0, ..., L+1\}$
- L = number of hidden layers (depth); N_ℓ = width of ℓ'th layer; $N_0 = d$ and $N_{L+1} = 1$. We will often take constant hidden depth, $N_1 = ... = N_L = N$.
- $\phi : \mathbb{R} \to \mathbb{R}$ activation function (assumed the same in all layers except the first)
- network recursion $z_i^{(\ell)} = \begin{cases} \frac{1}{\sqrt{N_{\ell-1}}} \sum_{j=1}^{N_{\ell-1}} w_{ij}^{(\ell)} \, \phi(z_j^{(\ell-1)}) + b_i^{(\ell)} & \ell \geq 2 \\ \frac{1}{\sqrt{d}} \sum_{j=1}^{d} w_{ij}^{(1)} \, x_j + b_i^{(1)} & \ell = 1 \end{cases}$
  with $z^{(0)} = x$ and $z^{(L+1)} = f$; parameters are $\theta = \{w^{(\ell)}, b^{(\ell)}\}_{\ell=1}^{L+1}$. The explicit factors of $1/\sqrt{N_{\ell-1}}$ are a normalization convention, explained in Section 3.2: with them in place, all weights and biases can be taken of order one.

3.2 Normalizing the network

Consider a fully connected network of depth L with layer widths N_ℓ as in Section 3.1. Suppose we keep L fixed but make the N_ℓ very large. Suppose also that the network is initialized randomly, with all weights and biases drawn independently from normal distributions of *order-one* variance,

$$w_{ij}^{(\ell)} \sim \mathcal{N}(0, \sigma_w) \,, \qquad b_i^{(\ell)} \sim \mathcal{N}(0, \sigma_b) \,, \tag{3.1}$$

with σ_w, σ_b fixed as the widths grow. (Here and throughout, $\mathcal{N}(0, \sigma)$ denotes a Gaussian of *variance* σ. Letting the variances depend on the layer changes nothing below.) The question is how to normalize the sums in the network recursion so that preactivations stay of order one from layer to layer. From

$$z_i^{(\ell)} = \frac{1}{\sqrt{N_{\ell-1}}} \, w_i^{(\ell)} \cdot \phi(z^{(\ell-1)}) + b_i^{(\ell)} \tag{3.2}$$

(where we've written $w_i = (w_{i,1}, ..., w_{i,N_{\ell-1}})$ and $z^{(\ell-1)} = (z_1^{(\ell-1)}, ..., z_{N_{\ell-1}}^{(\ell-1)})$ as vectors) we get

$$\begin{aligned} \mathrm{Var}(z_i^{(\ell)}) &= \frac{1}{N_{\ell-1}} \sum_{j=1}^{N_{\ell-1}} \mathrm{Var}\big(w_{ij}^{(\ell)}\big) \, \mathbb{E}\Big[\phi\big(z_j^{(\ell-1)}\big)^2\Big] \; + \; \mathrm{Var}\big(b_i^{(\ell)}\big) \\ &= \sigma_w \cdot \frac{1}{N_{\ell-1}} \sum_{j=1}^{N_{\ell-1}} \Big(\mathrm{Var}\big(\phi(z_j^{(\ell-1)})\big) + \mathbb{E}\big[\phi(z_j^{(\ell-1)})\big]^2\Big) + \sigma_b \,. \end{aligned} \tag{3.3}$$

Here we've conditioned on the previous layer, and used that for independent mean-zero X and arbitrary Y, $\mathrm{Var}(XY) = \mathrm{Var}(X) \, \mathbb{E}[Y^2]$, along with additivity of variance for independent summands. The average in the second line involves only the activation

function and the order-one preactivations of the previous layer, so it stays of order one as $N_{\ell-1} \to \infty$. (We'll sharpen this estimate in Section 3.7.) Thus

$$\mathrm{Var}(z_i^{(\ell)}) \approx \sigma_w \cdot O(1) + \sigma_b = O(1)\,. \tag{3.4}$$

The prefactor $1/\sqrt{N_{\ell-1}}$, the square root of the number of inputs (the "fan-in") feeding each neuron, is exactly what keeps the preactivations of order one at large width. We call this the *fan-in normalization*; it goes back to LeCun and collaborators in the 90's [51]. In Section 4 it will also be called the *NTK normalization*, and in Section 5 we will meet the alternative *mean-field normalization*, in which the output layer carries $1/N$ instead of $1/\sqrt{N}$.

Remark (other conventions)

Common practice, and the convention of much of the literature, is to put no explicit prefactors in the network and instead to initialize with width-dependent variances,

$$\tilde{w}_{ij}^{(\ell)} := \frac{w_{ij}^{(\ell)}}{\sqrt{N_{\ell-1}}} \sim \mathcal{N}\Big(0, \frac{\sigma_w}{N_{\ell-1}}\Big)\,. \tag{3.5}$$

At initialization the two conventions describe *exactly* the same distribution of network functions: (3.5) is a change of variables on parameter space, and the network function is the same function of x. Everything in this section, which concerns initialization only, is insensitive to the choice. During training, however, the two conventions are *not* equivalent: gradient descent is not invariant under rescaling coordinates on parameter space, as we will explain in Section 4.3. Keeping every parameter of order one, with the factors of N displayed explicitly in the network function, makes the comparison between different scaling limits transparent, and it is the convention we use throughout these notes. When reading the literature, expect the factors of N to sit in the variances instead, and translate with (3.5).

3.3 Gaussian processes

A *Gaussian process* $f : \mathbb{R}^m \to \mathbb{R}$ with mean function μ and covariance kernel K is a random function such that on any p inputs $(x^1, \ldots, x^p) \in (\mathbb{R}^m)^p$ the vector of function values is Gaussian

$$\begin{pmatrix} f(x^1) \\ \vdots \\ f(x^p) \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu(x^1) \\ \vdots \\ \mu(x^p) \end{pmatrix}, \begin{pmatrix} K(x^1,x^1) & \cdots & K(x^1,x^p) \\ \vdots & \ddots & \vdots \\ K(x^p,x^1) & \cdots & K(x^p,x^p) \end{pmatrix} \right)\,. \tag{3.6}$$

Here $\mu : \mathbb{R}^m \to \mathbb{R}$ and $K : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}$ are fixed, deterministic functions that control the mean and covariance of any tuple of values of f. The function K, often called the kernel of the process, is symmetric: $K(x, x') = K(x', x)$.

Let 'dx' denote a measure on $\mathbb{R}^m$ (it need not be the usual Lebesgue measure). If K, as integral kernel, has an inverse K^{-1} satisfying

$$\int dx' K^{-1}(x,x')K(x',x'') = \delta(x-x'') \tag{3.7}$$

then there's also a "physics definition" of a Gaussian process. It's not entirely rigorous, as it invokes multi-dimensional functional integrals. Mathematically, it's best thought of as a heuristic tool, which comes with a package of axiomatic manipulations whose actual results can (and should) all be fully justified by other means. The manipulations are all quite intuitive, though, and familiar to quantum field theorists, so we mention this formalism here. The idea is that, just like a single Gaussian variable 'z' has a probability measure

$$P(z) = \frac{1}{Z} e^{-\frac{1}{2}K^{-1}(z-\mu)^2}\,, \qquad Z = \int dz\, e^{-\frac{1}{2}K^{-1}(z-\mu)^2} = \sqrt{2\pi K}\,, \tag{3.8}$$

a Gaussian process f has a (non-rigorous) measure

$$\begin{aligned} P[f] &= \frac{1}{Z} e^{-\frac{1}{2}\int dx\, dx'\,(f(x)-\mu(x))K^{-1}(x,x')(f(x')-\mu(x'))}\,, \\ Z\ \text{`}=\text{'} &\int Df\, e^{-\frac{1}{2}\int dx\, dx'\,(f(x)-\mu(x))K^{-1}(x,x')(f(x')-\mu(x'))}\,. \end{aligned} \tag{3.9}$$

If the kernel and the measure theory on the input space $\mathbb{R}^m$ are nice enough to allow K to be diagonalized, with a discrete basis of orthonormal eigenfunctions $\{\phi_k(x)\}_{k=1}^{\infty}$ and eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots > 0$ (so that K^{-1} has eigenvalues $1/\lambda_k$), then $P[f]$ can be understood as an infinite-dimensional Gaussian. Writing $f(x) - \mu(x) = \sum_{k=1}^{\infty} a_k \phi_k(x)$,

$$P[f] = \frac{1}{Z} e^{-\frac{1}{2}\sum_{k=1}^{\infty} a_k^2/\lambda_k}\,, \qquad Z = \prod_{k=1}^{\infty} (2\pi\lambda_k)^{1/2}\,. \tag{3.10}$$

One virtue of the physics formalism is that it organizes expectation values of products of $f(x)$'s. Let $\mu = 0$ for simplicity (to generalize to any μ, just shift $f \mapsto f - \mu$ below). Let $J : \mathbb{R}^m \to \mathbb{R}$ be an arbitrary (nice...) function and define

$$Z[J] = \int Df\, e^{-\frac{1}{2}\int dx\, dx'\, f(x)K^{-1}(x,x')f(x') + \int dx\, J(x)f(x)}\,. \tag{3.11}$$

The integral evaluates formally to

$$Z[J] = Z[0] \cdot e^{\frac{1}{2}\int dx\, dx'\, J(x)K(x,x')J(x')}\,. \tag{3.12}$$

Then expectation values can be expressed via derivatives, which by (3.12) become

$$\mathbb{E}\big[f(x^1)\ldots f(x^p)\big] = \frac{1}{Z[0]} \frac{\delta}{\delta J(x^1)} \cdots \frac{\delta}{\delta J(x^p)} Z[J]\bigg|_{J=0} \tag{3.13}$$

$$= \sum_{\text{pairs } \{i_1,i_2\}\cup\ldots\cup\{i_{p-1},i_p\}=\{1,2,\ldots,p\}} \prod_{j=1}^{p/2} K(x^{i_{2j-1}}, x^{i_{2j}})\,. \tag{3.14}$$

The sum is over partitions of $\{1, ..., p\}$ into pairs, and vanishes if p is odd. Eqn. (3.14) is called Wick's Theorem in QFT, Isserlis's Theorem in probability (and it's correct, despite the non-rigorous derivation here).

Exercise 3.1 (Wick's theorem). If you haven't seen this before, derive (3.12) by completing the square in (3.11), and then derive (3.14) by differentiating. Check the first two cases explicitly: $\mathbb{E}[f(x^1)f(x^2)] = K(x^1, x^2)$, and $\mathbb{E}[f(x^1)f(x^2)f(x^3)f(x^4)] = K^{12}K^{34} + K^{13}K^{24} + K^{14}K^{23}$ (with $K^{ij} = K(x^i, x^j)$). How many terms appear at $p = 6$?

Another virtue of the physics formalism is that it neatly connects *higher cumulants* to *deviations from Gaussianity*. In general, the p-th cumulant $\kappa_p(X_1, ..., X_p)$ of random variables $X_1, ..., X_p$ is defined recursively via

$$\mathbb{E}(X_1 \cdots X_p) = \sum_{\text{partition } \Pi \text{ of } \{1,...,p\}} \prod_{\text{blocks } \{i_1,...,i_k\} \subset \Pi} \kappa_k(X_{i_1}, ..., X_{i_k}) \tag{3.15}$$

For example, $\kappa_1(X) = \mathbb{E}(X)$, $\kappa_2(X_1, X_2) = \mathrm{Cov}(X_1, X_2) = \mathbb{E}(X_1X_2) - \mathbb{E}(X_1)E(X_2)$. In physics, the κ_p's are called connected correlation functions; they measure the part of a joint expectation that's not captured by expectations of subsets of the variables involved. For a Gaussian process,

$$\kappa_2(f(x^1), f(x^2)) = K(x^1, x^2)\,; \qquad \kappa_p(f(x^1), ..., f(x^p)) = 0 \quad \text{for all } p > 2\,, \tag{3.16}$$

as can easily be seen directly from (3.14), where joint expectations for $p > 2$ all factor into products of pairs. Thus, nonvanishing higher κ_p's are a measure of non-Gaussianity.

This appears in the functional-integral formalism as follows. Suppose that $f : \mathbb{R}^m \to \mathbb{R}$ is a random function that's governed by a functional probability distribution

$$P[f] = \frac{1}{Z}\, e^{-S[f]}\,, \tag{3.17}$$

for some functional $S : \{\text{functions } f\} \to \mathbb{R}$. (In physics S is called the action.) Then just as in (3.11), we can introduce the generating function

$$\begin{aligned} Z[J] &:= \int Df\, e^{-S[f] + \int dx\, J(x)f(x)} \\ \Rightarrow \quad \mathbb{E}\big[f(x^1)...f(x^p)\big] &= \frac{1}{Z[0]} \frac{\delta}{\delta J(x^1)} ... \frac{\delta}{\delta J(x^p)} Z[J]\Big|_{J=0} \end{aligned} \tag{3.18}$$

The cumulants appear naturally as derivatives of $\log Z[J]$, otherwise known as the (sourced) free energy:

$$\kappa_p(f(x^1), ..., f(x^p)) = \frac{\delta}{\delta J(x^1)} ... \frac{\delta}{\delta J(x^p)} \log Z[J]\Big|_{J=0}\,, \tag{3.19}$$

which in turn means (formally) that

$$Z[J] = \exp \sum_{p=1}^{\infty} \frac{1}{p!} \int dx^1 ... dx^p \, \kappa_p(f(x^1), ..., f(x^p)) \, J(x^1) ... J(x^p) \,, \tag{3.20}$$

a generating function of cumulants. The original $S[f]$ is the inverse Legendre transform of $\log Z[J]$ (by definition of $Z[J]$). Expanded as a formal series in the cumulants (assuming they are small), and assuming $\mathbb{E}[f(x)] = 0$ for simplicity, it takes the form

$$\begin{aligned} S[f] = \frac{1}{2} \int & dx^1 \, dx^2 \, \kappa_2^{-1}(x^1, x^2) f(x^1) f(x^2) \\ - \int & dx^1 dx^2 dx^3 \, dy^1 dy^2 dy^3 \; \kappa_2^{-1}(x^1, y^1) \, \kappa_2^{-1}(x^2, y^2) \\ & \times \; \kappa_2^{-1}(x^3, y^3) \, \kappa_3(y^1, y^2, y^3) \, f(x^1) f(x^2) f(x^3) \\ - \int & dx^1 dx^2 dx^3 dx^4 \, (\kappa_2^{-1})^4 \big[\kappa_4 - \kappa_3 \kappa_2^{-1} \kappa_3 \;\; \text{terms} \big] \\ & \times \; f(x^1) f(x^2) f(x^3) f(x^4) \; - \ldots \end{aligned} \tag{3.21}$$

Physicists might recognize this as a sum of Feynman diagrams, with cumulants giving the vertices. A key structure is that the p-th order term in the effective action contains only κ_p and lower cumulants. In particular, f is a Gaussian process if and only if $S[f]$ is quadratic; and small deviations from Gaussianity, measured by cumulants, provide the higher-order terms in $S[f]$.

3.4 The NNGP correspondence

A classic result in the theory of neural networks is that, at fixed depth and large width, with the fan-in normalization of Section 3.2, the network function at initialization converges to a Gaussian process. The theorem appeared in the 90's [62] for a single hidden layer, and was generalized more recently to deeper networks and different architectures, *cf.* [52, 95, 36].

In this theorem, the source of stochasticity in the network function comes from the random initializations of network parameters. One can think of randomly initializing a swarm of networks and asking what sorts of values the network function outputs. The result is significant because of its universality, and because it feeds into (and greatly simplifies) further analyses of training.

The derivation proceeds via induction. We'll just sketch it here, for MLP's.[2] For simplicity we'll assume that biases have been set to zero, and that all weights have the same order-one variance σ_w, as in (3.1), with the fan-in normalization of Section 3.2.

In the first network layer, the preactivations are

$$z_i^{(1)}(x) = \frac{1}{\sqrt{d}} \sum_{j=1}^{d} w_{ij}^{(1)} x_j \,, \qquad w_{ij}^{(1)} \sim \mathcal{N}(0, \sigma_w) \,. \tag{3.22}$$

[2]For transformers, the limit analogous to "large width" that leads to a Gaussian process is a large number of attention heads.

At fixed x, each $z_i^{(1)}(x)$ is a sum of d i.i.d. Gaussian variables (with coefficients x_j), and thus is Gaussian itself, with $\mathbb{E}[z_i^{(1)}] = 0$ and

$$\mathrm{Cov}\big(z_i^{(1)}(x), z_i^{(1)}(x')\big) = \frac{1}{d}\sum_{j,j'=1}^{d} \underbrace{\mathrm{Cov}(w_{ij}, w_{ij'})}_{\delta_{jj'}\sigma_w} x_j x'_{j'} = \sigma_w \frac{x \cdot x'}{d}\,, \tag{3.23}$$

thus $z_i^{(1)}$ is a Gaussian process (GP) with

$$z_i^{(1)} \sim \mathcal{N}(0, K^{(1)})\,, \qquad K^{(1)}(x, x') = \sigma_w \frac{x \cdot x'}{d}\,. \tag{3.24}$$

In the second layer, we've got

$$z_i^{(2)} = \frac{1}{\sqrt{N_1}} \sum_{j=1}^{N_1} w_{ij}^{(2)} \phi(z_j^{(1)})\,, \qquad w_{ij}^{(2)} \sim \mathcal{N}(0, \sigma_w)\,. \tag{3.25}$$

If we condition on the preactivations of the first layer, the same argument gives

$$z_i^{(2)} \sim \mathcal{N}(0, K^{(2)})\,, \qquad K^{(2)}(z^{(1)}, z^{(1)\prime}) = \sigma_w \frac{\phi(z^{(1)}) \cdot \phi(z^{(1)\prime})}{N_1}\,. \tag{3.26}$$

Thus we again get a Gaussian process. The problem is that if we don't condition on the preactivations, the kernel of this GP is not deterministic, but becomes a random variable itself: $\phi(z^{(1)}) \cdot \phi(z^{(1)\prime})$. This is where the infinite-width limit comes in. Heuristically, the central limit theorem implies that the sum of products of (non-Gaussian!) random variables on the LHS of (3.25) will become Gaussian as $N_1 \to \infty$. More concretely, we can calculate and bound the variance

$$\mathrm{Var}\big(K^{(2)}(z^{(1)}, z^{(1)\prime})\big) = \frac{\sigma_w^2}{N_1^2} \mathrm{Var}\big(\phi(z^{(1)}) \cdot \phi(z^{(1)\prime})\big) \tag{3.27}$$

$$= \frac{\sigma_w^2}{N_1^2} N_1 \cdot \mathrm{Var}\big(\phi(z_1^{(1)})\phi(z_1^{(1)\prime})\big) \to 0 \quad \text{as } N_1 \to \infty \tag{3.28}$$

as long as the first-layer preactivations are order one (independent of N_1), and the activation function is reasonable (any standard choice will be). Thus, as $N_1 \to \infty$, we can replace the stochastic kernel $K^{(2)}$ by its expectation,

$$\bar{K}^{(2)}(x, x') := \sigma_w \, \mathbb{E}_{z_1^{(1)}(x), z_1^{(1)}(x') \sim \mathcal{N}(0, K^{(1)})}[\phi(z_1^{(1)}(x))\phi(z_1^{(1)}(x'))] \tag{3.29}$$

Each subsequent layer is the same. At infinite width, the kernels become deterministic, and obey a recursion

$$\bar{K}^{(\ell)}(x, x') = \sigma_w \, \mathbb{E}_{u(x), u(x') \sim \mathcal{N}(0, \bar{K}^{(\ell-1)})}[\phi(u(x))\phi(u(x'))]\,, \tag{3.30}$$

with $\bar{K}^{L+1}(x, x')$ ultimately governing a GP for the network function $f(x)$.

Remark (exact kernels)

For a few special activation functions, the Gaussian expectation in (3.30) can be evaluated in closed form, giving explicit NNGP kernels. Write $K_{11} = \bar{K}^{(\ell-1)}(x,x)$, $K_{22} = \bar{K}^{(\ell-1)}(x',x')$ and $K_{12} = \bar{K}^{(\ell-1)}(x,x')$ for the previous layer's kernel values. For ReLU, $\phi(u) = \max(u,0)$, one finds the *arc-cosine kernel* of Cho and Saul [21],

$$\begin{aligned} \bar{K}^{(\ell)}(x,x') &= \frac{\sigma_w}{2\pi}\sqrt{K_{11}K_{22}}\left(\sin\vartheta + (\pi-\vartheta)\cos\vartheta\right), \\ \cos\vartheta &= \frac{K_{12}}{\sqrt{K_{11}K_{22}}}, \end{aligned} \tag{3.31}$$

and for the sigmoid-like activation $\phi = \mathrm{erf}$ Williams [93] found

$$\bar{K}^{(\ell)}(x,x') = \frac{2\sigma_w}{\pi}\arcsin\left(\frac{2K_{12}}{\sqrt{(1+2K_{11})(1+2K_{22})}}\right). \tag{3.32}$$

Iterating either formula through the layers gives the exact infinite-width kernel of a deep ReLU or erf network in a few lines of code. (Deriving (3.31)–(3.32) is a pleasant but somewhat lengthy Gaussian-integral workout.)

Exercise 3.2 (The NNGP correspondence, empirically (lab)). Fix two unit-norm inputs $x, x' \in \mathbb{R}^d$ with, say, $d = 8$ and $x \cdot x' = 0.5$. For widths $N \in \{3, 30, 300\}$, sample $M = 10^4$ independent initializations of the one-hidden-layer erf network $f(x) = \frac{1}{\sqrt{N}}\sum_{i=1}^{N} c_i \,\mathrm{erf}\big(a_i \cdot x/\sqrt{d}\big)$ with $c_i, a_{ij} \sim \mathcal{N}(0,1)$ (that is, $\sigma_w = 1$).

(a) Make a histogram of the values $f(x)$ across the ensemble, and overlay the Gaussian density with the variance $\bar{K}^{(2)}(x,x)$ predicted by (3.32); watch the visibly non-Gaussian $N = 3$ histogram converge.

(b) Make a scatter plot of the pairs $\big(f(x), f(x')\big)$ and compare the sample 2×2 covariance with the predicted kernel matrix.

(c) For each initialization, record the stochastic kernel $K^{(2)}$ of (3.26), and verify that its variance across initializations decays like $1/N$ (that's the mechanism by which the kernel becomes deterministic).

3.5 NNGP via functional integrals

One can also analyze large width at initialization via functional integrals. This just reorganizes the above steps. We'll illustrate how it works for a network with one hidden

layer and zero biases, $N_2 = 1$, $N_1 = N$, $N_0 = d$,

$$f(x) = \frac{1}{\sqrt{N}} \sum_{i=1}^{N} c_i \, \phi\Big(\frac{1}{\sqrt{d}} \, a_i \cdot x\Big) \,, \qquad c_i \in \mathbb{R}, \; a_i, x \in \mathbb{R}^d \,. \tag{3.33}$$

The initial distribution of weights, all of variance σ_w, is

$$P(c, a) = \frac{1}{Z} \, e^{-\frac{1}{2\sigma_w}|c|^2 \, - \, \frac{1}{2\sigma_w}|a|^2} \,. \tag{3.34}$$

To push this forward to a distribution on the first-layer preactivations $z_i = \frac{1}{\sqrt{d}} a_i \cdot x$, we integrate against a delta function:

$$P(c, z) = \int da \, P(c, a) \, \prod_i \delta\Big(z_i(x) - \tfrac{1}{\sqrt{d}} a_i \cdot x\Big) \,. \tag{3.35}$$

Here the $z_i(x)$ are *arbitrary* functions of x, and are only tied to the network values $\frac{1}{\sqrt{d}} a_i \cdot x$ via the delta functions. Eqn. (3.35) represents a functional probability density on the functions z_i. To evaluate it, introduce a second set of "Lagrange multiplier" functions $\tilde{z}_i$, and write

$$\delta\Big(z_i(x) - \tfrac{1}{\sqrt{d}} a_i \cdot x\Big) = \int D\tilde{z}_i \, e^{\, i \int dx \, \tilde{z}_i(x)(z_i(x) - \frac{1}{\sqrt{d}} a_i \cdot x)} \,, \tag{3.36}$$

and then do the exact Gaussian integral over a's, and then over $\tilde{z}$'s:

$$\begin{aligned} P(c, z) &= \frac{1}{Z} \int da \, D\tilde{z} \, \exp\Big[- \tfrac{1}{2\sigma_w}|c|^2 - \textstyle\sum_i \tfrac{1}{2\sigma_w}|a_i|^2 \\ &\qquad\qquad + i \textstyle\sum_i \int dx \, \tilde{z}_i(x)(z_i(x) - \tfrac{1}{\sqrt{d}} a_i \cdot x) \Big] \qquad (3.37) \\ &= \frac{1}{Z} \int da \, D\tilde{z} \, \exp\Big[- \tfrac{1}{2\sigma_w}|c|^2 + \textstyle\sum_i \Big[- \tfrac{1}{2\sigma_w} \big(a_i + \tfrac{i\sigma_w}{\sqrt{d}} \textstyle\int dx \, \tilde{z}_i(x) x\big)^2 \\ &\qquad - \tfrac{\sigma_w}{2d} \textstyle\int dx \, dx' \, \tilde{z}_i(x)(x \cdot x')\tilde{z}_i(x') + i \int dx \, \tilde{z}_i(x) z_i(x) \Big] \Big] \qquad (3.38) \\ &= \frac{1}{Z'} \int D\tilde{z} \, \exp\Big[- \tfrac{1}{2\sigma_w}|c|^2 - \tfrac{\sigma_w}{2d} \textstyle\sum_i \int dx \, dx' \, \tilde{z}_i(x)(x \cdot x')\tilde{z}_i(x') \\ &\qquad\qquad + i \textstyle\sum_i \int dx \, \tilde{z}_i(x) z_i(x) \Big] \qquad (3.39) \\ &= \frac{1}{Z''} \exp\Big[- \tfrac{1}{2\sigma_w}|c|^2 - \tfrac{1}{2\sigma_w} \textstyle\sum_i \int dx \, dx' \, z_i(x) \big(\tfrac{x \cdot x'}{d}\big)^{-1} z_i(x') \Big] \,. \qquad (3.40) \end{aligned}$$

We clearly see the first-layer GP kernel $K^{(1)}(x, x') = \frac{\sigma_w}{d} x \cdot x'$ appearing.

To push-forward to the network outputs $f = \frac{1}{\sqrt{N}} c \cdot \phi(z)$, we just do this again:

$$P(f) = \int dc\, Dz\, P(c,z)\, \delta\Big(f(x) - \tfrac{1}{\sqrt{N}} c \cdot \phi(z(x))\Big) \tag{3.41}$$

$$= \frac{1}{Z''} \int dc\, Dz\, D\tilde{f}\, \exp\Big[-\tfrac{1}{2\sigma_w}|c|^2 - \tfrac{1}{2\sigma_w} \textstyle\sum_i \int dx\, dx'\, z_i(x) \big(\tfrac{x\cdot x'}{d}\big)^{-1} z_i(x') + i \int dx \tilde{f}(x)(f(x) - \tfrac{1}{\sqrt{N}} c \cdot \phi(z(x)))\Big] \tag{3.42}$$

$$= \frac{1}{Z'''} \int Dz\, D\tilde{f}\, \exp\Big[-\tfrac{1}{2\sigma_w} \int dx\, dx' \big(\tfrac{x\cdot x'}{d}\big)^{-1} z(x)\cdot z(x') - \tfrac{\sigma_w}{2N} \int dx\, dx' \tilde{f}(x)[\phi(z(x))\cdot\phi(z(x'))]\tilde{f}(x') + i \int dx\, \tilde{f}(x) f(x)\Big] \tag{3.43}$$

The problem is that it's now impossible to do the integral over the z's exactly, because (for general activation function ϕ) the action is no longer quadratic in them. However, as $N \to \infty$, we can treat $\exp\big[-\frac{\sigma_w}{2N} \int dx\, dx' \tilde{f}(x)[\phi(z(x))\cdot\phi(z(x'))]\tilde{f}(x')\big]$ as a perturbation, and approximate the *exponent* $[\phi(z(x))\cdot\phi(z(x'))]$ by its expectation value under the Gaussian measure $\exp\big[-\frac{1}{2\sigma_w} \int dx\, dx' \big(\frac{x\cdot x'}{d}\big)^{-1} z(x)\cdot z(x')\big]$ (with kernel $K^{(1)}$) on the z's. To be clear, the approximation that's happening is of the form

$$\mathbb{E}\, e^{\frac{1}{N}X} = e^{\frac{1}{N}\mathbb{E}X} + O(\mathrm{Var}(X)/N^2) \tag{3.44}$$

Up to $O(1/N)$ corrections, that produces the deterministic kernel of the previous section,

$$\bar{K}^{(2)}(x,x') = \sigma_w\, \mathbb{E}_{z\sim\mathcal{N}(0,K^{(1)})}\big[\phi(z(x))\,\phi(z(x'))\big]\,,$$

and we get

$$P(f) = \frac{1}{Z''''} \int D\tilde{f}\, \exp\Big[-\tfrac{1}{2} \int dx\, dx'\, \tilde{f}(x)\bar{K}^{(2)}(x,x')\tilde{f}(x') + i \int dx\, \tilde{f}(x) f(x)\Big] + O\Big(\tfrac{1}{N}\Big) \tag{3.45}$$

$$= \frac{1}{Z'''''} \exp\Big[-\tfrac{1}{2} \int dx\, dx'\, f(x)\bar{K}^{(2)}(x,x')^{-1} f(x')\Big] + O\Big(\tfrac{1}{N}\Big)\,. \tag{3.46}$$

3.6 L/N

The inductive argument of Section 3.4 does not hold at infinite L. There are two potential problems.

First, a factor of σ_w appears with every layer. These factors will accumulate, and can lead either to vanishing kernels (which also turns out to imply vanishing gradients during training), in which case the network will collapse; or to unbounded kernels (and increasingly large gradients), in which case the network will be chaotic and again

untrainable. We'll revisit this in Section 3.7. With careful tuning, the issue can be avoided. For example, in a deep linear network (DLN) with $\phi(z) = z$, one can just take $\sigma_w = 1$, known as LeCun initialization [51]. In a ReLU network, the correct choice instead is $\sigma_w = 2$, known as He initialization [38].

Even with σ_w tuned, according to the activation function, so that on average the GP kernels remain order-one from layer to layer, non-Gaussianities can still accumulate. The upshot is that it is not width alone, say of typical size N, but rather the ratio L/N of depth-to-width that controls the effective behavior of a network — including its departure from pure Gaussianity at large N.

This behavior is analyzed in Chapters 3-4 of [71] using functional-integral methods. In particular, Chapter 3 considers the simple example of DLN's, and produces exact recursion relations for higher cumulants of preactivations in each layer. It is shown that in a network of constant hidden width N, the leading non-Gaussianity is

$$\begin{aligned} \kappa_4^{(L+1)} &= \left[\left(1+\frac{2}{N}\right)^L - 1\right](\kappa_2^{(L+1)})^2 \\ &= \left[(e^{2L/N}-1) - \frac{1}{N}\frac{2L}{N}e^{2L/N} + O(N^{-2})\right](\kappa_2^{(L+1)})^2 \,, \end{aligned} \tag{3.47}$$

where the expansion on the RHS is at finite L/N and $L, N \to \infty$. For small L/N, the non-Gaussianity remains small but finite, of size L/N. For large L/N, $\kappa_4^{(L+1)}$ blows up, as do all the other cumulants. The network outputs will fluctuate chaotically from one instantiation to another, and the network again becomes untrainable.

Here's a more direct analysis, in a special case, following [37, Lecture 2]. Consider a DLN of length L with input and hidden layers all of width N:

$$\begin{aligned} z^{(\ell)} &= \frac{1}{\sqrt{N}}\, w^{(\ell)} z^{(\ell-1)} \,, \qquad z^{(0)} = x \,, \\ f(x) &= z^{(L+1)} = N^{-\frac{L+1}{2}}\, w^{(L+1)} \cdots w^{(1)} x \,, \end{aligned} \tag{3.48}$$

with $w_{ij}^{(\ell)} \sim \mathcal{N}(0,1)$, i.e. $\sigma_w = 1$ (and input dimension $d = N$). Conditional on $z^{(\ell-1)}$, the vector $z^{(\ell)}$ is Gaussian $z^{(\ell)} \sim \mathcal{N}\big(0, \frac{1}{N} I_N |z^{(\ell-1)}|^2\big)$, so its norm satisfies the recursion

$$|z^{(\ell)}|^2 \sim |z^{(\ell-1)}|^2 \frac{\chi_N^2}{N} \,. \tag{3.49}$$

(Recall that χ_N^2 is the distribution of the sum of squares of N i.i.d. standard Gaussian variables.) Let's normalize the input so that $|x| = 1$. Then the recursion implies that

$$|f(x)|^2 \overset{d}{=} \prod_{k=1}^{L+1} \frac{\chi_N^2}{N} = \exp \sum_{k=1}^{L+1} \left(\log \chi_N^2 - \log N\right) , \tag{3.50}$$

with each factor of χ_N^2 independent. At large N, we have $\log \chi_N^2 - \log N = \mathcal{N}(-\frac{1}{N}, \frac{2}{N}) + O(\frac{1}{N^2})$, so at large N, L with L/N fixed

$$|f(x)|^2 \overset{d}{=} e^{\mathcal{N}(-\frac{L}{N}, 2\frac{L}{N})} \,, \tag{3.51}$$

up to higher $O(\frac{1}{N^2}$ corrections. Thus, as L/N grows, the expectation of $\mathbb{E}|f(x)| \to 0$ but the variance and higher cumulants blow up.

The lesson once again is

- $N \to \infty$, $L/N = 0$: Gaussian process, no correlations across layers (and, as we'll see in Section 4, no "feature learning").
- $N, L \to \infty$, L/N small: controlled correlations, trainable network (and, as we'll see, feature learning during training). In open-weight LLM's today, $L/N \approx 0.005$–0.01; see Exercise 3.3.
- $N, L \to \infty$, L/N large: chaotic and untrainable.

Exercise 3.3 (Depth-to-width ratios in the wild). **(★)** Look up the depth L (number of transformer blocks) and the width N (model/embedding dimension) for a few families of open-weight language models — *e.g.* GPT-2, the Llama family, the Qwen family — and compute L/N for each. How much does the ratio vary across families, across sizes within a family, and across the years? Is everyone training in the "controlled correlations" window?

3.7 Edge of chaos

We briefly remark that the analysis of how to tune initialization variances σ_w, σ_b (etc.) leading to stable network behavior over a large number of layers (the first point in Section 3.6) is sometimes called finding the "edge of chaos."

The term dates back to dynamical systems theory from the 80's and 90's, referring to the boundary between ordered and chaotic phases. It was adapted to neural networks around 2016 [65, 75]. The idea was to treat the progression through layers of a network as a dynamical system, and to ask: given two nearby inputs x, x', for what range of hyperparameters (σ_w, σ_b, etc.) and to what depth do they remain nearby, without either a) collapsing; or (b) blowing up chaotically (with no correlation). The analysis roughly amounts to making sure the kernel recursion of (3.26), (3.30) leads to kernels that neither collapse nor blow up. Doing it carefully, theoretically, can produce elementary trainable networks (with no layer norms!) over 10,000 layers deep [94].

For MLP's with ReLu activations, the edge of chaos is just the one point $\sigma_w = 2$, $\sigma_b = 0$. Moving slightly away from this point, one can estimate the "critical depth" L_c at which the network either collapses or blows up – see [75]. For MLP's with other activation functions, the edge of chaos is more interesting. For $\phi = \tanh$, the phase diagram is shown in Figure 5.

The quantity 'χ_1' that controls the critical depth ($L_c = -1/\log \chi_1$) is the derivative of the layer-to-layer kernel recursion at its fixed point, which measures how fast nearby inputs separate. Importantly, the same quantity χ_1 also controls the size of gradients as they back-prop through a network *during training*. There's a direct link between behavior at initialization and behavior during training. We come to training next.

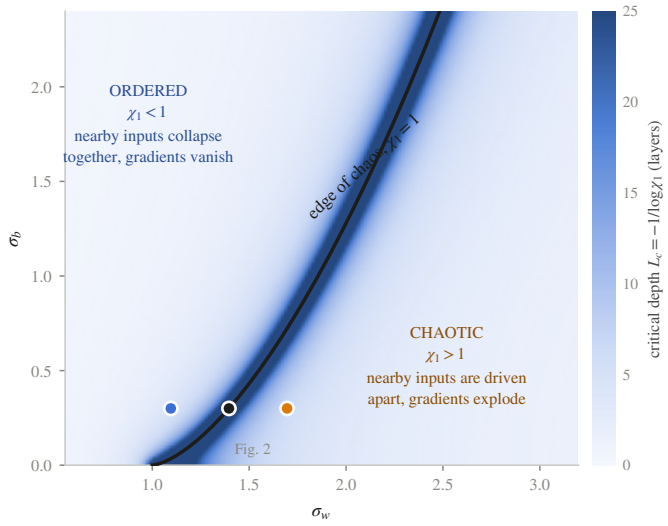


Figure 5: Order/chaos phase diagram for a tanh network, with shading indicating the critical depth to which a network is trainable.

4 The NTK and large width during training

In this section, we consider training networks that are randomly initialized as in Section 3.2, and their behavior at large width. The lesson will be that as width$\to\infty$ these networks effectively behave like linear models: their parameters θ never stray far from their values θ_0 at initialization, hence

$$f_\theta(x) \approx f_\theta^{\text{lin}}(x) := f_{\theta_0}(x) + (\theta - \theta_0) \cdot \nabla_\theta f_{\theta_0}(x)\,. \tag{4.1}$$

Under suitable conditions, the network itself also converges exponentially fast to a "memorizing network" that fits the training data perfectly. Heuristically, this happens because small changes in many parameters, as in (4.1), can add up to an order-one change in the network values themselves. Nevertheless, the network never experiences phase transitions or feature learning — we'll make this more precise below. The large-width regime (at finite depth and training data) is thus known as the *lazy* regime, or the NTK regime. (For a self-contained review of models that are linear in their parameters, and of why training them amounts to kernel ridge regression, see Appendix B.)

Although this regime does not describe the feature learning that makes practical networks interesting, it is important to understand for at least two reasons: it is an honest, analytically solvable model of training; and it is a precise lesson in what must be *avoided* if one wants genuine feature learning — a lesson we act on in Section 5.

We also emphasize the connection between Section 3 and the current analysis. Both involve the *same* large-width limit. At initialization, it leads to a Gaussian process. During training, we'll see that it leads to a linear model with lazy learning, converging once again to a GP at late training times.

4.1 Gradient flow and the NTK

Throughout this section, we approximate SGD with continuous-time *gradient flow* (*cf.* Section 2.1). This involves two approximations. First we update on the entire training dataset $\{(x^\mu, y^\mu)\}_{\mu=1}^n$ at each step, rather than on batches, so that SGD becomes GD:

$$\theta_{k+1} - \theta_k = -\eta_{GD}\, \nabla_\theta \mathcal{L}_\theta(\{x^\mu, y^\mu\})\big|_{\theta=\theta_k} \tag{4.2}$$

where

$$\mathcal{L}_\theta = \mathcal{L}_\theta(\{x^\mu, y^\mu\}) := \sum_{\mu=1}^n \ell_\theta(x^\mu, y^\mu) \tag{4.3}$$

is the full empirical loss.

Second, we scale down the learning rate $\eta_{GD} = \lambda^{-1}\eta$ and scale up the number of training steps, setting $k = \lfloor \lambda t \rfloor$ where $t \geq 0$ is a continuous "time" variable. As $\lambda \to \infty$, the gradient descent converges to a gradient flow

$$\dot\theta := \frac{d}{dt}\theta = -\eta\, \nabla_\theta \mathcal{L}_\theta\,. \tag{4.4}$$

All theorems in this section can be extended to SGD, adding additional error terms to bound the difference between SGD and GF. (See for example [84, Chapter 19].)

4.1.1 Geometry of gradient flows

Suppose that X is a smooth Riemannian manifold, with nondegenerate metric $g = g_{ab}dx^a dx^b$, inverse metric $g^{-1} = g^{ab}\frac{\partial}{\partial x^a}\frac{\partial}{\partial x^b}$, and a function $h : X \to \mathbb{R}$. Then one can define gradient flow with respect to (h, g) as

$$\dot x^a(t) = -\eta\, g^{ab}\frac{\partial h}{\partial x^b}\,, \qquad \text{or abstractly:} \quad \dot x = -\eta\, g^{-1}(dh)\,. \tag{4.5}$$

Note that this equation does not make sense (it's not coordinate-free) without an inverse-metric (a bilinear form on each cotangent space T^*X). The inverse-metric is used to relate the differential $dh = (\partial_b h)_{b=1}^{\dim X}$ on the RHS to a *vector field* $g^{-1}dh$, which is the right sort of geometric object to induce a flow on X.

In neural networks, $X = \mathbb{R}^P$ is the space of parameters θ and $h = \mathcal{L}$ is the loss function. One always assumes a constant Euclidean metric in parameter space $g = \sum_{i=1}^P (d\theta^i)^2 = \delta_{ij}d\theta^i d\theta^j$, in order to define SGD/GD/GF/etc. This is the source of some of the simplest inductive biases of trained models, and also why one expects "features" to have some linear structure. Eqn. (4.5) with the metric looks like

$$\dot\theta^i = -\eta\, \delta^{ij}\, \partial_j \mathcal{L}_\theta\,. \tag{4.6}$$

Now return to the general setting of Riemannian manifolds. Suppose we have a smooth map $f : X \to Y$ between two manifolds, and moreover that $h : X \to \mathbb{R}$ is the pull-back of a function $h^Y : Y \to \mathbb{R}$, meaning $h(x) = h^Y(f(x))$. Let $\{f^\alpha\}_{\alpha=1}^{\dim Y}$ be the

components of the map f, in local coordinates y^α on Y. Then the inverse-metric on X induces an inverse-metric on Y (called its push-forward):

$$f_*(g^{-1}) = f_*\left[g^{ab}\frac{\partial}{\partial x^a}\frac{\partial}{\partial x^b}\right] = \underbrace{g^{ab}\frac{\partial f^\alpha}{\partial x^a}\frac{\partial f^\beta}{\partial x^b}}_{g^{\alpha\beta}}\frac{\partial}{\partial y^\alpha}\frac{\partial}{\partial y^\beta}\,. \tag{4.7}$$

Schematically, $g^{\alpha\beta} = (Jg^{-1}J^T)$, where $J^\alpha{}_a = \frac{\partial f^\alpha}{\partial x^a}$ is the Jacobian matrix of f. Then the gradient flow on X induces a gradient flow on Y, given by

$$\dot{y}^\alpha = -\eta\, g^{\alpha\beta}\frac{\partial h^Y}{\partial y^\beta}\,. \tag{4.8}$$

Let's apply this to neural networks, where $f : \mathbb{R}^P \to \mathbb{R}^n$ is the function from parameters to network outputs on the training data (x^μ, y^μ), *i.e.* $f^\mu(\theta) = f_\theta(x^\mu)$ in our standard notation. The loss function is indeed a pullback of a function on $\mathbb{R}^n$, since it depends on parameters θ only through the network values on training data,

$$\mathcal{L}_\theta = \mathcal{L}^{(f)}(f(\theta))\,, \qquad \mathcal{L}^{(f)}(f) := \sum_{\mu=1}^n \ell(f^\mu, y^\mu)\,. \tag{4.9}$$

The push-forward of the gradient flow in parameter space becomes

$$\frac{d}{dt}f^\mu = -\eta\,\Theta^{\mu\nu}\frac{\partial\mathcal{L}^{(f)}}{\partial f^\nu}\,, \qquad \Theta^{\mu\nu} = (J\delta^{-1}J^T)^{\mu\nu} = \sum_{i=1}^P \frac{\partial f_\theta(x^\mu)}{\partial\theta^i}\frac{\partial f_\theta(x^\nu)}{\partial\theta^i}\,. \tag{4.10}$$

The inverse-metric $\Theta^{\mu\nu}$ on the space of function values is called the "neural tangent kernel" or NTK. The name was coined in [42]; it also appeared in [24] under a different name. Here we are emphasizing that it's a completely natural geometric object, for any neural network (in fact, for any ML model).

We mention one final bit of geometry. Let $\delta_{\mu\nu}$ be a standard Euclidean metric in the space of function values, with inverse $\delta^{\mu\nu}$. The gradient of the loss $\delta^{\mu\nu}\partial\mathcal{L}^{(f)}/\partial f^\nu$ that appears in (4.10) (contracted with an inverse metric) is sometimes called the *residual*, and denoted Δ^μ or r^μ. For MSE loss, we've got $\mathcal{L}^{(f)} = \frac{1}{2}\sum_{\mu=1}^n (f^\mu - y^\mu)^2 = \frac{1}{2}\delta_{\mu\nu}(f^\mu - y^\mu)(f^\nu - y^\nu)$. Then the residual is just the difference between function values and truth for the training data:

$$\Delta^\mu := \delta^{\mu\nu}\partial\mathcal{L}^{(f)}/\partial f^\nu = f^\mu - y^\mu\,. \tag{4.11}$$

Noting that y^μ are constants, the gradient-flow equation can be recast as

$$\frac{d}{dt}\Delta^\mu = -\eta\,\underbrace{\Theta^{\mu\rho}\delta_{\rho\nu}}_{\Theta^\mu{}_\nu}\Delta^\nu\,. \tag{4.12}$$

Those who have studied gauge theory may recognize this as an equation for *parallel transport*, with respect to an $n\times n$ matrix-valued connection $\eta\,\Theta^\mu{}_\nu$. If one knows the

dependence of Θ on time (which in general is far from trivial — see the exercise below) the solution is a time-ordered exponential

$$\Delta^\mu(t) = \left(P \exp\left[-\eta \int \Theta(t)\, dt \right] \right)^\mu_\nu \Delta^\nu(0)\,. \tag{4.13}$$

Exercise 4.1 (The NTK from the chain rule). Derive the flow equation (4.10) for the function values f^μ directly, using only the chain rule — no geometry required. (The geometric derivation explains *why* an object like $\Theta^{\mu\nu}$ had to appear; the chain rule shows it appears in three lines.)

Exercise 4.2 (How fast does the kernel move?). Show that the time derivative of the NTK satisfies

$$\frac{d}{dt}\Theta^{\mu\nu} = -\eta \left[\nabla_\theta f^\mu \cdot \nabla^2_\theta f^\nu \cdot \nabla_\theta f^\rho \right] \frac{\partial \mathcal{L}^{(f)}}{\partial f^\rho} + (\mu \leftrightarrow \nu)\,, \tag{4.14}$$

where $\nabla^2_\theta f^\nu = \nabla^2_\theta f_\theta(x^\nu)$ is the Hessian. Bounding the Hessian ultimately leads to the constant-NTK theorem in Section 4.3 below.

4.2 Features

The Jacobian matrix $J^\mu{}_i = \partial f^\mu / \partial \theta^i$ measures how function values on the training data depend on small variations of the network weights. Assuming there are more parameters than training data, its singular value decomposition looks like

$$J = USV^T\,, \qquad S = \Big[\mathrm{diag}(s_1, ..., s_n) \;\; \mathbf{0}_{n\times(P-n)} \Big]\,. \tag{4.15}$$

The right singular vectors (columns of V) define natural modes in parameter space, while the left singular vectors (columns of U) are the natural modes in function-value space — where "natural" means with respect to how network outputs depend on weights. The n left singular vectors (together with their singular values s_i) can be thought of, abstractly, as the *features* that a network has learned. A common definition of feature *learning* is that these singular vectors+values change during training.

The analysis of features can equivalently be performed via the NTK. As a matrix, we've got

$$\Theta = JJ^T = US^2U^T\,, \tag{4.16}$$

with eigenvectors = the feature vectors, and eigenvalues = $\{s_i^2\}$. Thus feature learning happens when the NTK changes during training.

Remark

A closely related "dual" object that appears in other parts of learning theory is

the Fisher information metric on parameter space $\mathbb{R}^P$. Its empirical version is

$$\mathcal{F} := J^T J = f^*(\delta)\,, \tag{4.17}$$

the pull-back (via the network function) of constant Euclidean metric on the space of function values on the training data. In coordinates, $\mathcal{F}_{ij} = \frac{\partial f^\mu}{\partial \theta^i}\delta_{\mu\nu}\frac{\partial f^\mu}{\partial \theta^j}$. The Fisher metric has eigenvalues $(s_1^2, ..., s_n^2, \overbrace{0, ..., 0}^{P-n})$, and is automatically degenerate when $P > n$ (more parameters than training data points). This is one of the starting points of singular learning theory.

4.3 Constant NTK at large width

Now we get to the main theoretical result and simplification of this section. Suppose we've got an MLP of fixed depth L, and fixed training data size n, and take the width of all hidden layers to infinity. We'll assume the hidden widths all scale in a similar way $N_L, N_{L-1}, ..., N_1 \approx N \to \infty$, and we initialize the network the standard way from Section 3.2. We want to show that for almost all samples of training data,

- The NTK $\Theta^{\mu\nu}$ remains constant throughout training, up to corrections that vanish as $N \to \infty$.
- The network itself linearizes: at any training time t, $f_{\theta(t)}(x) = f_{\theta_0}(x) + (\theta(t) - \theta_0)\cdot\nabla_\theta f_{\theta_0}(x)$

A rigorous derivation is somewhat technical, and we refer the reader to [24, 42], [71, Ch. 11], [84, Ch. 20] for complete arguments, explained from a variety of different perspectives. We'll just sketch the key ideas behind the result. See also [37, Lecture 3].

We'll simplify life by setting biases to zero, so the network becomes

$$f_\theta(x) = z^{(L+1)}(x)\,, \quad z_i^{(\ell)} = \frac{1}{\sqrt{N_{\ell-1}}}\sum_j w_{ij}^{(\ell)}\phi(z_j^{(\ell-1)})\,, \quad w_{ij}^{(\ell)} \sim \mathcal{N}(0, \sigma_w)\,, \tag{4.18}$$

in the fan-in normalization of Section 3.2, with order-one weights.

Size of the NTK. The first task is to estimate how large the NTK is at large width, because that decides how the learning rate must scale. By the chain rule, the gradient with respect to the layer-ℓ weights is an outer product,

$$\frac{\partial f}{\partial w_{ij}^{(\ell)}} = \frac{1}{\sqrt{N_{\ell-1}}}\,\beta_i^{(\ell)}\,\phi(z_j^{(\ell-1)})\,, \qquad \beta_i^{(\ell)} := \frac{\partial f}{\partial z_i^{(\ell)}}\,, \tag{4.19}$$

where the backpropagated sensitivities $\beta^{(\ell)}$ obey $\beta^{(L+1)} = 1$ and

$$\beta_i^{(\ell)} = \phi'(z_i^{(\ell)})\,\frac{1}{\sqrt{N_\ell}}\sum_k w_{ki}^{(\ell+1)}\,\beta_k^{(\ell+1)}\,. \tag{4.20}$$

The NTK is a sum over layers,

$$\Theta^{\mu\nu} = \sum_{\ell=1}^{L+1} \sum_{i,j} \frac{\partial f^\mu}{\partial w_{ij}^{(\ell)}} \frac{\partial f^\nu}{\partial w_{ij}^{(\ell)}}$$
$$= \sum_{\ell=1}^{L+1} \underbrace{\Big[\sum_i \beta_i^{(\ell)}(x^\mu)\, \beta_i^{(\ell)}(x^\nu) \Big]}_{B_{\mu\nu}^{(\ell)}} \underbrace{\Big[\frac{1}{N_{\ell-1}} \sum_j \phi(z_j^{(\ell-1)}(x^\mu))\, \phi(z_j^{(\ell-1)}(x^\nu)) \Big]}_{\to\, \bar{K}_{\mu\nu}^{(\ell)}/\sigma_w} . \qquad (4.21)$$

The second bracket is an average of $N_{\ell-1}$ terms. At initialization and large width, the law of large numbers replaces it by $\bar{K}_{\mu\nu}^{(\ell)}/\sigma_w$, the Gaussian-process kernel of (3.30) evaluated on the two inputs, which is of order one. (For $\ell = 1$ the bracket is exactly $x^\mu \cdot x^\nu/d$, which is $\bar{K}_{\mu\nu}^{(1)}/\sigma_w$.) The first bracket is handled by the same recursion run backwards. Define

$$\dot{K}_{\mu\nu}^{(\ell)} = \dot{K}^{(\ell)}(x^\mu, x^\nu) := \mathbb{E}\big[\phi'(z_i^{(\ell-1)}(x^\mu))\, \phi'(z_i^{(\ell-1)}(x^\nu))\big] \qquad \text{(for any fixed } i\text{)}\,, \qquad (4.22)$$

which is of order one for reasonable activation functions. Using the independence at initialization of the weights $w^{(\ell+1)}$ from everything below them, one finds $B_{\mu\nu}^{(\ell)} \approx \sigma_w\, \dot{K}_{\mu\nu}^{(\ell+1)}\, B_{\mu\nu}^{(\ell+1)}$ with $B_{\mu\nu}^{(L+1)} = 1$, and hence

$$\mathbb{E}\big[\Theta^{\mu\nu}\big] \;\approx\; \sum_{\ell=1}^{L+1} \sigma_w^{L-\ell}\; \dot{K}_{\mu\nu}^{(L+1)} \cdots \dot{K}_{\mu\nu}^{(\ell+1)}\; \bar{K}_{\mu\nu}^{(\ell)} . \qquad (4.23)$$

This is a sum of $L+1$ terms, each of order one, with no factors of N anywhere: *in the fan-in normalization the NTK is of order one at large width*, and every layer contributes to it at the same order. (Equation (4.23) is the standard recursion for the infinite-width NTK [42].) By (4.10), an order-one learning rate η then moves the function values f^μ by an order-one amount per unit time, which is what we want, and no rescaling of η with width is needed in this convention. For the one-hidden-layer network $f = \frac{1}{\sqrt{N}} \sum_i c_i \phi(a_i \cdot x)$ one can check this by hand:

$$\Theta^{\mu\nu} = \frac{1}{N} \sum_{i=1}^{N} \Big[\phi(a_i \cdot x^\mu)\, \phi(a_i \cdot x^\nu) + c_i^2\, \phi'(a_i \cdot x^\mu)\, \phi'(a_i \cdot x^\nu)\; x^\mu \cdot x^\nu \Big] = O(1)\,, \qquad (4.24)$$

an average of N order-one terms. (Here and below we absorb the first-layer factor $1/\sqrt{d}$ into the inputs x, since d is fixed.)

Note

How to scale the learning rate. The rule just used is general, and we will use it again in Section 5. What must stay of order one as $N \to \infty$ is the *motion of the network values*, $\frac{d}{dt} f^\mu = -\eta\, \Theta^{\mu\nu}\, \partial \mathcal{L}^{(f)}/\partial f^\nu$, since those are what fit the data. So: work out how the NTK scales with N, and choose η so that $\eta\, \Theta = O(1)$. In the

fan-in normalization $\Theta = O(1)$ and $\eta = O(1)$. In the mean-field normalization of Section 5, $\Theta = O(1/N)$ and η must grow like N.

Other conventions for the parameters are changes of variables, and they can be compared through the same rule. Suppose one puts no prefactors in the network and initializes $\tilde{w}^{(\ell)} = w^{(\ell)}/\sqrt{N_{\ell-1}} \sim \mathcal{N}(0, \sigma_w/N_{\ell-1})$ as in (3.5). Gradient flow with respect to the Euclidean metric $\sum(d\tilde{w})^2$ is *not* the same flow as gradient flow with respect to $\sum(dw)^2$. Under the change of variables, the inverse metric δ^{ij} of Section 4.1.1 picks up a factor $N_{\ell-1}$ in every layer-ℓ block, so that in the original coordinates the flow reads $\dot{w}^{(\ell)} = -\eta\, N_{\ell-1}\, \partial\mathcal{L}/\partial w^{(\ell)}$. The NTK computed in the $\tilde{w}$ coordinates is correspondingly larger, $\tilde{\Theta} = \sum_\ell N_{\ell-1}\, \Theta_{(\ell)}$ in terms of the layer contributions of (4.21), and an order-one motion of f then requires $\tilde{\eta} \sim 1/N$. A rescaling of the parameters is thus equivalent to a layer-dependent rescaling of the learning rate, and two conventions define the same training dynamics only if their learning rates are matched layer by layer. With a *single* learning rate $\tilde{\eta} \sim 1/N$ in the $\tilde{w}$ convention, for instance, the hidden layers move as in the fan-in normalization, but the first layer, whose fan-in d does not grow with N, is effectively frozen. When comparing learning rates across papers, first translate the parametrizations into one another; for a systematic treatment, see [96].

Caution: the estimate (4.23) is at fixed depth L and fixed data size n. The NTK grows like L with depth, and the operator norm of the $n \times n$ matrix Θ grows with n, so for deep networks or large datasets the learning rate must shrink accordingly, and the constant-NTK statements below need n and L to stay small compared with N.

The Hessian is small. To see that the NTK is approximately constant during training, we consider its time derivative, which by Exercise 4.2 is bounded by the operator norm of the Hessian of the network function,

$$\left\| \frac{d}{dt}\Theta \right\| \lesssim \eta\, \|\nabla_\theta^2 f\| \cdot \|\Theta\| \cdot |\Delta|\,. \tag{4.25}$$

The Hessian is a very large but very sparse matrix, and in the fan-in normalization its operator norm is small. We'll sketch how this works for one hidden layer ($L = 1$) with one-dimensional output, following [37, Lecture 3]. Writing

$$f_\theta(x) = \frac{1}{\sqrt{N}} \sum_{i=1}^{N} c_i\, \phi(a_i \cdot x)\,, \qquad c_i \in \mathbb{R}\,, \quad a_i \in \mathbb{R}^d\,, \tag{4.26}$$

the Hessian splits into blocks, one per hidden neuron,

$$\nabla_\theta^2 f_\theta(x) = \bigoplus_{i=1}^{N} (\nabla_\theta^2 f)_i\,, \qquad (\nabla_\theta^2 f)_i = \frac{1}{\sqrt{N}} \begin{pmatrix} 0 & \phi'(a_i \cdot x)\, x^T \\ \phi'(a_i \cdot x)\, x & c_i\, \phi''(a_i \cdot x)\, x x^T \end{pmatrix}. \tag{4.27}$$

The operator norm of a block-diagonal matrix is the largest operator norm of its blocks. Each block has fixed size $(d+1) \times (d+1)$, contains only order-one quantities (at least

at initialization), and carries an explicit factor $1/\sqrt{N}$. Thus

$$\|\nabla_\theta^2 f\| = \max_{i=1,\dots,N} \|(\nabla_\theta^2 f)_i\| = O\Big(\frac{1}{\sqrt{N}}\Big) \qquad \text{as } N \to \infty\,. \tag{4.28}$$

It's reasonable to expect that this bound continues to hold during training, and with a bit more work it can be proven (though we won't do it here). It also holds for networks of arbitrary fixed depth. A more careful analysis, keeping track of cancellations among the fluctuating terms, improves the resulting bound on the kernel drift to $O(1/N)$; the crude $O(1/\sqrt{N})$ is enough for the argument.

Putting the pieces together. First, the parameters do not move far in total. From $\dot\theta = -\eta \sum_\mu \Delta^\mu \nabla_\theta f^\mu$,

$$|\dot\theta|^2 = \eta^2 \Big| \sum_{\mu=1}^{n} \Delta^\mu \nabla_\theta f^\mu \Big|^2 \le \eta^2\, |\Delta|^2\, |\nabla_\theta f|^2 \le \eta^2\, n\, |\Delta|^2\, \|\Theta\| = O(1)\,, \tag{4.29}$$

where $|\nabla_\theta f|^2 = \sum_\mu \Theta^{\mu\mu} = \operatorname{tr}\Theta$. So by the mean value theorem, $\theta(t) - \theta_0 = t\,\dot\theta(\bar t)$ for some $\bar t \in [0,t]$, and

$$|\theta(t) - \theta_0| = O(1) \qquad \text{for any fixed } t\,. \tag{4.30}$$

An order-one total displacement, shared among $P \approx Nd$ parameters, means that each individual weight moves only by $O(1/\sqrt{N})$: relative to its order-one initial value, every weight barely changes. Second, the kernel freezes. By (4.25) and (4.28),

$$\big\|\Theta(t) - \Theta(0)\big\| \le t\, \|\dot\Theta(\bar t)\| \lesssim t\,\eta\, \|\nabla_\theta^2 f\|\, \|\Theta\|\, |\Delta| = O\Big(\frac{1}{\sqrt{N}}\Big)\,, \tag{4.31}$$

so the order-one kernel Θ becomes constant as $N \to \infty$. Third, the network linearizes. By Taylor's theorem with remainder,

$$f_\theta(x) - f_{\theta_0}(x) = \nabla_\theta f(x)\big|_{\theta_0} \cdot (\theta - \theta_0) + \frac{1}{2}(\theta - \theta_0) \cdot \nabla_\theta^2 f(x)\big|_{\bar\theta} \cdot (\theta - \theta_0) \tag{4.32}$$

for some $\bar\theta$ on the segment between θ_0 and θ. Setting $\theta = \theta(t)$ and using (4.30) and (4.28), the quadratic term is bounded by

$$\Big|\frac{1}{2}(\theta - \theta_0) \cdot \nabla_\theta^2 f(x)\big|_{\bar\theta} \cdot (\theta - \theta_0)\Big| \le \frac{1}{2}\|\nabla_\theta^2 f\|\, |\theta - \theta_0|^2 = O\Big(\frac{1}{\sqrt{N}}\Big) \cdot O(1)\,, \tag{4.33}$$

so the linear approximation becomes exact as $N \to \infty$, even though the linear term, and with it the network function, moves by an order-one amount. In words: each of the N neurons has $O(1/\sqrt{N})$ leverage on the output, so an order-one change in f can be assembled from $O(1/\sqrt{N})$ changes in every weight, and changes that small leave the features $\nabla_\theta f$ untouched.

4.4 Lazy learning

When the NTK is constant during training — confusingly often called the *NTK regime* — a network undergoes what is known as "lazy learning." It is not learning features, since the eigenvalues s_μ^2 and eigenvectors u_μ of the NTK are (by construction) constant. Nevertheless, the network function converges to its training data exponentially fast: from (4.12) with constant Θ, we have

$$\Delta^\mu(t) = (e^{-\eta\, t\, \Theta})^\mu{}_\nu \Delta^\nu(0)\,, \tag{4.34}$$

where we recall that $\Delta^\mu(t) = f^\mu - y^\mu = f_{\theta(t)}(x^\mu) - y^\mu$. Writing the residuals in an eigenbasis for the NTK, $\Delta = \hat{\Delta}^\mu u_\mu$ we see that

$$\hat{\Delta}^\mu(t) = e^{-\eta\, t\, s_\mu^2} \hat{\Delta}^\mu(0)\,. \tag{4.35}$$

Each feature is learned at a characteristic timescale $t_\mu \approx 1/(\eta\, s_\mu^2)$.

Exercise 4.3 (Lazy predictions on test points). Derive that on an arbitrary test point x, not necessarily part of the training data,

$$f_{\theta(t)}(x) = f_{\theta_0}(x) - \Theta(x, x^\mu)\Theta^{-1}_{\mu\nu}(\mathbb{1} - e^{-\eta\, t\, \Theta})^\nu{}_\rho \Delta^\rho(0) \tag{4.36}$$

where $\Theta(x, x^\mu) = \nabla_\theta f(x) \cdot \nabla_\theta f(x^\mu)$ is the NTK for a test-train pair (which remains constant during training, just like the train-train NTK $\Theta^{\mu\nu}$).

From Exercise 4.3, we see that at the end of training $t \to \infty$ the network obeys

$$f_{\theta_\infty}(x) = f_{\theta_0}(x) - \Theta(x, x^\mu)\Theta^{-1}_{\mu\nu}(f_{\theta_0}(x^\nu) - y^\nu)\,. \tag{4.37}$$

This is an affine-linear transformation of the Gaussian process $f_{\theta_0}(x)$ at initialization, so it is again a Gaussian process. It's straightforward to check that the new mean and covariance are

$$\begin{aligned} \mathbb{E}\big[f_{\theta_\infty}(x)\big] &= \Theta^{x\mu}\Theta^{-1}_{\mu\nu} y^\nu \\ \mathrm{Cov}\big[f_{\theta_\infty}(x), f_{\theta_\infty}(x')\big] &= K^{xx'} - 2\Theta^{x\mu}\Theta^{-1}_{\mu\nu}K^{\nu x'} + \Theta^{x\mu}\Theta^{-1}_{\mu\nu}K^{\nu\rho}\Theta^{-1}_{\rho\sigma}\Theta^{\sigma x'}\,, \end{aligned} \tag{4.38}$$

where $K^{xx'} = K(x, x')$, $\Theta^{x\mu} = \Theta(x, x^\mu)$, etc.

4.5 Summary

So far we've seen that in the fan-in normalization (Section 3.2), with order-one weights and an order-one learning rate η, at large width N we've got

$$\begin{gathered} \|\Theta\| \sim |\nabla_\theta f|^2 \sim 1\,, \qquad \|\nabla^2_\theta f\| \sim \frac{1}{\sqrt{N}}\,, \qquad |f_\theta - f_{\theta_0}| \sim 1\,, \\ |\theta - \theta_0| \sim 1 \;\; (\text{i.e. } N^{-1/2} \text{ per weight})\,, \end{gathered} \tag{4.39}$$

which forces Θ to be constant and the network to linearize as a function of its parameters.

All this breaks down if

- Depth L is large as well, with L/N nonzero. (See [71, Ch. 11].)
- Data samples n become large, with n/N nonzero.
- Learning rate becomes large.

Both L and n large are interesting cases for modern neural networks!

Exercise 4.4 (Numerics: the lazy network (lab)). (★) This lab, and its continuation in Exercise 5.3, trains one small network in both the fan-in and the mean-field normalization and tests the claims of Sections 4 and 5 numerically. The setup runs in seconds, in about sixty lines of NumPy; reproduce it, then modify it.

The starter model. Inputs are one-dimensional: $n = 30$ equispaced training points $x^\mu \in [-2, 2]$, with targets from a two-neuron teacher, $y(x) = 2\tanh(3x) - \frac{3}{2}\tanh(x)$. The student is a one-hidden-layer tanh network

$$f(x) = \frac{1}{N^\gamma} \sum_{i=1}^{N} c_i \tanh(a_i x)\,, \qquad c_i, a_i \sim \mathcal{N}(0,1)\,, \qquad N = 300\,, \tag{4.40}$$

trained by full-batch gradient descent on $\mathcal{L} = \frac{1}{2}\sum_\mu (f(x^\mu) - y^\mu)^2$, with residuals $\Delta^\mu = f(x^\mu) - y^\mu$. Here take the fan-in exponent $\gamma = \frac{1}{2}$, learning rate $\eta = 0.02$, and 4×10^4 steps. The empirical NTK to monitor is the $n \times n$ matrix

$$\begin{aligned} \Theta^{\mu\nu} = \frac{1}{N^{2\gamma}} \sum_{i=1}^{N} \Big[&\tanh(a_i x^\mu)\tanh(a_i x^\nu) \\ &+ c_i^2 \big(1 - \tanh^2(a_i x^\mu)\big)\big(1 - \tanh^2(a_i x^\nu)\big)\, x^\mu x^\nu \Big]\,. \end{aligned} \tag{4.41}$$

(a) *It fits.* Train and check that the loss becomes small. (The network is overparametrized; fitting is not the interesting part.)

(b) *Initialization.* Over about 200 seeds, histogram $f_0(x)$ at a fixed input, say $x = 1$. You should see an order-one Gaussian, the Gaussian process of Section 3.4, and the sample covariance across two inputs should match the tanh GP kernel.

(c) *The kernel freezes.* For $N \in \{30, 100, 300, 1000\}$ (a few seeds each), record $\|\Theta_{\text{end}} - \Theta_0\|/\|\Theta_0\|$ and plot it against N on log–log axes. Expected: it decays, with slope between $-\frac{1}{2}$ (the guaranteed bound (4.31)) and -1 (the typical measurement). Also record the relative parameter displacement $|\theta_{\text{end}} - \theta_0|/|\theta_0|$ and compare with (4.30).

(d) *Mode by mode.* Diagonalize Θ_0 and plot $|(U^T\Delta)_k(t)|$ on a semilog axis for a few k: straight lines with slopes $-\eta s_k^2$, the features being learned at $t_k = 1/(\eta s_k^2)$, largest eigenvalue first, as in (4.35). Compare with the Bayesian version, Exercise 4.5(b).

(e) *Prediction.* Compare the trained function on fresh test points with the frozen-kernel prediction (4.36).

4.6 Bayesian learning at large width: kernel learning

So far, we have considered MLP's at large width, trained via gradient flow (as an approximation to SGD/GD). We found that their NTK remains constant and that their late-time distribution is a Gaussian process given by (4.38).

Suppose that we consider the same large-width network, but "train" Bayesian inference instead, on the empirical MSE loss $\mathcal{L}(\theta) = nL_n(\theta) = \frac{1}{2}\sum_{\mu=1}^{n} |f(x^\mu) - y^\mu|^2$. (Recall from Section 2.1 that, practically speaking, this means training via Langevin dynamics.) Then we expect a Bayesian posterior

$$P(\theta) = \frac{1}{Z} e^{-\beta \mathcal{L}(\theta)}\, \pi(\theta)\,, \qquad \pi(\theta) \propto e^{-\frac{1}{2\sigma_w}|\theta|^2}\,, \tag{4.42}$$

where the prior $\pi(\theta)$ is the Gaussian of order-one variance from (3.1), and the network carries the fan-in normalization of Section 3.2.

We'd like to push the distribution forward to the network function $f_\theta(x)$. At large-width, the push-forward of the prior is our familiar Gaussian process from Section 3.4; expressed via a functional integral as in Section 3.5 it becomes

$$\pi[f] = \frac{1}{Z'} e^{-\frac{1}{2}\int dx\, dx'\, f(x) K^{-1}(x,x') f(x')} \tag{4.43}$$

with $K(x,x') = \bar{K}^{(L+1)}(x,x')$ the final-layer GP kernel. Since the loss is explicitly a function of network values, we immediately get the late-time trained distribution

$$P[f] = \frac{1}{Z''}\, e^{-\frac{1}{2}\left[\beta \sum_{\mu=1}^{n} (f(x^\mu) - y^\mu)^2 + \int dx\, dx'\, f(x) K^{-1}(x,x') f(x')\right]} \tag{4.44}$$

Note that the action here is quadratic in function values, so this is still a Gaussian process!

Exercise 4.5 (Bayesian lazy learning: kernel ridge regression from the Gaussian posterior). (★) Lazy gradient flow at large width solved itself in Section 4.3: the NTK froze, and the k-th kernel eigenmode of the residual decayed as $e^{-\eta s_k^2 t}$, so that it is learned at the characteristic time $t_k = 1/(\eta s_k^2)$. Here you derive the Bayesian-learning version of the same statement, with β in the role of training time. Restricted to the training values $\mathbf{f} = (f(x^1), \ldots, f(x^n))$, the prior is Gaussian with covariance the kernel matrix $\hat{K}^{\mu\nu} := K(x^\mu, x^\nu)$, so the posterior on training values is

$$p(\mathbf{f}) \;\propto\; \exp\left[-\tfrac{1}{2}\,\mathbf{f}^T \hat{K}^{-1} \mathbf{f} - \tfrac{\beta}{2}\,|\mathbf{f} - \mathbf{y}|^2\right]. \tag{4.45}$$

(a) *Posterior mean.* The exponent is quadratic, so the posterior is Gaussian

and everything follows from completing the square. Show that the posterior mean of the training values is $\boldsymbol{\mu} = \hat{K}(\hat{K} + \beta^{-1} I_n)^{-1}\mathbf{y}$.

(b) *Eigenmodes.* Diagonalize the kernel, $\hat{K} u_k = \lambda_k u_k$, and show that mode by mode

$$u_k \cdot \boldsymbol{\mu} = \frac{\lambda_k}{\lambda_k + \beta^{-1}}\, u_k \cdot \mathbf{y} = \frac{\beta\lambda_k}{1 + \beta\lambda_k}\, u_k \cdot \mathbf{y}\,. \tag{4.46}$$

Conclude that the eigenmodes of the GP kernel at initialization are learned one by one as β grows, mode k switching on at the characteristic scale $\beta_k = 1/\lambda_k$, large-eigenvalue modes first (*spectral bias*).

(c) *Compare with gradient flow.* Place this alongside (4.35), $\hat{\Delta}^k(t) = e^{-\eta s_k^2 t}\hat{\Delta}^k(0)$, that is $t_k = 1/(\eta s_k^2)$. What is the same, and what differs? Two things to notice: the correspondence $\beta \leftrightarrow \eta t$ between the amount of data (or inverse noise) and training time; and the different shapes of the two filters, the power law $1/(1 + \beta\lambda_k)$ in β versus the exponential $e^{-\eta s_k^2 t}$ in t (compare Exercise B.5). Also: the Bayesian calculation uses the GP kernel K, the dynamical one the NTK Θ, two different frozen kernels for the same network.

(d) *Prediction.* Show that the full posterior over functions is again a Gaussian process,

$$P[f] \propto e^{-\frac{1}{2}\int dx\, dx'\, [f(x) - \mu(x)]\Sigma(x,x')^{-1}[f(x') - \mu(x')]}\,, \tag{4.47}$$

with mean and covariance

$$\mu(x) = \sum_{\mu,\nu} K(x, x^\mu)\big(\hat{K} + \beta^{-1} I_n\big)^{-1}_{\mu\nu} y^\nu\,,$$
$$\Sigma(x,x') = K(x,x') - \sum_{\mu,\nu} K(x, x^\mu)\big(\hat{K} + \beta^{-1} I_n\big)^{-1}_{\mu\nu} K(x^\nu, x')\,.$$

The posterior mean is *kernel ridge regression* with ridge β^{-1} (Appendix B.4), and $\beta \to \infty$ interpolates the data exactly. [*Hint:* the exponent is quadratic in f; complete the square, and use the Woodbury identity to relate the two natural forms of the answer.]

These equations embody what's sometimes known as *kernel regression*, with *ridge parameter* $\beta^{-1} = T$. (Appendix B.4 derives the same formula for a finite linear model, with no Bayesian input.) If the training samples are sufficiently dense, the mean can be an approximated by an integral in terms of the untrained kernel

$$\mu(x) \approx \int dx'\, dx''\, K(x,x')\big(K + (n\beta)^{-1}\mathrm{id}\big)^{-1}(x', x'')\, y(x'')\,. \tag{4.48}$$

Writing the data "truths" $y(x) = \sum y_k \varphi_k(x)$ in an eigenbasis for K, with ordered

eigenvalues $\lambda_1 \geq \lambda_2 \geq ...$, we find

$$\mu(x) = \sum_{k=1}^{\infty} \frac{\lambda_k}{\lambda_k + (n\beta)^{-1}} y_k \varphi_k(x) \approx \sum_{k \text{ s.t. } \lambda_k \gtrsim (n\beta)^{-1}} y_k \varphi_k(x) \,. \tag{4.49}$$

This is the Bayesian version of lazy learning. For given n, β, the "modes" of the data distribution with $\lambda_k \gtrsim (n\beta)^{-1}$ are learned by the network, and the rest are projected out.

Recall that in the Bayesian context, the rescaled inverse-temperature $n\beta$ should play a role somewhat analogous to *training time* in SGD/GD. We can see this explicitly here. For example, in SGD/GD lazy learning, the characteristic time to learn an *NTK mode* s_μ^2 is $t \approx 1/(\eta\, s_\mu^2)$ (see (4.35)). Correspondingly, in Bayesian lazy learning, the minimum inverse-temperature required to learn a *GP mode* is $\beta n \approx 1/\lambda_k$.

5 Mean-field scaling

Sections 3 and 4 analyzed one and the same large-width limit: the fan-in normalization (Section 3.2) with order-one weights and an order-one learning rate, and width $N \to \infty$ at fixed depth and data. At initialization the network converges to a Gaussian process; during training the NTK freezes, the network linearizes, and (by the definition of Section 4.2) no features are learned. It is tempting to conclude that infinite width *implies* laziness. The lesson of this section is that it does not, and that laziness was a consequence of a choice of *parametrization*, rather than of width. Changing the normalization of the output layer by a single extra factor of $\sqrt{N}$ produces a different, equally well-defined infinite-width limit in which the network starts at zero, individual weights move an order-one distance, the NTK evolves in time, and features are genuinely learned.

This alternative limit is called the *mean-field limit*. The name comes by analogy to a classic setup in statistical physics: an interacting particle system with weak all-to-all couplings, whose $N \to \infty$ behavior is governed by a self-consistent "mean field." We'll first explain what physicists mean by a mean-field theory (Section 5.1), then run the analogy in detail for one-hidden-layer networks (Section 5.2), see how the NTK story of Section 4 changes (Section 5.3). Finally we'll discuss some practical applications, including interpolating scalings, μP, and what this framework is actually used for (Sections 5.4 to 5.5).

The mean-field limit of one-hidden-layer networks was constructed independently, and essentially simultaneously, by four groups [56, 19, 72, 81]; our treatment draws on all of them. Pedagogical accounts may be found in [84, Ch. 20] and [37, Lecture 4].

5.1 Mean-field theories in physics

"Mean-field theory" is one of the oldest and simplest approximation schemes in statistical mechanics: *replace the fluctuating environment felt by each degree of freedom with its average (a macroscopic order parameter), then demand self-consistency upon*

recalculating this average. For most systems this is an uncontrolled approximation, though often a remarkably successful one *cf.* [89, Chapter 1]. There is, however, a family of models — those with weak, all-to-all couplings — for which it becomes exact as $N \to \infty$; and networks in mean-field scaling will turn out to be precisely such a model.

5.1.1 Warm-up: the Curie–Weiss magnet

The Curie–Weiss magnet is the Ising model of Section 2.4 on the *fully connected* graph: every spin couples to every other, with the coupling scaled down to J/N so that the total force on any one spin, and the total energy, stay of order N^0 and N^1 respectively. Setting $J = 1$, the energy (2.19) becomes

$$E_N(s) = -\frac{1}{2N}\sum_{i,j=1}^{N} s_i s_j - B\sum_{i=1}^{N} s_i = -N\Big(\frac{m^2}{2} + Bm\Big)\,, \qquad m := \frac{1}{N}\sum_i s_i\,. \tag{5.1}$$

The second equality is exact, not an approximation: on the complete graph the energy depends on the spins only through the magnetization. Compare the mean-field energy (2.22) of the lattice model, with $dJ \to 1$. Everything that was approximate in Section 2.4 is therefore exact here. The partition function reduces to the one-dimensional integral (2.25) with the same entropy $h(m)$, the saddle-point equation is

$$m = \tanh\big(\beta(m + B)\big)\,, \tag{5.2}$$

and the model has the second-order transition of Figure 4 at $\beta_c = 1$ and the first-order transition of Figure 2 in B, exactly as analyzed in Section 2.5.

The reason mean-field theory is exact here is worth stating in a form that carries over to networks. The field felt by a single spin is $-\partial E_N/\partial s_i = m + B$: spin i does not care about any individual other spin, only about the empirical average m, the "mean field." The classic argument then has three features:

1. *Each degree of freedom couples to the others only through an average.* (True by construction here.)

2. *The average barely fluctuates.* If the spins are roughly independent, the central limit theorem makes the fluctuations of m of size $O(1/\sqrt{N})$; as $N \to \infty$ each spin sees a *deterministic* effective field $m + B$.

3. *The average is fixed self-consistently.* A single spin in the field $m + B$ has $\mathbb{E}(s_i) = \tanh(\beta(m + B))$; since m was *defined* as the average spin, consistency requires (5.2).

For a finite-dimensional lattice, where each spin has a few strongly coupled neighbors, steps 1–2 are approximations. For the $1/N$ all-to-all model they become exact as $N \to \infty$: the mean field is an average of N weakly correlated terms, and the law of large numbers kills its fluctuations. Networks in mean-field scaling will turn out to be models of exactly this kind.

Exercise 5.1 (Curie–Weiss by fixed-point iteration). Solve (5.2) numerically by the fixed-point iteration $m^{(k+1)} = \tanh\big(\beta(m^{(k)} + B)\big)$, for β below and above 1 and for a few values of B, and watch the iteration converge. Which fixed points are stable? Keep this iteration in mind: we will meet its infinite-dimensional cousin below.

5.1.2 The general shape of a mean-field model

Nothing in the story above needed binary spins. More abstractly, one can take N "particles" $\theta_i \in \mathbb{R}^p$ with an extensive energy of the form

$$E_N(\theta) = \sum_{i=1}^{N} V(\theta_i) + \frac{1}{2N} \sum_{i,j=1}^{N} U(\theta_i, \theta_j)\,, \tag{5.3}$$

with a single-particle potential V and a symmetric pair interaction U, again with the characteristic $1/N$ coupling. The force felt by particle θ_i is $-\nabla_{\theta_i} E_N$. More so, for particles dominated by drag (highly viscous motion), the dynamics at zero temperature are governed by familiar gradient-descent equations

$$\dot{\theta}_i = -\nabla_{\theta_i} E_N = -\nabla V(\theta_i) - \frac{1}{N} \sum_{j=1}^{N} \nabla_1 U(\theta_i, \theta_j)\,, \tag{5.4}$$

where ∇_1 differentiates the first slot of U. Once more each particle feels only an empirical average of forces.

It is worth packaging this observation in a way that will carry over verbatim to networks. Define the *empirical measure*

$$\rho_N := \frac{1}{N} \sum_{i=1}^{N} \delta(\theta - \theta_i)\,, \tag{5.5}$$

a probability measure on $\mathbb{R}^p$ recording where the collection of N particles sit. Then (5.4) says that every particle descends one and the same effective single-particle potential, shaped by where everyone currently is:

$$\dot{\theta}_i = -\nabla_\theta \Psi(\theta_i; \rho_N)\,, \qquad \Psi(\theta; \rho) := V(\theta) + \int U(\theta, \theta')\, d\rho(\theta')\,. \tag{5.6}$$

The particles are coupled *only through* ρ_N — the exact analogue of "every spin feels only m." The same three-step argument as before then suggests that as $N \to \infty$ the empirical measure becomes deterministic and self-consistent. We will carry this out carefully, for networks, in the next subsection.

5.2 Mean-field scaling for one-hidden-layer networks

Return to the one-hidden-layer network (4.26) of Sections 3.5 and 4.3, but now normalized with $1/N$ out front instead of $1/\sqrt{N}$, with the same *order-one* weights:

$$f_\theta(x) = \frac{1}{N}\sum_{i=1}^{N} c_i\,\phi(a_i \cdot x)\,, \qquad c_i,\, a_{ij} \sim \mathcal{N}(0,1)\,, \tag{5.7}$$

with $c_i \in \mathbb{R}$, $a_i, x \in \mathbb{R}^d$ (as in (4.26), the first-layer factor $1/\sqrt{d}$ has been absorbed into x). Compared with the fan-in normalization of Sections 3 to 4, mean-field scaling suppresses the output by *one extra factor of* $1/\sqrt{N}$. The function class is identical — only the coordinates on it have changed. However, gradient flow is *not* invariant under reparametrization: as emphasized in Section 4.1.1 and in the callout of Section 4.3, gradient descent presupposes the Euclidean metric on parameter space, and rescaling coordinates rescales that metric. Two parametrizations of the same function class can (and here will) flow along completely different trajectories.

For a mean-field analysis, our N particles are going to be the N hidden neurons, with parameters

$$\theta_i = (c_i, a_i) \in \mathbb{R}^{d+1}\,, \qquad i = 1, ..., N\,. \tag{5.8}$$

Their values are encoded in the empirical measure (5.5), and the network (5.7) depends on its parameters only through ρ_N,

$$f_\theta(x) = \int c\,\phi(a \cdot x)\, d\rho_N(c,a) =: f_{\rho_N}(x)\,. \tag{5.9}$$

We think of the network value f_ρ as a linear functional of ρ. Neurons are viewed as exchangeable particles, and the measure is their state.

5.2.1 Initialization

At initialization, the summands $c_i\phi(a_i \cdot x)$ in (5.7) are i.i.d. with mean zero (since c_i is independent of a_i and $\mathbb{E}[c_i] = 0$). With the $1/N$ prefactor, the law of large numbers gives, for each fixed x,

$$f_{\theta_0}(x) = \underbrace{\mathbb{E}\big[c\,\phi(a \cdot x)\big]}_{=0} + O(1/\sqrt{N}) \xrightarrow{N\to\infty} 0\,. \tag{5.10}$$

Contrast this with Section 3.4: with the $1/\sqrt{N}$ normalization, the same sum sits exactly at the scale of its fluctuations, and the CLT produces an order-one Gaussian process. Mean-field scaling instead sits at the scale of the *mean*. The network begins training as (essentially) the zero function; there is no random-function prior to speak of, and whatever the trained network becomes is built by training rather than inherited from initialization.

5.2.2 The loss is a mean-field energy

Let's train on MSE loss over the data $(x^\mu, y^\mu)_{\mu=1}^n$, as in Section 4:

$$\mathcal{L}(\theta) = \frac{1}{2}\sum_{\mu=1}^{n}\big(f_\theta(x^\mu) - y^\mu\big)^2 . \tag{5.11}$$

Inserting (5.7) and expanding the square, we get

$$\mathcal{L}(\theta) = \text{const} + \frac{1}{N}\sum_{i=1}^{N} V(\theta_i) + \frac{1}{2N^2}\sum_{i,j=1}^{N} U(\theta_i, \theta_j) , \tag{5.12}$$

with

$$V(\theta) = -\sum_{\mu=1}^{n} y^\mu \, c\,\phi(a\cdot x^\mu) , \qquad U(\theta,\theta') = \sum_{\mu=1}^{n} c\,\phi(a\cdot x^\mu)\; c'\,\phi(a'\cdot x^\mu) . \tag{5.13}$$

Comparing (5.12) with (5.3), we see that up to an overall factor of N, the loss is precisely a mean-field energy. In particular, $E_N := N\mathcal{L}$ has the canonical normalization from (5.3) and is extensive, while the loss itself can be interpreted as the *intensive* energy per neuron.

We can also identify physical roles for the two potentials! V rewards neurons for aligning their output function $c\,\phi(a\cdot\cdot)$ with the data, playing the role of the external field B; while U is a data-weighted interaction between *pairs* of neurons that penalizes redundant features, playing the role of the spin-spin coupling.

5.2.3 Equations of motion and the mean field

Because $\partial f/\partial\theta_i = O(1/N)$ in this parametrization, raw gradients of $\mathcal{L}$ are $O(1/N)$, and gradient flow at fixed learning rate would freeze as $N\to\infty$. We must therefore scale the learning rate *up*,

$$\eta = N\,\tilde\eta , \tag{5.14}$$

with $\tilde\eta$ fixed. (Equivalently, we could run vanilla gradient flow on the extensive energy $E_N = N\mathcal{L}$.) This is the rule of Section 4.3 at work: the NTK, computed below in (5.24), is $O(1/N)$, so keeping $\frac{d}{dt}f^\mu = -\eta\,\Theta^{\mu\nu}\Delta^\nu$ of order one requires $\eta\propto N$. Note the contrast with Section 4.3: there the NTK was of order one and the learning rate needed no rescaling; here the NTK *shrinks* like $1/N$ and we scale up. Recall also the caveat from that section about changes of variables. The mean-field network could equally be written without a prefactor, $f = \sum_i \tilde c_i\phi(a_i\cdot x)$ with $\tilde c_i = c_i/N \sim \mathcal{N}(0, 1/N^2)$; but then the *same* dynamics (5.15) requires a learning rate $\tilde\eta/N$ for the readout weights $\tilde c_i$ and $N\tilde\eta$ for the hidden weights a_i — one learning rate per layer, differing by a factor N^2. Keeping all weights of order one is what lets a single η do the job, and it is the reason we adopted that convention in Section 3.2.

With (5.14), the flow $\dot\theta = -\eta\nabla_\theta\mathcal{L}$ becomes, per neuron,

$$\dot c_i = -\tilde\eta \sum_{\mu=1}^{n} \Delta^\mu\, \phi(a_i\cdot x^\mu)\,, \qquad \dot a_i = -\tilde\eta\, c_i \sum_{\mu=1}^{n} \Delta^\mu\, \phi'(a_i\cdot x^\mu)\, x^\mu\,, \tag{5.15}$$

where $\Delta^\mu = f_{\rho_N}(x^\mu) - y^\mu$ is the residual of Section 4.1. Equivalently, in the potential form (5.6),

$$\begin{aligned} \dot\theta_i &= -\tilde\eta\, \nabla_\theta\, \Psi(\theta_i;\rho_N)\,, \\ \Psi(\theta;\rho) &= V(\theta) + \int U(\theta,\theta')d\rho(\theta') = \sum_{\mu=1}^{n} \Delta^\mu_\rho\, c\,\phi(a\cdot x^\mu)\,, \end{aligned} \tag{5.16}$$

with $\Delta^\mu_\rho = f_\rho(x^\mu) - y^\mu$.

As anticipated, the velocity of neuron i depends on its own state (c_i, a_i), and on all the other neurons only through the average $f_{\rho_N} = \frac{1}{N}\sum_j c_j\phi(a_j\cdot\cdot)$, via the residual Δ^μ. The network function f is the mean field here! We arrive at a dictionary:

Curie–Weiss	one-hidden-layer network, mean-field scaling
spin $s_i \in \{\pm 1\}$	neuron $\theta_i = (c_i, a_i) \in \mathbb{R}^{d+1}$
magnetization $m = \frac{1}{N}\sum_j s_j$	network function $f_\rho = \frac{1}{N}\sum_j c_j\phi(a_j\cdot\cdot)$
external field B	data term V (the targets y^μ)
pair coupling $-\frac{1}{N}s_is_j$	neuron-neuron interaction $\frac{1}{N}U(\theta_i,\theta_j)$
effective field $m + B$	residual $\Delta^\mu_\rho = f_\rho(x^\mu) - y^\mu$
extensive energy E_N	$N\mathcal{L}$ (the loss is intensive)
zero-temperature dynamics	gradient flow
Langevin at temperature T	noisy training, Section 2.1
$m = \tanh(\beta(m+B))$	self-consistent Gibbs measure, Equation (5.22) below

Exercise 5.2 (Structure of the mean-field equations).

(a) Derive (5.12)–(5.13) and (5.15), and check that (5.16) reproduces them, using the symmetry of U.

(b) (*Balancedness.*) For a positively homogeneous activation such as ReLU, $\phi'(z)z = \phi(z)$. Show that in this case (5.15) conserves $c_i^2 - |a_i|^2$ for every neuron, exactly, along the flow. The two layers of a given neuron therefore remain coupled for all time. (On the other hand, distinct neurons will decouple from each other, as we'll see momentarily.)

(c) Observe that for MSE loss $\dot c_i$ in (5.15) does not depend on c_i (only on a_i and the mean field). Where does the linearity of f_ρ in c enter?

5.2.4 From particles to measures: the continuity equation

We now pass to the $N \to \infty$ limit, in distinct steps.

Step 1: rewrite the N coupled ODEs as one equation for the empirical measure. (This remains exact at finite N.) Suppose particles move deterministically with some velocity field, $\dot{\theta}_t = v_t(\theta_t)$, and let ρ_t denote the law of one such particle. The expectation of a smooth test function g obeys

$$\begin{aligned} \frac{d}{dt}\int g\, d\rho_t &= \frac{d}{dt}\,\mathbb{E}\big[g(\theta_t)\big] = \mathbb{E}\big[\nabla g(\theta_t)\cdot v_t(\theta_t)\big] \\ &= \int \nabla g\cdot v_t\, d\rho_t = -\int g\,\nabla\cdot(\rho_t v_t)\,, \end{aligned} \tag{5.17}$$

integrating by parts in the last step. Since g was arbitrary,

$$\partial_t\rho_t + \nabla\cdot\big(\rho_t\, v_t\big) = 0\,. \tag{5.18}$$

This is known as the *continuity equation*, and it simply expresses conservation of probability. Applied to our system, with $v_t = -\tilde{\eta}\,\nabla\Psi(\cdot\,;\rho_{N,t})$ from (5.16), the empirical measure satisfies (weakly, since it is a sum of delta functions)

$$\partial_t\rho_{N,t} = \tilde{\eta}\,\nabla\cdot\Big(\rho_{N,t}\,\nabla_\theta\Psi\big(\theta;\rho_{N,t}\big)\Big)\,. \tag{5.19}$$

An important consequence of the mean-field ($1/N$, all-to-all) coupling here is that (5.19) is a closed equation in ρ_N.

Step 2: the limit. At finite N, $\rho_{N,t}$ is a random, atomic measure. (Atomic means it's a sum of delta functions; it's random due to random initialization.) As $N\to\infty$ with i.i.d. initialization $\theta_i\sim\rho_0$, the law of large numbers suggests — and the theorems of [56, 19, 72, 81] confirm — that $\rho_{N,t}$ converges (weakly) to ρ_t, a deterministic curve of smooth probability measures solving the limiting PDE

$$\partial_t\rho_t = \tilde{\eta}\,\nabla\cdot\Big(\rho_t\,\nabla_\theta\big(V + U*\rho_t\big)\Big)\,,\quad \text{with}\quad (U*\rho)(\theta) := \int U(\theta,\theta')\,d\rho(\theta')\,. \tag{5.20}$$

There are quantitative $O(1/\sqrt{N})$ error bounds at any fixed time horizon.

5.2.5 The McKean–Vlasov equation and self-consistency

In this context, the analogue of the Curie–Weiss self-consistency (5.2), namely $m = \tanh(\beta(m+B))$, is called the McKean–Vlasov equation. To get it, focus on a single representative neuron θ_i. As $N\to\infty$, we can replace the force on it by the ensemble average:

$$\dot{\bar{\theta}}_t = -\tilde{\eta}\,\nabla_\theta\Psi\big(\bar{\theta}_t;\rho_t\big)\,,\qquad \text{subject to}\qquad \rho_t = \mathrm{Law}(\bar{\theta}_t)\,. \tag{5.21}$$

The constraint $\rho_t = \mathrm{Law}(\bar{\theta}_t)$ is the entire content of the equation: the particle is driven by a field built from a distribution, and that distribution is *its own law*. (Probabilists call (5.21) a *nonlinear* Markov process, nonlinear not in θ but in the law.) The McKean–Vlasov equation is equivalent to the continuity equation; the particle trajectories of (5.21) are the characteristic curves of the PDE (5.20).

We remark on a few structural features:

- *Propagation of chaos.* Since neurons interact only through a quantity (ρ_N, equivalently f_{ρ_N}) that becomes deterministic as $N \to \infty$, they become asymptotically *independent*: any fixed collection of k neurons converges to k i.i.d. copies of (5.21), with cross-neuron correlations of size $O(1/N)$. Thus, independence at initialization *propagates* instead of degrading [86].

- *Gradient-flow structure.* Define the $N \to \infty$ loss as a functional of the measure, $\mathcal{L}[\rho] = \frac{1}{2}\sum_\mu (f_\rho(x^\mu) - y^\mu)^2$. Since f_ρ is linear in ρ, this functional is convex in ρ. This ultimately leads to global-convergence theorems. For example, it was shown in [19] that for homogeneous activations (e.g. ReLU), if training converges at all, it converges to a *global* minimizer of the convex functional $\mathcal{L}[\rho]$.

- *Temperature.* If one trains instead with the Langevin dynamics of Section 2.1, Eqn. (2.6), at temperature $T = \beta^{-1}$, one finds convergence to a stationary distribution that satisfies [56]

$$\rho_\infty(\theta) \;\propto\; e^{-\beta\,\Psi(\theta;\rho_\infty)}\,. \tag{5.22}$$

  This is another version of the "$m = \tanh(\beta(m+B))$" consistency condition, now for noisy training.

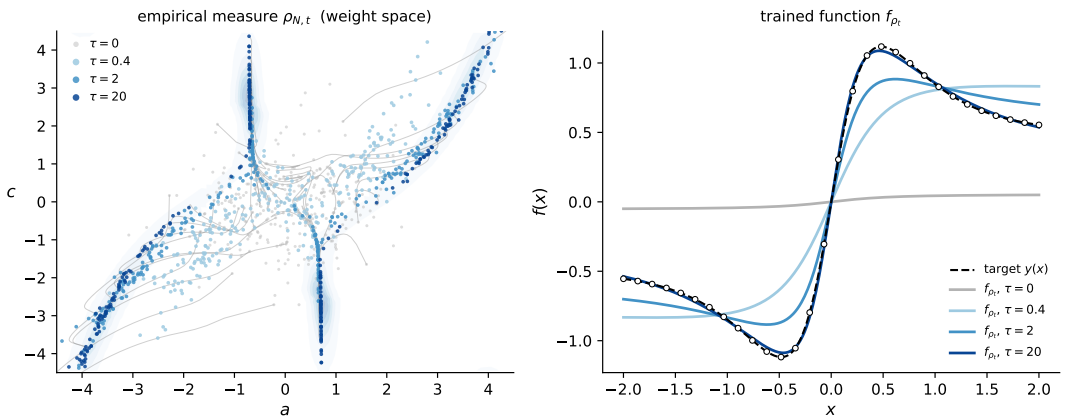


Figure 6: Mean-field training for a $d = 1$ example: $f_\rho(x) = \frac{1}{N}\sum_i c_i \tanh(a_i x)$, trained by the rescaled gradient flow (5.15) on $n = 30$ samples of the two-neuron teacher $y(x) = 2\tanh(3x) - \frac{3}{2}\tanh(x)$; $\tau = \tilde{\eta}\, t$ is rescaled training time. *Left:* the empirical measure $\rho_{N,t}$ in the (a, c) plane for $N = 300$ (dots, colored by time; thin gray lines trace 40 of the particle trajectories), drawn over the density of an $N = 30{,}000$ run at $\tau = 20$ (shaded contours), standing in for the limiting measure ρ_t. Individual neurons move order-one distances, and the measure condenses onto one-dimensional filaments — the configurations realizing the teacher. (The point symmetry $(a, c) \mapsto (-a, -c)$ reflects the oddness of tanh.) *Right:* the mean field itself, $f_{\rho_t}(x)$, converging to the target (dashed; circles mark the training data).

How would one actually *solve* the self-consistent system (5.21)? The same way one solves (5.2) (Exercise 5.1): freeze a guessed environment (ρ_t), solve the now-ordinary ODE, compute the law of the solution, feed it back, iterate. On short time horizons this map is a contraction, which is how existence, uniqueness, and propagation of chaos are all proven in one stroke [86]. Alternatively, solve the PDE (5.20) on a grid, which is feasible (only) for small d.

In Figure 6 we instead solve the system by simply training a network with $d = 1$. In the left panel, each dot is a neuron. Training is a flow of the cloud of neurons, and the cloud condenses onto the low-dimensional structures in weight space that realize the target — learning its features. Exercise 5.3 reproduces this figure, and sets it beside the frozen cloud of the lazy network.

Exercise 5.3 (Numerics: the mean-field network (lab)). (★) Continue the lab of Exercise 4.4: same data, same teacher $y(x) = 2\tanh(3x) - \frac{3}{2}\tanh(x)$, same student (4.40), but now with the mean-field exponent $\gamma = 1$ and the rescaled updates (5.15),

$$\begin{aligned} c_i &\mathrel{-}= \Delta\tau \sum_\mu \Delta^\mu \tanh(a_i x^\mu)\,, \\ a_i &\mathrel{-}= \Delta\tau\, c_i \sum_\mu \Delta^\mu \big(1 - \tanh^2(a_i x^\mu)\big) x^\mu\,, \end{aligned} \tag{5.23}$$

with $\Delta\tau = 5 \times 10^{-4}$ for 4×10^4 steps, so that the rescaled time $\tau = \tilde{\eta}\, t$ runs to 20. Note that there is no $1/N$ in these updates: that is the required $O(N)$ learning rate (5.14), already built in ($\eta = N\,\Delta\tau$). Monitor the NTK (4.41) with $\gamma = 1$.

(a) *It fits.* Train and check that the loss becomes small, if more slowly in rescaled time than the lazy run.

(b) *Initialization.* Over about 200 seeds, histogram $f_0(x)$ at $x = 1$ for several N: the distribution collapses to 0 with width $\sim 1/\sqrt{N}$. Check the scaling.

(c) *The kernel moves.* For $N \in \{30, 100, 300, 1000\}$, record $\|\Theta_{\text{end}} - \Theta_0\|/\|\Theta_0\|$ and plot it together with the lazy curve of Exercise 4.4(c): one plot, two curves, the whole dichotomy.

(d) *Watch the particles.* Scatter-plot the neuron cloud (a_i, c_i) at $\tau \approx 0,\ 0.4,\ 2,\ 20$ for both runs. Lazy: the cloud is static (each weight moves $O(1/\sqrt{N})$). Mean-field: the cloud transports and condenses onto thin filaments, the configurations realizing the teacher, as in Figure 6. Repeat at $N = 3000$: the pattern is the same, because you are looking at a deterministic limiting measure.

(e) *Now play.* The starter teacher is itself a two-neuron tanh network, so the student's features can align *exactly* with the teacher's. Is that feature

learning, or feature copying? Try teachers outside the model class and watch what the cloud does instead: $y(x) = \sin(2x)$; $y(x) = |x| - 1$; a ReLU student on the tanh teacher (and vice versa); or move to $d = 2$ with $y(x) = \tanh(x_1 + x_2)$ and plot the cloud of directions $a_i/|a_i|$. Which aspects of (c)–(d) are universal, and which were special to the aligned case?

5.3 The NTK is no longer constant: feature learning

Section 4.3 showed that in the fan-in normalization the NTK freezes as $N \to \infty$, and Section 4.2 defined feature learning as change of the NTK during training. Let's redo that computation in mean-field scaling. From (5.7), $\partial f/\partial c_i = \frac{1}{N}\phi(a_i \cdot x)$ and $\partial f/\partial a_i = \frac{1}{N} c_i \phi'(a_i \cdot x)\, x$, so the NTK of Section 4.1 is

$$\Theta(x, x') = \frac{1}{N^2} \sum_{i=1}^{N} k_{\theta_i}(x, x') = \frac{1}{N} \int k_\theta(x, x')\, d\rho_N(\theta)\,, \tag{5.24}$$

with the per-neuron kernel

$$k_\theta(x, x') := \phi(a \cdot x)\phi(a \cdot x') + c^2\, \phi'(a \cdot x)\phi'(a \cdot x')\; x \cdot x'\,. \tag{5.25}$$

Thus $\Theta = O(1/N)$, recovering the previous observation that gradients are small, which was the reason for the rescaling (5.14). The estimates of Section 4.3 also explain why the kernel is now free to move. The Hessian blocks (4.27) carry $1/N$ instead of $1/\sqrt{N}$, so $\|\nabla_\theta^2 f\| = O(1/N)$; but the learning rate is $\eta = N\tilde{\eta}$, so the bound (4.25) gives for the relative drift $\|\dot{\Theta}\|/\|\Theta\| \lesssim \eta\, \|\nabla_\theta^2 f\|\, |\Delta| = O(1)$. The bound no longer forces the kernel to freeze, and the equations of motion (5.15) will show that it does move. As $N \to \infty$, the meaningful object is $N\Theta = \int k_\theta\, d\rho_N$, and the function-space dynamics (4.10) now becomes a kernel gradient flow *with a time-dependent kernel*:

$$\frac{d}{dt} f^\mu = -\tilde{\eta}\big(N\Theta^\mu{}_\nu\big)\big|_{\rho_t}\, \Delta^\nu\,, \qquad N\Theta\big|_{\rho_t} = \int k_\theta\, d\rho_t\,. \tag{5.26}$$

At initialization the LLN makes $N\Theta \to \mathbb{E}_{\rho_0}[k_\theta]$, a deterministic kernel — the *same* limiting kernel (4.24) as in the fan-in normalization. The two limits start from the same place.

During training, however, they part ways. In Section 4.3, each parameter moved only $O(1/\sqrt{N})$, so ρ_N stayed within $O(1/\sqrt{N})$ of ρ_0 and the kernel was pinned, (4.31). Here, the equations of motion (5.15) move each particle an *order-one* distance. (Each neuron has only $O(1/N)$ leverage, so fitting the data *forces* the average neuron to move a distance $O(1)$.) Thus ρ_t transports macroscopically, and by (5.26) the kernel is dragged along at order one as well.

5.4 The scaling dial and Maximal Update Parameterization

It's quite instructive to look at an entire one-parameter family of scalings, which has fan-in/NTK scaling and mean-field scaling as its endpoints. The family can be encoded

via a prefactor

$$f_\theta(x) = \frac{1}{N^\gamma} \sum_{i=1}^{N} c_i \, \phi(a_i \cdot x) \,, \qquad \gamma \in [\tfrac{1}{2}, 1] \,, \tag{5.27}$$

where $c_i, a_{ij} \sim \mathcal{N}(0,1)$. For each γ, as $N \to \infty$, the learning rate must be simultaneously be scaled as $\eta = N^{2\gamma-1}\tilde{\eta}$ so that the function-space dynamics stays $O(1)$ (one can derive that from $\Theta \sim N^{1-2\gamma}$). Repeating the estimates above gives (see Exercise 5.4):

	f_{θ_0} at init	per-neuron motion	limit
$\gamma = \frac{1}{2}$	$O(1)$ (GP, CLT)	$O(N^{-1/2})$	lazy: Sections 3 and 4
$\frac{1}{2} < \gamma < 1$	$O(N^{\frac{1}{2}-\gamma}) \to 0$	$O(N^{\gamma-1}) \to 0$	lazy (frozen kernel, zero init)
$\gamma = 1$	$O(N^{-1/2}) \to 0$	$O(1)$	mean-field: this section

Notably, every intermediate γ collapses onto the kernel limit: the initialization GP is scaled away, but the neurons still move vanishingly little, so training is again linear in an (appropriately rescaled) frozen kernel [20, 56]. Mean-field, $\gamma = 1$, is the *unique* scaling in this family with both a well-defined limit and order-one feature movement; it's the "maximal update" that one can take without blowing up. This uniqueness can be generalized from one hidden layer to arbitrary depth (where consistency forces *layer-dependent* learning-rate and initialization scalings), and is known as the Maximal Update Parametrization, μP, in Yang and Hu's line of work on tensor programs [96]. In particular, mean-field scaling is μP for $L = 1$.

Exercise 5.4 (Lazy vs. rich along the dial). For the family (5.27):

(a) show $\Theta = O(N^{1-2\gamma})$ and hence that $\eta = N^{2\gamma-1}\tilde{\eta}$ is the right co-scaling;

(b) show the initialization output is $O(N^{1/2-\gamma})$ and per-neuron velocities are $O(N^{\gamma-1})$;

(c) conclude that the (normalized) kernel drift over any fixed training horizon is $O(N^{\gamma-1})$, vanishing for every $\gamma < 1$;

(d) reconcile $\gamma = \frac{1}{2}$ with the bounds of Section 4.3. Optional: relate γ to the output-scale parameter α of [20], where multiplying any model by $\alpha \to \infty$ produces lazy training.

5.5 What is this framework good for?

Nobody solves the McKean–Vlasov PDE (5.20) for a real network! There are practical payoffs, though, of three different kinds: *scaling prescriptions*, *mechanism identification*, and *reduced descriptions* that downstream theories build on.

Hyperparameter transfer (μP) is one key application. The mean-field analysis is, fundamentally, a theory of which joint scalings of initialization and learning rate keep the training dynamics nondegenerate and learning features as $N \to \infty$. Its deep-network generalization [96] turns this into an engineering recipe, μTransfer [97]:

parametrize the network so that its infinite-width limit is the feature-learning one, and then optimal hyperparameters become approximately width-independent, so one can tune a small proxy model and *transfer* the settings to the large one. In the original demonstration this retuned GPT-3 6.7B at a few percent of pretraining cost and outperformed the published model; variants of this parametrization are now standard in large-scale training.

The dial of Section 5.4 also lets one interpolate between regimes on real architectures, and measure where feature learning actually pays off. Kernel-regime behavior tends to be competitive on small datasets, while on CIFAR-scale image tasks trained networks beat their own frozen NTKs by large margins [30]. Diagnostics from this circle of ideas (kernel distance/alignment during training, e.g. the silent-alignment effect [4]) are now routine tools for asking "is my network actually learning features, or just doing kernel regression?"

The mean-field limit here holds data fixed while sending $N \to \infty$, which unfortunately does not yet make it a quantitative theory of generalization at realistic data/width ratios. The dynamical mean-field theory of Section 6 [12] will rectify this somewhat; it is a field-theoretic refinement of the current analysis, whose extensions do reproduce compute-optimal neural scaling laws, in particular at large dataset size [9].

5.6 Grokking as a lazy-to-rich transition: a reading exercise

We finish this section with two guided readings that apply various concepts introduced above to one of the most interesting and fundamental phenomenons in learning: grokking [67]. Grokking refers to the following behavior. One trains a small network on an algorithmic dataset — the canonical example is addition mod P, presented as pairs (n, m) with label $(n + m) \bmod P$, with only a fraction of all P^2 facts shown. The network reaches perfect *training* accuracy quickly, while *test* accuracy sits at chance: it has simply memorized. Then, orders of magnitude later in training and long after the train loss has flattened, test accuracy jumps abruptly to near 100%. Such delayed, abrupt generalization, with emergent capabilities, demands a mechanism.

Theory has produced (at least) two quantitative explanations, and this subsection and the next walk through one each: the *dynamical* lens of Kumar–Bordelon–Gershman–Pehlevan [49] (this subsection) — grokking as a delayed transition from lazy to rich dynamics — and the *equilibrium* lens of Rubin–Seroussi–Ringel [73] (Section 5.7) — grokking as a first-order phase transition with metastable lag. The two walkthroughs are parallel in structure and use largely disjoint toolkits from these notes. The dynamical one presumes comfort with Section 4 and Section 5.4; the Bayesian one is a denser read, and combines scaling analysis with a mean-field approximation.

The work of Kumar, Bordelon, Gershman, and Pehlevan [49] invokes no statistical-mechanics ensemble at all. There is only gradient descent, and the lazy/rich dial of Section 5.4, *unfolding in time*. The paper claims that a network groks when 1) it first fits the training data through its cheap, fast channel (kernel regression with the frozen NTK of its initialization), and that lazy fit memorizes without generalizing because the initial kernel is misaligned with the task; then 2) on a slower timescale controlled by

the network's laziness, the features finally move (the kernel rotates, as in Section 5.3), a generalizing solution is found, and test loss falls long after train loss did. Two beautiful features of this work are that its clearest examples grok under vanilla gradient descent with zero weight decay, and that the parameter norm increases during grokking. Both are direct counterexamples to earlier accounts of grokking built on weight decay and shrinking norms.

The paper is written almost entirely in the language these notes have already built (Sections 4 and 5), so the dialect box below is short. Read the introduction, the two-layer polynomial-regression model, and the experiments (modular arithmetic, MNIST, a one-layer transformer); skip the dynamical mean-field asymptotics on a first pass — that machinery belongs to Section 6. Then work through Exercise 5.5.

Note

A few translations of terminology from [49]:

- *Lazy / linearized dynamics*: the regime of Section 4.3 — the network moves only in the affine subspace (4.1) spanned by its initial gradients, so training is kernel regression with the frozen initial NTK (Sections 4.1 and 4.6).
- *Laziness parameter* α: an overall multiplier on the network output, $f \mapsto \alpha f$ (with loss and learning rate co-scaled). This is the continuous version of the γ-dial of Section 5.4, in the form introduced by [20]: $\alpha \to \infty$ is fully lazy, small α is rich.
- *Kernel–task (mis)alignment*: the language of Section 4.2 — how much of the target lies in the top eigenmodes of the initial NTK. Misaligned = the target's weight sits in small-eigenvalue modes, which the mode-filtering formula of Section 4.6 says are not learnable from the available data.
- *Sufficient statistics*: a handful of scalar functions of the weights whose closed dynamics determine the test loss — the dynamical analogue of the order parameters of Section 2.

Exercise 5.5 (Grokking as a lazy-to-rich transition). Read [49] as assigned above, and answer the following, citing where in the paper each answer lives. (Parts (a)–(e) have exact counterparts in Exercise 5.6, the other walkthrough; part (f) is the shared comparison question.)

(a) *What regime is the analysis set in, and what dial controls it?* There is no thermodynamic limit and no equilibrium ensemble here, so what plays the role that a "scaling limit" plays elsewhere in this section? How is the laziness parameter α implemented, and how does it relate to the γ-dial of Section 5.4? Section 5.2's scaling prescription had two halves, an output scale and a learning-rate scale. Which half is doing the work here? (The

other walkthrough meets the opposite half: see Exercise 5.6(a).)

(b) *What plays the role of the order parameter?* What quantity distinguishes the memorized state from the generalizing one, and how would you measure it in a real training run? Compare with the Fourier overlap $\Phi(w)$ that plays this role in the other walkthrough (Exercise 5.6(b)): what information do the two quantities share?

(c) *Necessary conditions.* The paper states three conditions for grokking to occur. Find them, and translate each into the language of these notes (Sections 4.3, 4.6 and 5.4).

(d) *What parameter is tuned?* What makes grokking more dramatic, and what makes it vanish entirely? What does the theory predict in the two extremes $\alpha \to \infty$ and α small? Contrast with the control parameters of Exercise 5.6(d).

(e) *Where does the delay come from?* Both theories must explain the gap between the train-loss drop and the test-loss jump. Here there is no metastability and no noise — so what produces two well-separated timescales in a single deterministic flow? What does each theory *require* to produce a delay (noise? weight decay? neither?), and how do the paper's zero-weight-decay, growing-norm experiments bear on earlier explanations of grokking?

(f) *Synthesis.* Are the equilibrium mechanism of Section 5.7 and the dynamical mechanism of this section incompatible, or two corners of one phenomenon? Design at least one experiment whose outcome would favor one over the other — consider, e.g., how the grokking delay should depend on training noise at fixed hyperparameters, or how the delay should fluctuate from seed to seed, under each mechanism.

5.7 Bayesian grokking: a first-order transition (alternative reading)

The alternative walkthrough takes the opposite starting point. Delayed, abrupt generalization should by now also trigger a reflex from Section 2.5: competing branches of a free energy, a first-order transition, and a metastable lag, exactly as dissected for the Ising perceptron in Section 2.6. The paper [73] makes this reflex precise, and it is a fitting capstone for these notes: it assembles, in one real (if small) deep-learning phenomenon, nearly every tool we have developed: the Gibbs posterior of noisy training (Section 2.1), GP kernels and kernel regression (Sections 3.3 and 4.6), mean-field scaling and self-consistency (this section), and the anatomy of first-order transitions (Section 2.5). It is, however, a genuinely dense paper, written in an idiosyncratic dialect of the "adaptive kernel" school [76]. This subsection is therefore structured as a reading exercise in the style of Section 1: the goal is to *recognize the structures*, not to

verify the computations. Read the paper's introduction, setup, and modular-arithmetic results (*do not* read the appendices on a first pass), then work through Exercise 5.6.

Warm-up. Before any statistical mechanics, it helps to know concretely what the network is trying to become. Gromov [34] writes down *exact* weights for a two-layer network with quadratic activation that solve modular addition: the first-layer weights are plane waves on $\mathbb{Z}_P$ — entries $\cos(2\pi kn/P)$, $\sin(2\pi kn/P)$ at various frequencies k — and the quadratic activation's product-to-sum identities convert them into the required functions of $(n+m)$ mod P. Skim [34] just long enough to believe this. The lesson is that "the Fourier coefficients of the first-layer weights" becomes the obvious coordinate system for measuring how much structure a network has actually learned.

Note

Some translations from the paper's language into that of these notes:

- *Langevin training at equilibrium*: as in Section 2.1 verbatim: noisy gradient descent with weight decay; at late times, the Gibbs posterior; the weight decay γ sets the Gaussian prior and the noise level sets the temperature.
- *(Adaptive) kernel*: the GP kernels of Section 3.3, but evaluated in the *posterior* rather than the prior, self-consistently: the kernel determines the posterior over hidden weights, which determines the kernel. This is a finite-temperature sibling of the self-consistent Gibbs measure (5.22).
- *Equivalent kernel (EK) approximation*: the dense-data approximation of Section 4.6, where sums over training points are replaced by integrals over the input distribution. This is a cousin of an annealed approximation.
- *Mean-field scaling*: the readout starts with variance σ_a^2/N, and $\sigma_a^2 \to 0$ as $N \to \infty$; how this relates to the $1/N$ of (5.7) is Exercise 5.6(a).
- *GFL ("Gaussian feature learning") phase*: refers to a disordered phase, in which the single-neuron posterior is a Gaussian, centered at zero overlap with the teacher's structure; the kernel adapts only perturbatively.
- *Mixed phase*: a finite fraction of neurons shift to nonzero overlap (here: onto Fourier modes of the task) while the rest remain Gaussian; this *coexistence* is a signal of a first-order transition, cf. Section 2.6.

Exercise 5.6 (Grokking as a first-order phase transition). Read [73] as assigned above, and answer the following, citing where in the paper each answer lives. (Parts (a)–(e) have exact counterparts in Exercise 5.5, the other walkthrough; its part (f) is the shared comparison question.)

(a) *What scaling limit is taken?* Look for where the output-layer weight decay is fixed, and for the phrase "mean-field scaling." How does the paper's

scaling relate to the parametrization (5.7) and to the γ-dial of Section 5.4? Careful — this is subtler than it looks: in an *equilibrium* (Bayesian) framework there is no learning rate, so which part of Section 5.2's scaling prescription survives, and in what combination?

(b) *What is the order parameter* for the modular-addition task? What does the posterior distribution of a *single neuron's* input weights look like on each side of the transition, and which figure of Section 5.2 does the ordered side resemble?

(c) *What are the main computational techniques?* List them, and match each to the section of these notes where its ancestor appears.

(d) *What parameter is tuned* to cross the phase transition in the paper's phase diagrams?

(e) *Where did the time go?* Grokking is a statement about training *time*, yet the paper computes a static equilibrium ensemble. How does a first-order transition in an equilibrium free energy produce *delayed, abrupt* generalization in a training run? Compare, line by line, with the metastability discussion for the Ising perceptron in Section 2.6.

6 Non-equilibrium physics, MSRJD, and DMFT

So far our statistical-mechanics toolkit has been an equilibrium one: Gibbs measures, partition functions, and free energies (Section 2), together with limits in which training simplifies to kernel regression (Section 4) or to a deterministic flow of measures (Section 5). This section introduces the formalism that confronts the *dynamics* of learning head-on. The strategy is borrowed from non-equilibrium statistical physics: write the stochastic dynamics itself as a path integral — the Martin–Siggia–Rose–Janssen–De Dominicis (MSRJD) construction — then average over the randomness (noise, initialization, and, crucially, the training data, treated as quenched disorder exactly as in Section 2.3), and finally extract closed, self-consistent equations for a small set of *two-time* order parameters: correlation functions $C(t, t')$ and response functions $R(t, t')$. The result is known as dynamical mean-field theory (DMFT), and it is the third and final incarnation of "mean field" in these notes: the statics of Section 2 determined scalars self-consistently, the McKean–Vlasov limit of Section 5 determined a measure, and DMFT determines whole functions of two times.

The payoffs in this section are a complete worked application — double descent as a *dynamical phase transition* in linear regression (Section 6.2) — and a guided tour of the modern DMFT program for deep learning (Section 6.3), whose scaling-law application closes the notes in Section 7. Throughout, we follow the spirit of Altland and Simons [2], which motivates the formalism in condensed-matter language with a good balance of detail and intuition, and we borrow freely from two pedagogical tutorials written for

neuroscientists, [22] and [85].

6.1 From stochastic dynamics to path integrals: the MSRJD formalism

When we model a system with fluctuations — or deliberately add noise to a deterministic model, as in the Langevin dynamics of Section 2.1 — differential equations become stochastic, and the difficulty of solving them jumps. A **stochastic differential equation**, at first order for example, takes the form

$$\frac{dx}{dt} = f(x) + \xi\,, \tag{6.1}$$

where $\xi(t)$ is a (Gaussian) white noise process — additive noise, $\xi = \xi(t)$, as opposed to multiplicative noise $\xi = \xi(x,t)$ — and $x = x(t)$ is now a random trajectory. The noise is entirely defined by the first and second moments,

$$\langle \xi(t) \rangle_\xi = 0, \qquad \big\langle \xi(t)\xi(t') \big\rangle_\xi = D\delta(t-t')\,, \tag{6.2}$$

where the expectations are with respect to the noise ensemble[3]. Obtaining closed-form solutions for the time evolution of the probability distribution for $x(t)$ is usually challenging. These evolutions can be attacked by the traditional routes — the Langevin and Fokker–Planck descriptions — for which the standard references are Risken [70], Gardiner [29], and van Kampen [91]. The objects of interest here are instead the moments of $x(t)$ and the probability density function $p(x,t)$. Methods developed in statistical physics (in particular those used in perturbative quantum field theory and classical non-equilibrium systems) using path integrals provide a way to calculate these statistical objects. The path integral, or functional integral, approach is convenient for perturbative calculations (expanding around an exactly solvable theory with a small parameter), but also permits importing other tools like renormalization group methods or other explicitly non-perturbative approaches when there is no small parameter, or the expansion breaks down (as they often do).

The conceptual recipe of the MSRJD formalism is short, and worth internalizing before the derivation:

- write down a *discrete* SDE, fixing a convention (Itô) that enforces causality;
- express the transition probability between time steps as a delta function imposing the update rule;
- assemble the product over infinitesimal steps into a path integral;
- write each delta function in Fourier representation, introducing an auxiliary *response field*;

[3] As a point of notation, η and ζ are also commonly used for noise, and we will distinguish between discrete and continuum delta functions as δ_{ij} and $\delta(t-t')$ respectively.

- average over the noise, then add sources and differentiate to extract statistics.

It turns out that the historic approach of Onsager and Machlup [63] is the "Lagrangian" version of this "Hamiltonian"-like approach; the two are related by a Legendre transform, or equivalently by integrating out the auxiliary field. That auxiliary field is aptly named the **response field**: as we will see below, its correlation functions compute how the system *responds* to external perturbations, and it is the genuinely new ingredient of the non-equilibrium formalism.

We now give the derivation, quickly but completely. The standard first step is to discretize time for a continuous equation, or simply start with a discrete update rule. Under this discretization $x(t) \to x_i$, where $i = 1, ..., N$, so that,

$$x_i = x_{i-1} + \Delta t\Big(f(x_{i-1}) + \xi_{i-1}\Big)\,, \tag{6.3}$$

where Δt is the temporal spacing. We can rearrange this so that the RHS is zero and write the transition probability between steps as delta function, and the product of these representing a trajectory solution. Defining a solution of the stochastic differential equation as $x[\xi]$, an expectation value of an observable $\mathcal{A}[x]$ is formally,

$$\langle \mathcal{A}[x]\rangle_\xi = \int \mathcal{D}x \mathcal{A}[x]\,\langle \delta(x - x[\xi])\rangle_\xi = \int \mathcal{D}x \mathcal{A}[x]\left|\frac{\delta X}{\delta x}\right|\delta(X)\,. \tag{6.4}$$

We used the shorthand $X_i \equiv x_i - x_{i-1} - \Delta t\big[f(x_{i-1}) + \xi_{i-1}\big]$ for the constraint imposed at each step. The measure here, is a typical functional path integral measure $\mathcal{D}x = \prod_i dx_i$ with $\delta(x - x[\xi])$ similarly factorized across steps. In particular we also picked a first order finite difference and the convention that the noise is evaluated at the previous time-step, which is called Itô discretization or convention. Many common processes are manifestly Itô, like SGD for example, where the noise drawn at a batch doesn't affect the weights at that iteration. There are other conventions, the most common alternative is Stratonovich (which is more like a midpoint rule), however, Itô has a unit (volume-preserving) functional determinant and other schemes require careful treatment to avoid anomalous terms from the determinant. In our case the matrix is triangular with a unit diagonal. Substituting in the definition of X,

$$\langle \mathcal{A}[x]\rangle_\xi = \int \mathcal{D}x\, \mathcal{A}[x]\, \big\langle \delta\big(\partial_t x - f(x) - \xi\big)\big\rangle_\xi\,. \tag{6.5}$$

Finally, we can use the Fourier representation of the delta, introducing an auxiliary variable $\tilde{x}$, which in the continuum is

$$\langle \mathcal{A}[x]\rangle_\xi = \int \mathcal{D}x\, \mathcal{D}\tilde{x}\, \mathcal{A}[x]\, e^{\,i\int dt\, \tilde{x}\,(\partial_t x - f(x) - \xi)}\,. \tag{6.6}$$

The final step is the payoff: the noise now appears *linearly* in the exponent, so the Gaussian average over ξ can be done exactly, $\big\langle e^{-i\int dt\,\tilde{x}\,\xi}\big\rangle_\xi = e^{-\frac{D}{2}\int dt\,\tilde{x}^2}$, leaving the

MSRJD functional integral

$$\begin{aligned} \langle \mathcal{A}[x] \rangle &= \int \mathcal{D}x\, \mathcal{D}\tilde{x}\; \mathcal{A}[x]\; e^{-S[x,\tilde{x}]}\,, \\ S[x,\tilde{x}] &= \int dt \left[-\,i\,\tilde{x}\,\big(\partial_t x - f(x)\big) + \frac{D}{2}\,\tilde{x}^2 \right]. \end{aligned} \tag{6.7}$$

Four structural comments, each of which does real work later:

- *The partition function is one.* Setting $\mathcal{A} = 1$, the integral is just $\int \mathcal{D}x$ of a normalized probability: $Z_{\rm dyn} = 1$, with no sources. Unlike in equilibrium, all the physics lives in correlation functions, obtained by coupling sources to both x and $\tilde{x}$ and differentiating — compare the $Z_{\rm dyn} = 1$ starting point of Section 6.2.

- *The response field earns its name.* Perturbing the drift, $f \to f + h(t)$, changes the action by $i \int \tilde{x}\, h$, so

$$R(t,t') := \left.\frac{\delta \langle x(t) \rangle}{\delta h(t')}\right|_{h=0} = \big\langle x(t)\, i\tilde{x}(t') \big\rangle\,. \tag{6.8}$$

  Correlators of $\tilde{x}$ compute responses to kicks. Causality is built in: $R(t,t') = 0$ for $t < t'$ (the Itô discretization is what guarantees this, along with $\langle \tilde{x}\,\tilde{x} \rangle = 0$).

- *Two-time observables.* The natural objects of the formalism are the correlation function $C(t,t') = \langle x(t)x(t') \rangle$ and the response $R(t,t')$. In *equilibrium* they are not independent: the fluctuation–dissipation theorem (FDT) locks them together, $T\,R(t,t') = -\partial_{t'} C(t,t')$ for $t > t'$ (with $T = D/2$ in our normalization) [48, 47]. A measured *violation* of the FDT is therefore the standard diagnostic that a system is genuinely out of equilibrium — the warning light used to great effect in Section 6.2.

- *Onsager–Machlup.* The action (6.7) is quadratic in $\tilde{x}$, so the response field can be integrated out exactly, giving the probability of a trajectory as $P[x] \propto e^{-\frac{1}{2D}\int dt\,(\partial_t x - f(x))^2}$: the Lagrangian form promised above, in which the most probable path extremizes a classical action.

Exercise 6.1 (The Ornstein–Uhlenbeck process and the FDT).

(a) Fill in the noise average leading to (6.7).

(b) For the linear drift $f(x) = -kx$ (the Ornstein–Uhlenbeck process), compute the stationary correlation and response functions, either by Fourier transforming the quadratic action or by solving the SDE directly: $C(\tau) = \frac{D}{2k} e^{-k|\tau|}$ and $R(\tau) = \theta(\tau)\, e^{-k\tau}$.

(c) Verify the fluctuation–dissipation theorem $TR(\tau) = -\partial_\tau C(\tau)$ for $\tau > 0$, with $T = D/2$.

(d) Explain, from the discrete-time construction, why $\langle \tilde{x}(t)\tilde{x}(t')\rangle = 0$ and why R is causal.

6.2 Double descent as a dynamical phase transition

As a complete worked application of the formalism, we describe recent work of Li and Goldenfeld [53], which uses dynamical mean-field theory to analyze the double-descent phenomenon and classifies it as a *dynamical phase transition*, all within the simplest possible setting: a teacher–student model of linear regression. Even this very simple model requires a long calculation, but the paper is quite readable despite the overhead. The headline: in the under-parametrized regime the optimization process is ergodic and relaxes to a finite-error solution, while the over-parametrized regime is a *non-ergodic* phase with genuinely non-equilibrium dynamics — an initial memorization stage leaves a persistent memory of the initial conditions and confines the dynamics to a subspace around the initialization. The diagnostic is the *response function* $R(t, t')$ of Section 6.1 — how a perturbation at time t' affects the model's state at time t; the same object appears as a Green function in quantum mechanics, signal analysis, and information theory. In the under-parametrized regime the response function decays: no memory. In the over-parametrized regime it saturates at a finite value: persistent memory of initialization. Consistently, the first regime obeys the fluctuation–dissipation theorem [48, 47, 33], while in the second the FDT breaks — the standard warning light, flagged in Section 6.1, that a system has gone out of equilibrium. The model here is quite simple, and the authors propose that the frameworks here may generalize across architectures, with different universality classes based on the variations of the task, proposing random feature models with multiplicative (state-dependent) noise as an extension, with the hopes of eventually attaining an RG framework for interpreting phase transitions in neural networks.

6.2.1 Model definition and formalism

Notation. We transcribe the paper into the conventions of these notes: P trainable parameters, n training samples, learning rate η, teacher weights w^*, student weights $\hat{w}$ (with response fields $\tilde{w}$), and the *overparametrization ratio*

$$\nu := P/n \tag{6.9}$$

as control parameter — the reciprocal of Section 2's load $\alpha = n/N$, so that the over-parametrized phase is $\nu > 1$. For readers consulting the paper, the dictionary is:

these notes	Li–Goldenfeld [53]
P (parameters)	d
n (samples)	N
$\nu = P/n$	$\alpha = d/N$
w^*, $\hat{w}$, $\tilde{w}$	β, $\hat{\beta}$, $\tilde{\beta}$
η (learning rate)	ζ

The batch-inclusion probability γ, the ridge λ, and the label-noise variance D are as in the paper.

Explicitly, the student and teacher share the same architecture

$$y^\mu = \frac{1}{\sqrt{P}} \sum_{i=1}^{P} w_i^* x_i^\mu + \epsilon^\mu \tag{6.10}$$

where P is the number of model parameters, $\mu = 1, \ldots, n$ indexes the dataset with inputs x^μ, the w_i^* are the teacher's weights, and ϵ^μ is label noise. The thermodynamic limit takes $n \to \infty$ and $P \to \infty$ with the overparametrization ratio $\nu = P/n$ — the paper's **complexity parameter** — held *fixed*. The three main regimes are now definable as under-parametrized: $\nu < 1$, near-criticality: $\nu \approx 1$, and over-parametrized: $\nu > 1$. We note their optimization procedure, which is SGD with both regularization controlled by λ (set to zero for most of the work) and batching controlled by the binary indicator variable s_μ which is drawn for each point in the dataset with probability γ,

$$\hat{w}_i(t+1) = \hat{w}_i(t) - \eta \frac{\partial \left[\frac{1}{2} \sum_{\mu=1}^{n} s^\mu \left(\hat{y}^\mu - y^\mu \right)^2 + \frac{\lambda}{2} \|\hat{\boldsymbol{w}}\|_2^2 \right]}{\partial \hat{w}_i(t)} , \tag{6.11}$$

From here they construct the MSRJD integral exactly as in Section 6.1; taking functional derivatives and averaging over the disorder (the dataset, the initialization, the batch selection, and the label noise) yields the correlation and response functions that are the main objects of study. We skip most of the details, but it is worth displaying the starting point — the dynamical partition function for gradient flow, equal to one just as promised in Section 6.1:

$$\begin{aligned} Z_{\text{dyn}} = \int \mathcal{D}\hat{\boldsymbol{w}}(t) \prod_{i=1}^{d} \delta \Big[& - \partial_t \hat{w}_i(t) - \lambda \hat{w}_i(t) + j_i(t) \\ & - \sum_{\mu=1}^{n} s^\mu(t) \frac{x_i^\mu}{\sqrt{d}} \Big(\sum_{k=1}^{P} \frac{x_k^\mu}{\sqrt{d}} \big(\hat{w}_k(t) - w_k^* \big) - \epsilon^\mu \Big) \Big] = 1 \, . \end{aligned} \tag{6.12}$$

We want to highlight the general structure of this equation as a path integral over an action with sources coupled to both the "physical" parameter fields and the response fields,

$$\begin{aligned} Z_{\text{dyn}}[\mathbf{j}, \tilde{\mathbf{j}}] = \Big\langle \int \mathcal{D}\hat{\boldsymbol{w}}(t) \mathcal{D}\tilde{\boldsymbol{w}}(t) \exp \Big(S_{\text{dyn}} & + \sum_{i=1}^{P} \int \tilde{j}_i(t) \hat{w}_i(t) dt \\ & + \sum_{i=1}^{n} \int j_i(t) \tilde{w}_i(t) dt \Big) \Big\rangle , \end{aligned} \tag{6.13}$$

which is a very common object to see in statistical field theory! In general S_{dyn} will hold all of the microscopic details of the theory, and this is a unique case with a fairly tractable model. In general we are not so lucky. The expectation value denoted by angled brackets is averaging over the input dataset $\{\mathbf{x}^\mu\}_{\mu=1}^n$, the initial condition $\hat{\boldsymbol{w}}(0)$,

the stochastic sampling $s_\mu(t)$, and the added label noise $\{\epsilon^\mu\}_{\mu=1}^n$. For comedic relief we also write their full dynamical function integrated over quenched disorder:

$$\begin{aligned} Z_{\text{dyn}} \propto \int \mathcal{D}\boldsymbol{\mathcal{B}}(t) \int \mathcal{D}\mathcal{Q}(t,t') \exp\Bigg(& - i\tilde{\boldsymbol{w}}\partial_t \hat{\boldsymbol{w}} - \lambda P\, Q_2(t,t) \\ & - P\, Q_1 \cdot \hat{Q}_1 - P\, Q_2 \cdot \hat{Q}_2 - P\, Q_3 \cdot \hat{Q}_3 - P\, Q_4 \cdot \hat{Q}_4 \\ & - \frac{n}{2} \log\det \begin{pmatrix} \gamma Q_2^\top - \gamma Q_4 + \delta & \gamma^2 Q_3 \\ Q_1 + D & \gamma Q_2 - \gamma Q_4^\top + \delta \end{pmatrix} + \tilde{\boldsymbol{j}}\hat{\boldsymbol{w}} + \boldsymbol{j}\tilde{\boldsymbol{w}} \\ & + \iint dt\, dt' \sum_k \Big[\big(\hat{w}_k(t) - w_k^*\big) \hat{Q}_1(t,t') \big(\hat{w}_k(t') - w_k^*\big) \\ & \qquad + i\hat{w}_k(t) \hat{Q}_2(t,t') \tilde{w}_k(t') + i\tilde{w}_k(t) \hat{Q}_3(t,t')\, i\tilde{w}_k(t') \Big] \\ & + \int dt \sum_k i\tilde{w}_k(t) \hat{Q}_4(t) w_k^* \Bigg), \end{aligned} \tag{6.14}$$

where $\mathcal{D}\boldsymbol{\mathcal{B}}(t)$ abbreviates the measure $\mathcal{D}\hat{\boldsymbol{w}}(t)\, \mathcal{D}\tilde{\boldsymbol{w}}(t)$ over the dynamical fields, and the new measure $\mathcal{D}\mathcal{Q}(t,t')$ is

$$\begin{aligned} \mathcal{D}\mathcal{Q}(t,t') = \left(\frac{P}{2\pi}\right)^4 & \mathcal{D}Q_1(t,t')\, \mathcal{D}\hat{Q}_1(t,t')\, \mathcal{D}Q_2(t,t')\, \mathcal{D}\hat{Q}_2(t,t') \\ & \times \mathcal{D}Q_3(t,t')\, \mathcal{D}\hat{Q}_3(t,t')\, \mathcal{D}Q_4(t)\, \mathcal{D}\hat{Q}_4(t)\,. \end{aligned} \tag{6.15}$$

Each of the $\mathcal{Q}$ variables represents a particular correlation function between two variables.

6.2.2 Overview of results via training and generalization error

They define the generalization error as a mean squared error (MSE)

$$\varepsilon_g(t) = \langle \left(\frac{1}{\sqrt{P}} \sum_{i=1}^P x_i^0 [\hat{w}_i(t) - w_i^*] - \epsilon^0 \right)^2 \rangle \tag{6.16}$$

After taking the thermodynamic limit, the generalization error is expressed in terms of the equal time correlator $C(t,t)$ determined from the DMFT equations and D, the noise variance:

$$\varepsilon_g(t) = C(t,t) + D \tag{6.17}$$

They find two forms of this equation and propose a third, a scaling relation near the critical point. At $\lambda = 0$ for $\nu < 1$, the asymptotic error is:

$$\varepsilon_g(t \to \infty) = D(\gamma\chi + 1) = \frac{D}{1-\nu}\,. \tag{6.18}$$

Note the pole as criticality is approached. The quantity $\chi = \int_0^\infty R(\tau)\, d\tau$ is the **susceptibility**; it is well-defined only when the response function decays, i.e. only in

equilibrium. Its appearance here signals that in this phase the response to perturbations of the initial conditions really does decay: asymptotically, the system carries no memory of its initialization.

The *non-ergodic phase* with persistent memory is captured by the asymptotic generalization error depending explicitly on the retention of the initial condition $C(0,0)$ and still showing sensitivity to $\nu \to 1$ analogous to the $\nu < 1$ regime near criticality:

$$\varepsilon_g(t \to \infty) = \frac{(\nu - 1)\, C(0,0)}{\nu} + \frac{D\nu}{\nu - 1}\,, \tag{6.19}$$

where fully, $C(0,0) = \langle \hat{w}^2(0) \rangle + r$. The term $\langle \hat{w}^2(0) \rangle$ is the variance of the initialization and r is the mean-squared norm of the ground-truth parameters per dimension. Note that here, the response function does not decay, because we carry memory of the initial conditions. It saturates at $R(\tau \to \infty) = \frac{\nu - 1}{\nu}$, which they call the *ergodicity-breaking degree*, quantifying the persistent memory of a perturbation applied to the initialization.

They also show similar trends with the training error, namely for $\nu < 1$ the error is asymptotically finite, as the model is still attempting to approximate the solution, while for $\nu > 1$ the training error vanishes as initialization-dependent memorization ensures the model always performs accurately on the training data. They write training error as

$$\lim_{t \to \infty} \varepsilon_t(t) = (\gamma\chi + 1)^{-2}\, \varepsilon_g(t \to \infty)\,. \tag{6.20}$$

For $\nu < 1$ it simplifies to $D(1 - \nu)$, while again, for $\nu > 1$, the system is no longer at equilibrium and the susceptibility $\chi = \int_0^\infty R(\tau) d\tau$ diverges, which drives the training error to zero.

To characterize the critical behavior of the model near $\nu \approx 1$, they calculate the critical exponents (σ, ϕ) and propose a scaling function. The first part is relatively straightforward, where from Eqs. (6.18) and (6.19), one finds $\varepsilon_g(t \to \infty)$ diverges from both sides $\nu \to 1^{\pm}$. This suggests the scaling $\varepsilon_g(t \to \infty) \sim |1 - \nu|^{-\sigma}$ with an exponent $\sigma = 1$ in DMFT. We remind the reader that these are asymptotic long-time steady-state results and both training and generalization error are time-dependent. Since the saturation time is ν-dependent, a general feature of phase transitions is a dynamic scaling data collapse should be present near the transition [33]. They make this argument and propose a scaling ansatz of form:

$$\varepsilon_g \approx \frac{D}{|1 - \nu|^{\sigma}}\, F\!\left(\frac{|1 - \nu|^{\phi} (\gamma t)}{4} \right)\,, \tag{6.21}$$

where they introduce a new critical exponent ϕ and write the universal scaling function $F(z)$, and γ is still the probability of samples being included. They make two observations about the form of F near and at the critical point. First, near the point, $\nu \to 1^{\pm}$ corresponds to $z \to 0$, they argue the singularity in front must be absorbed into F and it takes the asymptotic form $F(z) \sim z^{\sigma/\phi}$. Second, at $\nu = 1$, since the generalization error grows in time instead of saturating, $\epsilon_g(t) \sim (\gamma t)^{\sigma/\phi}$. The exponent ϕ therefore characterizes this growth in training time. Notably, the form of Eq. (6.21) implies a characteristic time scale, which they denote $\tau(\nu) = (\gamma |1 - \nu|^{\phi})^{-1}$ to describe

convergence in the scaling of the error. Since the error grows, we would expect this to diverge as $\nu \to 1$, which it does, and is an iconic signature of a phase transition, called **critical slowing down**. In MC calculations for lattice systems for example, this shows up in the autocorrelation time increasing, and requiring more time to get uncorrelated samples as we approach the critical point. A great deal of energy goes into designing methods to circumvent this slowing down when attempting to measure precise values of critical exponents using MC techniques, and motivates the choice of entirely different paradigms like RG flow or conformal bootstrap where applicable [33].

Exercise 6.2 (Double descent in the lab). Simulate the model of this subsection: teacher $w^* \sim \mathcal{N}(0, r\, I_P)$, inputs $x^\mu \sim \mathcal{N}(0, I_P)$, labels $y^\mu = w^* \cdot x^\mu/\sqrt{P} + \epsilon^\mu$ with noise variance D; student $\hat{w}$ trained by the update (6.11) with ridge $\lambda = 0$ and full batches ($\gamma = 1$), from $\hat{w}(0) \sim \mathcal{N}(0, \sigma_0^2 I_P)$. Take $P = 200$ and $\nu = P/n \in \{0.5, 0.9, 1.1, 2\}$.

(a) Plot training and generalization error against time, and verify the asymptotics (6.18) and (6.19) — including the divergence of *both* branches as $\nu \to 1$: double descent, resolved in time.

(b) Probe ergodicity breaking: run two clones from initializations differing by a small random $\delta\hat{w}(0)$, and track the normalized separation $|\Delta\hat{w}(t)|^2/|\Delta\hat{w}(0)|^2$. Verify that it decays to zero for $\nu < 1$ but saturates for $\nu > 1$ — at the predicted plateau $(\nu-1)/\nu$, the paper's ergodicity-breaking degree, measured in twenty lines of code.

(c) Near $\nu = 1$, extract the relaxation time of $\varepsilon_g(t)$ and check for the critical slowing down $\tau \sim |1-\nu|^{-\phi}$ of Eq. (6.21).

6.3 Dynamical mean-field theory (DMFT)

Dynamical mean-field theory, of which Section 6.2 was one instance, is a general strategy for reducing the number of effective degrees of freedom in an interacting dynamical system: one follows a *single* representative degree of freedom, moving in a fluctuating background that stands in for all the others — and one demands that the statistics of the background be determined, self-consistently, by the statistics of the representative trajectory itself. This is the same trichotomy as every other mean field in these notes (couple through averages; argue fluctuations are controlled; close self-consistently), upgraded from scalars and measures to two-time functions. An excellent intuition-building tutorial in a biophysics context is [8], and [23] is a broad modern review. In quantum many-body theory the same idea has been enormously effective for otherwise intractable systems [31, 46]. Below we first clarify what *self-consistency* means in a dynamical setting, then survey the recent deep-learning literature; a natural technical entry point for the latter is the out-of-equilibrium DMFT of the perceptron [1].

6.3.1 Self-consistency

The term *self-consistency* describes the demand that the predictions of a theory agree with the approximations used to obtain them. A simple example, taken from [85], makes the issue vivid (and is worked in detail in Exercise 6.3). Consider the stochastic process $dx(t) = f(x(t))\,dt + dW(t)$, with W a Wiener process (so that $\dot{W}$ is the white noise of Section 6.1), and suppose $f(x_0) = 0$ with $f'(x_0) < 0$: a stable fixed point of the noiseless dynamics. One can expand around it, writing $x(t) = x_0 + \delta x(t)$, to obtain the approximation

$$\frac{d}{dt}\delta x(t) = f'(x_0)\delta x(t) + \xi(t) + \mathcal{O}(\delta^2 x(t))\,, \tag{6.22}$$

where ξ is also centered gaussian white noise. The mean and variance of these fluctuations can be computed straightforwardly via Fourier transform, and in the temporal basis the variance is $\langle \delta x^2 \rangle = \frac{D}{-2f'(x_0)}$, however using these results with the Taylor expansion of $f(x)$ to second order gives a correction to the mean

$$\langle x \rangle \approx x_0 + \frac{1}{2}\frac{f''(x_0)}{f'(x_0)}\frac{D}{2f'(x_0)} \tag{6.23}$$

which is *inconsistent* with the value x_0 assumed for the mean when constructing the approximation: the noise, filtered through the curvature of f, shifts the very point we expanded around.

Exercise 6.3 (Self-consistency in the smallest possible example).

(a) Derive the stationary variance $\langle \delta x^2 \rangle = D/(-2f'(x_0))$ of the linearized dynamics (compare Exercise 6.1(b)).

(b) Carrying the Taylor expansion of f to second order, derive the quoted correction to the mean.

(c) Propose the self-consistent repair: expand instead around a point x_*, chosen so that the *effective* drift vanishes, $f(x_*) + \frac{1}{2}f''(x_*)\langle \delta x^2 \rangle_* = 0$, with $\langle \delta x^2 \rangle_* = D/(-2f'(x_*))$ determined simultaneously. Note the structure: two coupled equations, each an input to the other — a miniature of the Curie–Weiss condition of Section 5.1, and of the DMFT equations to come.

A framework for resolving this, and also attempting to enforce that the small parameter we expand about is actually the expansion parameter involves using *n-particle irreducible effective actions* (nPI). A full discussion of this is well beyond the scope of this document, but is covered very well in a pedagogical introduction [7], where the main idea is enforcing self-consistency via increasingly multi-local sources that generate n-point correlation functions. The first section emphasizes that performing standard perturbation theory with an anharmonic oscillator yields so-called *secular* terms which are powers of time t that appear next to the actual expansion parameter ϵ, and using

the nPI framework effectively resums these terms properly at each order in ϵ. The 1PI approach is directly related to a local approximation with mean field theory, while in the 2PI approach, the object determined self consistently is the 2-point function, which in the local approximation is related to the local propagator and is the basis of dynamical mean field theory.

6.3.2 Recent applications in deep learning

Fortunately, one rarely needs the general nPI machinery in practice. The working route — structurally the same trick by which large-N models in physics (*e.g.* the G–Σ formulation of the SYK model) are solved — is to insert bilocal *collective fields* for the correlation and response functions via delta functions, exactly as the Q's of Section 6.2, and then to evaluate their path integral by saddle point in a suitable $N \to \infty$ limit. This is how the deep-learning DMFT of [12, 9] is constructed.

A brief, opinionated tour of the literature, oldest first.

Spin-glass origins. The mean-field dynamics of disordered systems is where all of this machinery was forged. Amit, Gutfreund, and Sompolinsky [3] brought spin-glass methods into neural networks (in the guise of Hopfield-style associative memories), and Sompolinsky, Crisanti, and Sommers [83] derived the DMFT of random recurrent networks, discovering a transition to chaos. The structural point to retain: an ordered system like the Ising magnet has one global coupling $J \sum_{\langle ij \rangle} \sigma_i \sigma_j$ and is already hard; a *disordered* system has random couplings $\sum_{ij} J_{ij} \sigma_i \sigma_j$, and it is precisely the all-to-all randomness that makes mean-field methods exact — the same structural feature that data-dependent couplings give learning problems, starting with the perceptron of Section 2.

The Bordelon–Pehlevan program. For deep networks trained by gradient descent at fixed dataset, the DMFT of [12] takes the infinite-width limit and finds that the order parameters are precisely the *feature kernels* $\Phi(t, s)$ and *gradient kernels* $G(t, s)$ — two-time generalizations of the NNGP kernel and NTK of Sections 3 to 4. The lazy regime of Section 4 is recovered when these kernels freeze; rich, feature-learning dynamics is their genuine evolution, computed self-consistently. (Fair warning: the notation is heavy — response fields and conjugate order parameters abound — but the structure is exactly MSRJD plus a saddle point.) The program now includes finite-width kernel fluctuations [13], hyperparameter transfer across *depth* [11], and the dynamical theory of neural scaling laws [9] that we meet again in Section 7.9.

The Mignacco–Urbani line. A complementary series of works treats the *data* as quenched disorder at proportional scalings of dimension and sample size, in the spirit of Section 2: DMFT for Gaussian-mixture classification [58], and a DMFT characterization of the effective noise of SGD at late times [59] — the dynamical refinement of the batch-noise discussion of Section 2.1.

Rigor. What began as a physics heuristic is increasingly a theorem: rigorous derivations of DMFT for first-order methods now exist [17, 32], with a growing line of extensions — e.g. to adaptive Langevin diffusions, with propagation-of-chaos guarantees and convergence of the linear response [27]. A recent tour de force in this direction [60]

solves the dynamics of a two-layer network at scale — a long calculation, but a well-organized paper — finding: the emergence of a slow timescale tied to the growth of the network's complexity (in the Gaussian/Rademacher sense); an inductive bias toward low complexity when the initialization has low complexity; a dynamical *decoupling* of the feature-learning and overfitting regimes; and a non-monotone test error, with a "feature unlearning" regime at very late times.

The connective tissue of this whole section, worth stating once more: the MSRJD path integral is to training dynamics what the Gibbs measure was to Bayesian learning — the object on which every physics technique (saddle points, disorder averages, collective fields, perturbation theory) can be brought to bear.

7 Scaling laws

In this final section we return to the theme of Section 1 and discuss the *neural scaling laws*. Over many orders of magnitude, the test loss of large models falls as a clean power law in each resource spent on it: parameters, data, and compute. These laws were found by fitting curves to measurements, and they have proved to be a remarkably simple, robust, and consequential empirical observation, one that any successful model of deep learning should be able to capture. Many models do capture their form, and several go further and make quantitative predictions for the exponents, based on features of the data distribution.

The organizing question of the section is: *where do the exponents come from?* We first assemble the empirical picture (Sections 7.1 to 7.2): the three laws, what each one holds fixed, and the joint laws that govern compute-optimal training, where the Kaplan–Chinchilla discrepancy lived and was resolved. We then turn to theory. Section 7.3 states the mechanism common to every proposal — power laws in the data produce power laws in the loss — and Section 7.4 surveys the main proposals in one table. The rest of the section works out four rows of that table, in order of increasing machinery: the frequency statistics of skills (Section 7.5, run as a derive-it-yourself exercise, with its payoff for "emergent capabilities" in Section 7.6); the geometry of data and the spectra of kernels (Section 7.7); and solvable large-N field theories, static and then dynamical (Sections 7.8 to 7.9). We close with the newest measurements, for which there is no theory yet: post-training and test-time scaling (Section 7.10).

Notation. We keep the conventions of these notes: P is the parameter count and n the dataset size. Be warned that the scaling-law literature writes N for our P and D for our n (their N is *not* our width!), so the famous exponents α_N, α_D are our α_P, α_n. For reading the sources:

these notes	scaling-law literature [44, 41, 57, 55]
P (parameters)	N
n (dataset size)	D
α_P, α_n (exponents)	α_N, α_D
m (quanta learned, Section 7.5)	n

Note also that in this section α (no subscript) denotes the Zipf exponent of the quantization model, not the load of Section 2.

7.1 The empirical laws

Kaplan et al. [44] established, for autoregressive language models, that when a model is not bottlenecked by the other resources its test loss obeys

$$L(P) \propto P^{-\alpha_P}\,, \qquad L(n) \propto n^{-\alpha_n}\,, \qquad L(C) \propto C^{-\alpha_C}\,, \tag{7.1}$$

where P is the number of (non-embedding) parameters, n the dataset size in tokens, and C the training compute in FLOPs. The measured exponents were $\alpha_P \approx 0.076$, $\alpha_n \approx 0.095$, and $\alpha_C \approx 0.050$, with the trends holding over up to seven orders of magnitude. Three points of bookkeeping are usually left implicit. They are worth making explicit before asking where the laws come from.

Why FLOPs, and why $C \approx 6Pn$. Compute is measured in floating-point operations because FLOPs are the only measure of training effort that does not depend on how the job is run. Wall-clock time depends on the hardware and the parallelism. The number of training *steps* depends on the batch size: the same n tokens can be one giant batch or a million small ones, and below the "critical batch size" (which [44] measure separately) the loss depends only on the number of tokens processed. FLOPs count the arithmetic itself, which is also what energy and money are proportional to. The standard estimate is

$$C \;\approx\; 6\,P\,n\,. \tag{7.2}$$

Per token, the forward pass costs about $2P$ FLOPs: essentially every parameter sits in some matrix multiplication and is used once, for one multiply and one add. The backward pass costs about twice the forward pass, since each layer needs the gradient with respect to its activations and the gradient with respect to its weights, two matrix multiplications of the same size as the forward one. That gives $6P$ FLOPs per token, times n tokens. The $\approx$ absorbs attention (whose per-token cost grows with context length rather than parameter count), embeddings, and optimizer overhead.

The single-epoch assumption. The estimate (7.2) assumes that every token processed is a fresh token, so that "tokens processed" and "dataset size" are the same number. This was true of the runs on which the laws were measured, which used well under one epoch of the corpus. Conceptually the two are different resources. The *statistical* resource is the number of distinct tokens n; the *computational* resource is the number of tokens processed, $T = E\,n$ over E epochs, so that in general $C \approx 6PT$. Fixing n and growing C by repetition buys optimization, not statistics: the loss converges to the n-limited floor and then overfits. In the language of Section 2.1, more epochs means larger β at a fixed quenched sample, while fresh data changes the quenched free energy itself. Empirically the one-epoch idealization is forgiving: for up to about four epochs, repeated tokens are nearly as valuable as fresh ones, after which their value decays quickly [61]. This matters increasingly, now that frontier training is data-constrained.

The compute law is not independent. Since $C \approx 6Pn$ depends only on the other two resources, how can compute ever be the unbottlenecked one? The answer is that $L(C)$ is measured by a different protocol. It is the lower envelope

$$L(C) \;=\; \min_{P,\,n\,:\,6Pn\,=\,C} L(P,n)\,, \tag{7.3}$$

traced out by training many model sizes and keeping the best loss at each value of compute. On this curve compute is the binding constraint by construction, and "not bottlenecked" means "optimally allocated." This makes α_C a derived exponent. Minimizing an additive joint law $L = A\,P^{-\alpha_P} + B\,n^{-\alpha_n}$ at fixed Pn balances the two terms, and gives

$$\frac{1}{\alpha_C} \;=\; \frac{1}{\alpha_P} + \frac{1}{\alpha_n}\,, \qquad P \propto C^{\,\alpha_n/(\alpha_P+\alpha_n)}\,, \quad n \propto C^{\,\alpha_P/(\alpha_P+\alpha_n)}\,. \tag{7.4}$$

The harmonic sum makes α_C smaller than either single-resource exponent, since a factor of C must be split between growing P and growing n. Kaplan's values give $\alpha_C \approx 0.042$, against the measured 0.050: roughly consistent, with the gap traceable to the form of the fit and to early-stopping details.

Universal form; data-dependent exponents. The same power-law form holds far beyond language. Henighan et al. [39] trained autoregressive transformers on many kinds of data and found power laws everywhere, but with exponents that depend on the data:

modality	α_P	α_C
language	0.070	0.048
images 32×32	0.13	0.10
images 8×8	0.24	0.19
video	0.24	0.14
math (procedural)	0.16	0.17

("Math" here means procedurally generated problems, in the style of the DeepMind mathematics dataset. They are emitted by a known program, so the supply is effectively unlimited. Remember this case: it is the one modality whose data distribution is known in closed form.) The exponents vary by a factor of 3–5 across modalities. Architecture, by contrast, barely matters: within reasonable architectures, changing the shape (depth vs. width at fixed size) or even the family moves the constants and the plateaus, not the slopes [44]. This asymmetry is the central clue of the section: *whatever sets the exponent lives in the data.* Section 7.7 makes it quantitative.

7.2 Joint laws, bottlenecks, and the Chinchilla revision

Each single-resource law assumes the other resources are unlimited. Joint fits describe the crossover to being bottlenecked, with the loss governed by whichever resource is scarcest. Two parametric forms dominate the literature. Kaplan et al. used

$$L(P,n) = \Big[\Big(\frac{P_c}{P}\Big)^{\alpha_P/\alpha_n} + \frac{n_c}{n}\Big]^{\alpha_n}\,, \tag{7.5}$$

while the Chinchilla analysis of Hoffmann et al. [41] refit with the additive ansatz

$$L(P, n) \;\approx\; E + A\,P^{-0.34} + B\,n^{-0.28}\,. \tag{7.6}$$

The additive form is easy to read. $E \approx 1.69$ is the irreducible entropy of text, the noise floor that no amount of scale removes; the two deficits decay independently; and the bottleneck is simply the larger deficit. Both forms reduce to the single-resource laws (7.1) in the appropriate corners.

The difference between the fits mattered a great deal in practice, because of what they say about the compute-optimal allocation in (7.3). Kaplan et al.'s analysis gave $P \propto C^{0.73}$ and $n \propto C^{0.27}$: with more compute, grow the model. For two years the field trained ever-larger models on comparatively little data. Chinchilla's fit gives $P \propto C^{0.5}$ and $n \propto C^{0.5}$: grow both equally, or as a rule of thumb about 20 tokens per parameter. The prediction was tested directly. At the compute budget of DeepMind's 280B-parameter Gopher, the new rule said to train a model four times smaller on four times more data, and the resulting 70B-parameter Chinchilla beat Gopher across the board. (Both models, and the law, are named in DeepMind's animal series.) Chinchilla's allocation is now the accepted baseline. The discrepancy was later dissected by Porian et al. [66]: Kaplan's fit was skewed by three small-scale artifacts — how last-layer and embedding compute were counted, over-long warmup, and optimizer settings that were not retuned with scale — and with these corrected, the two experiments agree. One coda: frontier models now deliberately *over*-train relative to Chinchilla, on many more than 20 tokens per parameter, because a smaller model at equal loss is cheaper at inference time, a cost the training-compute-optimal analysis does not see.

With the empirical picture assembled, here is what a theory of (7.1) owes us: (i) why power laws at all, over so many decades; (ii) the values of the exponents (why so small? why do they depend on the modality but not on the architecture?); (iii) relations between exponents (is α_n determined by α_P?); and (iv) when the laws break. The models surveyed below each answer a subset.

7.3 Where do the laws come from?

The basic answer, common to every proposal surveyed below, is that the loss curve reads off the coarse distribution of structure in the data. Power laws go in with the data and come out in the loss.

The prototype of "power laws in" is *Zipf's law*: rank the words of a large corpus by frequency, and the frequency of the word of rank k falls off as $f_k \propto 1/k$, over about four decades of rank (Figure 7). It is a scale-free tail of ever rarer structure, and it is not special to words. Sequences of n consecutive words (n-grams, the unit of pre-neural language modeling) are Zipf-like at every n, with slopes that flatten as n grows. Facts, meaning discrete pieces of world knowledge attested in text, inherit the heavy-tailed popularity of the entities they mention, and a model's ability to answer a factual question tracks the number of training documents that support it [43]. The covariance and kernel spectra of text and image data are measured power laws (Sections 7.7 to 7.8). A learner working through such data never runs out of structure to absorb. It moves

ever further down the tail, at ever slower returns. That is, qualitatively, why the loss is a power law rather than an exponential (there is structure at all scales) and why it does not simply plateau (the tail does not end).

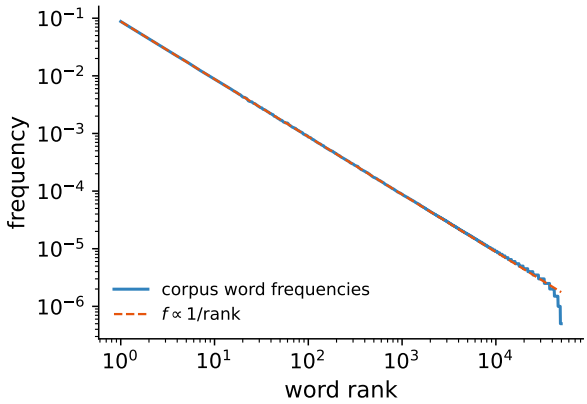


Figure 7: Zipf's law, illustrated on a synthetic corpus: 2×10^6 tokens drawn from a $1/k$ distribution over a vocabulary of 5×10^4 words, with the empirical rank–frequency curve against the $f \propto 1/\text{rank}$ guide. The curve is a consistency check rather than data, since the words were sampled from a Zipf distribution to begin with. Real corpora look the same over roughly four decades, with deviations at both ends: the Zipf–Mandelbrot correction at low rank, and a steeper tail beyond about 10^4 words.

The exponents of these tails differ, and the differences matter, because in every model below the loss exponent is inherited from the data exponent. Pure Zipf is in fact a marginal case. Write the tail of skill frequencies as $p_k \propto k^{-(\alpha+1)}$. The quantization model of Section 7.5 gives a loss exponent equal to α, and exact Zipf ($f_k \propto 1/k$, that is, $\alpha = 0$) is the boundary at which the tail sum decays only logarithmically and power-law scaling degenerates. Read this way, the measured $\alpha_P \approx 0.076$ for language says that its effective skill distribution is $p_k \sim k^{-1.08}$, barely steeper than pure Zipf. Language scaling is slow because its structure sits just above the marginal $1/k$ tail. Image data has steeper tails and scales faster; compare the modality table of Section 7.1.

Where does the theory stand? The single-resource laws have been derived from this mechanism by many models of quite different flavor, and a few models derive multi-resource laws; the survey below keeps score. Three things are missing. First, no theory computes the data's exponents from first principles. Every model converts measured data statistics into loss curves, and the statistics themselves are an empirical input. This is true even for procedurally generated math, where the generating program is known exactly but the computation has not been carried out. Second, there are no joint laws with genuine feature learning. The best dynamical derivations live in lazy, random-feature settings (Section 7.9), while in solvable cases feature learning is known to improve exponents for structured targets, so the corrections need not be small. Third, the bridge from the smooth loss to capabilities is missing. Benchmark performance can jump within a decade of scale while the loss glides smoothly [92]; this has been contested

as an artifact of the metric [74], and the quantization model of Section 7.6 gives it a sharper reading. A fourth gap is newer: the post-training and test-time scaling laws of Section 7.10 have no theory at all.

7.4 A survey of theoretical proposals

The table below organizes the most prominent derivations of neural scaling laws by four questions: what models the *data* (equivalently, what a "feature" is and how features are distributed); what models the *network*, if anything; whether actual *exponents* are predicted, and in terms of what; and whether *multi-resource* laws are obtained.

proposal	**data model / "features"**	**NN framework**	**exponents?**	**multi-resource?**
quantization; Michaud et al. [57]	discrete skill "quanta," Zipf-used, learned in order	none (a counting argument)	in terms of the Zipf exponent α; plus the relation $\frac{1}{\alpha_n} = \frac{1}{\alpha_P} + 1$	separate P, n, S laws
data manifold; Sharma–Kaplan [78]	smooth target on a d-dimensional manifold	piecewise-linear interpolation (function class only; no training)	yes: $\alpha_P \approx 4/d$	no (P only)
percolation; Brill [14]	latent subtask clusters at criticality, with power-law cluster sizes	none: a generic nonparametric learner + regression estimates	from percolation critical exponents	two regimes; not joint
kernel eigenlearning; Bahri et al. [5], Bordelon–Canatar–Pehlevan [10, 16, 79]	power-law kernel spectra + target coefficients	**frozen kernel**: infinite-width NTK, or GP/Bayesian	in terms of the spectral exponents	partially: resolution- vs. variance-limited regimes
solvable field theory; Maloney–Roberts–Sully [55]	latent power-law covariance, extended by random features	linear model on **fixed random features** (frozen kernel); large-N RMT, statics	in terms of the data's spectral exponent	yes: joint $L(P, n)$, with duality $P \leftrightarrow n$
DMFT; Bordelon–Atanasov–Pehlevan [9]	power-law (source/capacity) spectra	DMFT of gradient-flow training on **fixed random features** (frozen kernel; dynamics)	yes, including the time exponent	fully joint $L(t, P, n)$ + compute frontier

Five remarks for reading the table. First, everyone agrees on the origin. In every row the loss exponent is inherited from a power-law tail in the data, whether of skill frequencies, cluster sizes, manifold volume, or covariance spectra. The rows differ in what the tail is a tail *of*, and in how much network machinery is retained. Second,

and this deserves emphasis: *every row that models the network at all does so with a frozen kernel.* The eigenlearning row works with the infinite-width NTK or GP kernel of Sections 4.3 and 4.6; the two field-theory rows train only the readout of a fixed set of random features, which is a frozen kernel by construction (Exercise B.6(b)). The manifold row models only the network's function class, as a count of linear pieces, and never trains it; the quantization and percolation rows contain no network. So none of the current derivations involves feature learning, the very mechanism that Sections 5 to 6 argued is what makes finite networks differ from kernels. Third, equilibrium routes (Bayesian GP learning curves and their finite-width refinements) and dynamical routes (gradient flow, DMFT) give matching exponents wherever they overlap. This is the "two limits of one system" correspondence of Section 2.1 and Section 6 at work again. Fourth, only the two field-theory rows currently predict joint laws, and only the dynamical one prices training time, deriving a Chinchilla-like compute-optimal frontier rather than assuming it. Fifth, no row computes the data's own exponent. The closest attempt is the percolation proposal, which derives power-law cluster-size statistics from a critical model of the data distribution; its two criticality regimes (power-law-distributed discrete subtasks, and a dominant data manifold) map onto the quantization and manifold rows respectively. We will not develop the percolation model further. The remaining subsections work through the other rows, in order of machinery.

Do the predicted exponents agree with experiment? Where the comparison has been made, the answer is encouraging but not yet sharp. The quantization model's authors extracted "quanta" from a small language model by clustering gradients, measured how often each is used in the training distribution, and found a power law whose exponent roughly matches the model's scaling exponent, as the model requires [57]; on the other hand, the model's parameter-free relation between α_P and α_n is violated by the measured ordering of Kaplan's exponents (Exercise 7.1(e)). The manifold prediction $\alpha_P \approx 4/d$ was confirmed in teacher–student experiments, where d and α_P can be measured independently, and tested more loosely on image classifiers and GPT-type language models [78]; for language, taking $\alpha_P \approx 0.076$ at face value gives $d \approx 50$ (Exercise 7.2). The kernel predictions have been tested on standard architectures and datasets, with exponents computed from measured kernel spectra, and the predicted duality between the P- and n-exponents holds up [5]. The two field-theory rows take the data's spectral exponent as input. Their qualitative predictions — exponents independent of architecture, $\alpha_P \approx \alpha_n$, and equiparameterization as the compute-optimal rule — match the Chinchilla-era measurements, but a quantitative test needs the covariance spectrum of the training data, which is rarely reported for the large runs. The honest summary is that these theories convert an exponent measured in the *data* into an exponent measured in the *loss*, and the conversion has held up wherever both have been measured.

7.5 The quantization model: exponents from the statistics of skills

The quantization model of Michaud, Liu, Girit, and Tegmark [57] is a minimal model of scaling. It has no architecture, no optimizer, and essentially no mathematics beyond a

tail sum, and it produces all three laws of (7.1), plus a sharp point of view on "emergent capabilities." The name is deliberately provocative: the proposal is that what a network learns comes in *discrete units*, dubbed *quanta*.[4] The assumptions, in the form we'll use:

- **QH1 (discreteness).** The computations and knowledge relevant to the prediction task decompose into enumerable *quanta*: basic skills or facts (*e.g.* "carry the one," "Paris follows 'the capital of France is'," ...). Each quantum is either fully learned or not learned at all, with no partial credit.
- **QH2 (ordered acquisition).** Networks learn quanta roughly in order of decreasing usefulness, so a given model has learned the first m quanta of a fixed sequence.
- **QH3 (Zipfian use).** Quantum k is *used* on a fraction

$$p_k = \frac{k^{-(\alpha+1)}}{\zeta(\alpha+1)} \tag{7.7}$$

of samples ($\alpha > 0$ a constant), and on those samples, knowing it reduces the loss by a constant Δ (independent of k).

Consequently, a model that has learned the first m quanta has expected loss

$$L_m = L_\infty + \Delta \sum_{k>m} p_k \,. \tag{7.8}$$

That is the entire model. Everything else is asking how m is limited by each resource.

Exercise 7.1 (Three scaling laws from three assumptions). (★)

(a) Estimate the tail sum in (7.8) to show $L_m - L_\infty \propto m^{-\alpha}$.

(b) *Parameter-limited (data and training unlimited).* Suppose each quantum costs a constant capacity of c parameters to learn. Derive α_P in terms of α.

(c) *Data-limited (capacity and training unlimited).* Suppose a quantum is learnable only if it is used by at least τ training samples, for constant τ. Given n i.i.d. samples, which quanta are learnable? Derive α_n.

(d) *Training-limited, single epoch.* Suppose instead that learning quantum k requires τ gradient updates on batches containing it. After S steps of online training (fresh batches), derive α_S.

(e) *Falsifiability.* Parts (b)–(c) predict a parameter-free relation between α_P and α_n: write it down, and note which of the two exponents must be

[4] Michaud's blog post ericjmichaud.com/quanta is an excellent companion to the paper and can substitute for it in this exercise. But attempt Exercise 7.1 *before* reading either; the derivations are short, and they are more instructive to find than to follow.

larger. Compare with the measured values quoted after (7.1). Something is wrong — what? List the assumptions you used and discuss which are most suspect. (This is a discussion question with no settled answer; see the solution for candidates.)

7.6 Emergence: smooth laws from staggered cliffs

The quantization model's deepest payoff is conceptual. Under QH1, the learning of any *single* quantum is a step function: a capability is absent, then abruptly present. Yet the aggregate loss (7.8) is a smooth power law. There is no contradiction. *A smooth law is a sum of many staggered cliffs*, each weighted by p_k and individually invisible in the aggregate, since quantum k contributes only the small fraction p_k of the loss. Michaud et al. sharpen this with a genetics metaphor. A test sample is *monogenic* if its loss is governed by a single quantum, so that its loss curve during training is a plateau followed by a cliff; it is *polygenic* if many quanta contribute, in which case its loss falls smoothly. Both kinds of per-token curves are observed in real LLMs. From this perspective, "emergent capabilities" [92] are neither magic nor mirage [74]: any benchmark dominated by a few quanta *must* show abrupt arrival, while any sufficiently aggregated metric *must* look smooth. Emergence is a property of what you measure, not (necessarily) of a phase transition inside the model.

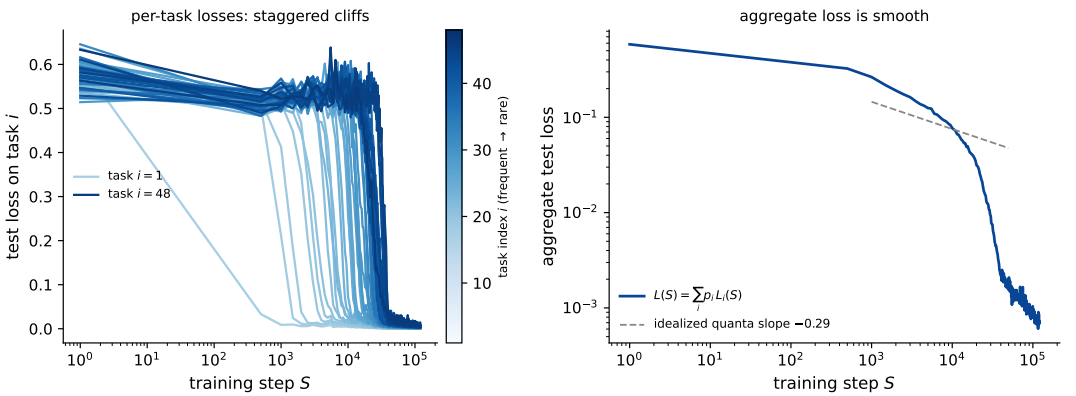


Figure 8: Multitask sparse parity [57]: smooth scaling from staggered cliffs. Setup: $T = 48$ subtasks with Zipfian frequencies $p_i \propto i^{-(\alpha+1)}$, $\alpha = 0.4$; inputs are a one-hot task indicator (T bits) concatenated with 24 random ± 1 bits; the label is the parity of a fixed random 3-subset of the bits, the subset depending on the task. Model: one-hidden-layer ReLU MLP, width 512, MSE loss on ± 1 labels, trained online (fresh batches of 256; Adam, learning rate 2×10^{-3}, 1.2×10^5 steps; fixed seed). *Left:* test loss per subtask vs. training step. Each subtask is learned in an abrupt cliff, frequent tasks first. *Right:* the frequency-weighted aggregate $L(S) = \sum_i p_i L_i(S)$ is smooth; the dashed guide is the *idealized* quanta-model slope $-\alpha/(\alpha+1)$ from Exercise 7.1(d). The measured decay is visibly steeper, as discussed in the text.

Lab: multitask sparse parity. The entire mechanism fits in a laptop-scale experiment, shown in Figure 8; students are encouraged to reproduce and modify it (all details are in the caption). A network is trained online on T separate parity subtasks with Zipfian frequencies. The input is a one-hot task indicator concatenated with random bits, and the label is the parity of a small task-specific subset of the bits. Parity is a needle-in-a-haystack skill with no partial credit, *i.e.* an honest hand-built quantum. The result: each subtask's test loss is a staggered cliff (frequent tasks fall first), while the frequency-weighted aggregate loss decays smoothly. Note also the honest failure visible in the right panel. The measured aggregate decays *more steeply* than the idealized quanta slope $-\alpha/(\alpha+1)$ of Exercise 7.1(d), because the cliffs bunch up rather than spreading Zipf-like; the rare tasks are learned sooner than the threshold rule $S\,p_k \gtrsim \tau$ predicts. The subtasks share a hidden layer and an adaptive optimizer, so they are not learned independently, and assumption (iv) in the solution to Exercise 7.1(e) fails.

7.7 Exponents from geometry and from spectra

The quantization model takes the Zipf exponent α as *input*. The next two arguments produce exponents as *output*, from properties of the data.

7.7.1 Geometry: the data-manifold argument

Sharma and Kaplan [78] propose the simplest continuous mechanism. A network with ReLU activations computes a piecewise-linear function, so regression on a data manifold of intrinsic dimension d is a problem of *piecewise-linear interpolation*. A network with P parameters can afford $\sim P$ linear pieces; tiling a d-dimensional manifold with P pieces gives spacing $s \sim P^{-1/d}$; a linear patch approximates a smooth target to accuracy $\sim s^2$ (Taylor); and MSE is the *square* of the error. Hence

$$L(P) \sim (s^2)^2 \sim P^{-4/d}\,, \qquad \alpha_P \approx \frac{4}{d}\,. \tag{7.9}$$

Exponents are small because data manifolds are high-dimensional. The formula is testable, since d can be measured with intrinsic-dimension estimators, which is exactly what [78] do.

Exercise 7.2 (Exponents from the data manifold).

(a) Fill in the derivation of (7.9), stating precisely where smoothness of the target is used.

(b) How does the answer change for mean-absolute-error loss?

(c) Taking the measured $\alpha_P \approx 0.076$ for language models at face value, estimate the effective dimension of the "manifold of text." Discuss whether the input dimension, the embedding dimension, or something else is being measured.

(d) What does this mechanism predict for the *data* exponent α_n, if n samples are what limits the resolution of the tiling?

7.7.2 Spectra: scaling laws from kernel eigenvalues

The second mechanism uses machinery we already own: the Bayesian kernel regression of Section 4.6, whose mode-filtering formula said that with n samples (and inverse temperature β), the modes of the GP kernel that get learned are those with $\lambda_k \gtrsim (n\beta)^{-1}$. Suppose the kernel spectrum and the target's decomposition are power laws,

$$\lambda_k \sim k^{-a} \quad (a > 1)\,, \qquad y_k^2 \sim k^{-b} \quad (b > 1)\,, \tag{7.10}$$

in a shared eigenbasis $\{\varphi_k\}$. The unlearned modes dominate the test error:

$$L(n) \;\sim \sum_{k\,:\,\lambda_k \lesssim (n\beta)^{-1}} y_k^2 \;\sim \sum_{k > k_*} k^{-b} \;\sim\; k_*^{1-b}\,, \qquad k_* \sim (n\beta)^{1/a}\,, \tag{7.11}$$

giving $L(n) \sim n^{-(b-1)/a}$: *the scaling exponent is a ratio of spectral decay rates.* Everything about the data and the architecture enters through two measurable power laws. This is the starting point of the "eigenlearning" analyses [10, 16, 79], which sharpen (7.11) into precise learning curves (including the overfitting factor whose divergence produces the double-descent peak of Section 6). Bahri et al. [5] organize the possibilities into a clean 2×2 taxonomy: *resolution-limited* regimes (exponents from spectra or geometry, as here and in (7.9)) versus *variance-limited* regimes (universal exponent 1), in each of P and n.

Exercise 7.3 (Spectral scaling laws, and quanta as eigenmodes).

(a) Derive (7.11) from the mode-filtering formula of Section 4.6, and check that the crossover modes $k \sim k_*$ do not change the exponent.

(b) Show that the quantization model's data-scaling law is the special case $b = a$ of (7.11) under the dictionary: quanta = eigenmodes, Zipf law = spectrum, $a = \alpha + 1$. (Hint: in the quanta model, p_k plays *both* roles, setting the learning threshold *and* the contribution $\Delta\, p_k$ to the loss, so the two power laws coincide.)

(c) In the quanta model, what would it mean for $b \neq a$? Invent a modification of QH3 that realizes it.

7.8 A solvable large-N field theory

Maloney, Roberts, and Sully [55] supply what the arguments above lack: a *microscopic, exactly solvable* model in which the joint law $L(P, n)$ is computed, not modeled. The ingredients are: a generative data model in which observed inputs are (nonlinear functions of) random projections of a latent space, engineered so that the data covariance

has a power-law spectrum $\lambda_k \sim k^{-(1+\zeta)}$, as measured in real image and text data (the spectral exponent is the paper's γ; we reserve γ for the scaling dial of Section 5.4); a random-feature *linear* student, so we are in the lazy limit of Section 4, whose P random features are its trainable parameters, trained on n samples; and the joint thermodynamic limit $P, n \to \infty$ at fixed ratio. The computation proceeds at large P and n, where expectations over data and features become Gaussian integrals, the test loss becomes a product of resolvents (Green's functions), and the $1/P$ expansion organizes into planar ('t Hooft) diagrams that can be resummed exactly.[5]

Frozen, but random. A natural first reaction is that this model should be trivial. The student is a linear model on fixed random features. Its kernel never moves, so we are squarely in the lazy regime of Section 4, and in the strict infinite-width lazy limit the loss would not depend on P at all. The P-scaling law is a *finite-size effect on the lazy limit*. At finite P the feature Gram matrix is a *random* matrix, correlated with the random sample, and its rank caps how many modes of the data spectrum the student can resolve. All of the physics lives in the disorder average over this randomness, which is what the resolvents and the planar diagrams compute. Frozen does not mean deterministic.

Three notable results:

- *Where exponents come from:* the loss inherits the *data's* spectral exponent, $L \sim \min(P, n)^{-\zeta}$ up to logarithms. Remarkably, generic nonlinear feature maps *extend* the data's power-law spectrum rather than reshaping it. This explains why exponents are roughly architecture-independent and can only be improved by changing the data (or the task).

- *Duality and Chinchilla:* the loss is (to leading order) *symmetric* under $P \leftrightarrow n$. This predicts $\alpha_P \approx \alpha_n$, consistent with observation, and opposite in spirit to the quanta model's asymmetric prediction (Exercise 7.1(e)). It makes *equiparameterization* $P \propto n$ compute-optimal, a first-principles Chinchilla rule.

- *Where laws break:* scaling stops when $\min(P, n)$ exceeds the latent dimensionality of the data model, and the loss plateaus. Power laws are a statement about the resolvable structure in the data, and they end when the structure is exhausted.

7.9 All the knobs at once: a dynamical theory

Everything so far is statics, or one knob at a time. The current frontier is the *dynamical* mean-field theory of Section 6 applied to scaling. Bordelon, Atanasov, and Pehlevan [9] solve a random-feature model trained by gradient flow, with power-law data spectra, in the joint limit of large width, large data, and long time. They obtain a single self-consistent prediction for $L(t, P, n)$, with the correlation and response functions of Section 6 as the order parameters. Each resource, when it is the binding constraint,

[5] This is also a working example for Section 6's toolkit: the same model can be solved dynamically, and its double-descent peak, at the interpolation threshold $P \sim n$, is regularized by ridge or early stopping, connecting back to the divergence of the overfitting factor mentioned in Section 7.7.

contributes its own power law, recovering the resolution-limited exponents of Section 7.7 in the appropriate corners. Moreover, the theory predicts the full *compute-optimal frontier*: minimizing L at fixed compute $C \propto Pt$ yields Chinchilla-like allocation rules and the frontier exponent, rather than assuming them.

A fair question: the features here are again frozen and random, exactly the setting of Section 7.8, so is dynamical mean-field theory overkill? No, for two reasons. First, time is a genuine third resource. Predicting $L(t, P, n)$ and the compute frontier requires the whole training trajectory, and the statics only sees the $t \to \infty$ corner. Second, the naive mode-by-mode solution of the lazy regime, in which each kernel eigenmode of the residual decays as $e^{-\eta\lambda_k t}$ as in (4.34), applies to a *deterministic* kernel. Here the kernel's eigenvalues and eigenvectors are random and correlated with the sampled data, so the disorder average couples the modes. The DMFT correlation and response functions are the dynamical generalization of the static resolvents of Section 7.8, and at $t \to \infty$ the equations collapse back onto the static answer, as they must. What the theory is *not* yet doing is feature learning. The adaptive-kernel version of the formalism exists [12], but it has not been pushed through power-law data to closed-form scaling laws; these are the "joint laws with genuine feature learning" missing from Section 7.3. Since feature learning is known to improve exponents for structured targets in solvable cases, the correction need not be small.

7.10 Beyond pretraining: post-training and test-time scaling

Frontier models are no longer just pretrained. They are fine-tuned on curated demonstrations (SFT), optimized against reward models or verifiable rewards (RLHF, RLVR), and allowed to spend variable compute at inference time. Do scaling laws extend to these stages? Increasingly, yes. But the functional forms change, and the changes are instructive.

Transfer and fine-tuning still look Kaplan-like. The value of pretraining for a downstream task can be quantified as the *effective data transferred*, meaning the amount of fine-tuning data that the pretrained initialization is worth, and it scales as a power law in both the fine-tuning dataset size and the model size [40].

Reward-model optimization introduces a new scaling axis: not data or parameters, but the distance moved from the base policy, $d = \sqrt{\mathrm{KL}(\pi \,\|\, \pi_0)}$. Gao, Schulman, and Hilton [28] found that the true ("gold") reward follows simple laws in d, namely $d\,(a - b \log d)$ for best-of-n sampling and $d\,(a - b\,d)$ for RL, with coefficients that scale smoothly with the size of the reward model. Note the shape: not a power law but a rise and a *fall*. Optimizing a learned proxy eventually degrades the true objective. This is a Goodhart peak, quantified.

RL on verifiable rewards, the training recipe of the reasoning models, scales by *saturating sigmoids* rather than power laws. In the largest systematic study to date, performance as a function of RL compute is predictable, but by a fit with an asymptote, an efficiency, and a midpoint, and most recipe choices move the efficiency without changing the asymptote [45]. A physics gloss suggests why the form differs, though it is a hypothesis rather than a result. Pretraining mines an effectively unbounded Zipf

tail of new structure, hence power laws. RL post-training mostly elicits and reweights capabilities the base model already contains, on a bounded task distribution. In quanta language, pretraining acquires quanta, while RL sharpens the policy over quanta it already has, and one runs out. *Test-time compute* is the other new axis: the coverage achieved by repeated sampling grows roughly as a power law in the number of samples [15], and train-time and test-time compute can be traded against each other along a compute-optimal frontier [82].

What does not yet exist is a joint law over all four budgets (pretraining, SFT, RL, inference), and any *theory* at all, even at the level of the quantization model, that derives the sigmoid or the Goodhart peak from a model of the data and skill distribution. The empirical forms are waiting for a statistical-mechanics treatment.

That every ingredient of the pretraining synthesis — Gibbs measures, kernels, mean-field limits, MSRJD actions — appeared earlier in these notes as a physics import is, we hope, a fair summary of the state of the field. A theory of the scaling laws is not finished, but it is recognizably being built, and it is being built out of statistical physics. The post-training laws are younger still: their regularities are only now being settled, and there is no theory of them yet. For a student entering the field, that is a map of where the open territory begins.

A The replica method

Section 2.3 introduced the quenched free energy $-\mathbb{E}_{\text{data}} \log Z$ and the annealed approximation (2.18) that replaces it by $-\log \mathbb{E}_{\text{data}} Z$, and Section 2.6 used that approximation to analyze the Ising perceptron. This appendix explains how the quenched average is computed exactly, by the *replica method*, and what the annealed approximation misses. We keep the Ising perceptron of Section 2.6 as the running example, in the notation and normalization of that section: N binary weights $\theta \in \{\pm 1\}^N$, a binary teacher θ^*, Gaussian inputs, $n = \alpha N$ examples, and the partition function (2.39). The classic reference for everything here is Engel and Van den Broeck [26].

A.1 What the annealed approximation gets wrong

Jensen's inequality gives $\log \mathbb{E} Z \geq \mathbb{E} \log Z$, so the annealed free energy $-\log \mathbb{E} Z$ is always a lower bound on the quenched one. To see what drives the gap, write $Z = e^{-N\varphi}$, where the free-energy density φ fluctuates from dataset to dataset. Exponentiation turns modest fluctuations of φ into enormous fluctuations of Z: a dataset whose φ is below the typical value by a constant is exponentially rare, but it contributes an exponentially large Z, and the two exponentials compete at the same order in N. The average $\mathbb{E} Z$ can therefore be dominated by vanishingly rare datasets, while the typical dataset, which is what $\mathbb{E} \log Z$ describes, behaves differently. (This is the familiar behavior of lognormal random variables, whose mean sits far in the upper tail. The mean is a poor summary of a heavy-tailed quantity.)

For the Ising perceptron the direction of the error follows. The datasets that dominate $\mathbb{E} Z$ are those that constrain the student unusually weakly, and weakly constrained

students sit at low overlap. The annealed average therefore overweights poorly aligned students, and overestimates the amount of data needed to learn: it puts the transition to perfect generalization at $\alpha_c^{\text{ann}} \approx 1.45$, where the exact calculation sketched below gives $\alpha_c \approx 1.245$ [35]. The structure of the transition, two basins exchanging dominance, is the same in both.

Remark

Annealing does not always err so gently. Here the model is realizable (a teacher exists) and the fluctuations of φ are tame, so annealing shifts a number without changing the picture. In general the annealed computation can fail qualitatively, most famously for models with discrete weights at large load, where it can predict negative entropies (fewer than one consistent student, nonsense for a count) and spurious transition points.

A.2 Replicas

The full calculation computes $\mathbb{E}_{\text{data}} \log Z$ through the identity

$$\mathbb{E} \log Z = \lim_{Q \to 0} \frac{\mathbb{E} Z^Q - 1}{Q} \qquad \big(\text{from } Z^Q = e^{Q \log Z} = 1 + Q \log Z + O(Q^2)\big)\,. \tag{A.1}$$

The strategy is to compute $\mathbb{E} Z^Q$ for *integer* Q, where it has a transparent meaning, obtain a formula analytic in Q, and continue it to $Q \to 0$. For integer Q, Z^Q is the partition function of Q independent copies, or *replicas*, of the student, all judged against the same dataset. For the Ising perceptron, from (2.39),

$$\mathbb{E} Z^Q = \frac{1}{2^{NQ}} \sum_{\theta^1, \dots, \theta^Q \in \{\pm 1\}^N} \mathbb{E}_{\text{data}} \prod_{\mu=1}^{n} \prod_{a=1}^{Q} \Theta\big(y^\mu \, (\theta^a \cdot x^\mu)\big)\,. \tag{A.2}$$

The data average is easy, because the inputs are Gaussian. For fixed replicas and teacher, each example enters only through the $Q+1$ numbers $u^a = \theta^a \cdot x^\mu / \sqrt{N}$ and $v = \theta^* \cdot x^\mu / \sqrt{N}$, which are exactly jointly Gaussian (they are linear in x^μ), with mean zero and a covariance determined entirely by the *overlap matrix*

$$\begin{gathered} q^{ab} = \frac{\theta^a \cdot \theta^b}{N}\,, \qquad R^a = \frac{\theta^a \cdot \theta^*}{N}\,: \\ \mathbb{E}[u^a u^b] = q^{ab}\,, \quad \mathbb{E}[u^a v] = R^a\,, \quad \mathbb{E}[v^2] = 1\,. \end{gathered} \tag{A.3}$$

The product of step functions asks that every u^a have the sign of v, and the probability of that event is some function $e^{-e(q,R)}$ of the overlaps alone: a Gaussian orthant probability in $Q+1$ dimensions. Since the examples are independent, the data average of the product over μ is the n-th power, $e^{-n\, e(q,R)} = e^{-\alpha N\, e(q,R)}$. For $Q = 1$ the only overlap is R, the orthant is a wedge, and $e(R) = -\log(1 - \varepsilon(R))$ is the energy term of Section 2.6.

The disorder is gone. In its place, the replicas are coupled through the overlaps q^{ab}, and the sum over the Q binary vectors can be organized by the value of the overlaps, exactly as the sum over one binary vector was organized by R in Exercise 2.3(c). Writing $e^{N\,h(q,R)}$ for the number of Q-tuples of students with prescribed overlaps (for $Q=1$ this is the binomial count of Exercise 2.3(c)), and replacing the sum over overlaps by an integral,

$$\mathbb{E}Z^Q \;\approx\; \int dq\, dR\; e^{-N\,s_Q(q,R)}\,, \qquad s_Q(q,R) := Q\log 2 - h(q,R) + \alpha\, e(q,R)\,. \tag{A.4}$$

This is the replicated version of the one-variable action (2.44) of Section 2.6, to which it reduces at $Q=1$. At large N the integral is dominated by the minimum of s_Q over the overlaps, and the replica identity (A.1) then gives the quenched free-energy density

$$\Phi(\alpha) := -\frac{1}{N}\,\mathbb{E}_{\text{data}}\log Z \;=\; \lim_{Q\to 0}\;\frac{1}{Q}\;\min_{q,R}\; s_Q(q,R)\,. \tag{A.5}$$

Compare the annealed approximation, which is $\min_R s_1(R)$: the annealed calculation keeps a *single* replica, and so never sees the coupling q^{ab} between replicas that the shared data induce. That coupling is precisely the information about dataset-to-dataset fluctuations that Appendix A.1 found missing.

Replica symmetry. To make (A.5) concrete one needs h and e as functions of the $Q(Q+1)/2$ overlaps, and a way to continue the result in Q. The standard ansatz, *replica symmetry*, is to take $q^{ab}=q$ for all $a\neq b$ and $R^a=R$. It is natural, since the saddle-point equations are symmetric under permuting replicas, and it reduces the problem to two scalars. Under it, both h and e become one-dimensional Gaussian integrals: the orthant probability collapses to an integral over a single shared Gaussian "field" of the Gaussian tail function $H(x)=\int_x^\infty \frac{dz}{\sqrt{2\pi}}e^{-z^2/2}$, and the count h is handled by introducing conjugate variables for the constraints, which reduces it to a one-site problem in the manner of the mean-field Ising model of Section 2.4. The resulting expression is analytic in Q and can be differentiated at $Q=0$. As a caution, symmetric equations *can* have asymmetric solutions, and this happens in physical systems, famously in spin glasses; the assumption must be checked, and for the perceptron problems considered here it holds.

The result. For the Ising perceptron, the replica-symmetric calculation [35] reproduces the two-basin structure found in Exercise 2.3: an interior solution with $R^*(\alpha)<1$ that drifts toward the teacher as data accumulate, and the teacher solution $R=1$, which exchange dominance in a first-order transition. Only the number moves, from $\alpha_c^{\text{ann}}\approx 1.45$ to $\alpha_c\approx 1.245$. A guided walk through the calculation follows.

Exercise A.1 (The quenched calculation, via replicas). Work through the following program for the Ising perceptron, in as much detail as your appetite allows; full details are in [26, chs. 2 and 7].

(a) *Replicate.* For integer Q, write $\mathbb{E}Z^Q$ as in (A.2): a Q-fold copy of the system, coupled only through the shared data.

(b) *Average over data.* Show that for fixed replicas and teacher, $u^a = \theta^a \cdot x/\sqrt{N}$ and $v = \theta^* \cdot x/\sqrt{N}$ are exactly jointly Gaussian with the covariance (A.3), and that the data average factorizes across examples, each contributing the same function $e^{-e(q,R)}$ of the overlaps. Check that at $Q = 1$, $e(R) = -\log(1 - \varepsilon(R))$ with ε the wedge formula (2.41).

(c) *Push forward to the order parameters.* Insert delta functions fixing q^{ab} and R^a (in integral representation, with conjugate variables $\hat{q}^{ab}$, $\hat{R}^a$), so that the sum over the binary vectors factorizes over sites $i = 1, \dots, N$ and becomes the N-th power of a Q-spin sum. Conclude that $\mathbb{E}Z^Q$ takes the form (A.4), and that the annealed action (2.44) is its $Q = 1$ case.

(d) *Replica symmetry.* Take $q^{ab} = q$ for $a \neq b$ and $R^a = R$ (and likewise for the conjugates). Show that the orthant probability becomes $e^{-e_{\mathrm{RS}}} = 2 \int Dt\, H(\cdot)^Q$ for an appropriate argument of the tail function H, built from q, R, and a single Gaussian variable t, and that the site sum becomes a one-dimensional Gaussian integral of $(2\cosh(\cdot))^Q$. Continue both in Q and expand to first order at $Q = 0$.

(e) *Use the symmetry of Gibbs learning.* A student drawn uniformly from the consistent set is statistically indistinguishable from the teacher, since the teacher is itself a uniformly random binary vector consistent with the data. Argue that this forces $q = R$ at the saddle: the overlap between two Gibbs students equals their overlap with the teacher. One scalar remains, with one conjugate.

(f) *Solve.* The saddle-point equations for $R(\alpha)$ are transcendental. Solve them numerically and show that there are two competing solutions, an interior one and $R = 1$, whose free energies cross at $\alpha_c \approx 1.245$. Compare with the annealed $\alpha_c^{\mathrm{ann}} \approx 1.45$ of Exercise 2.3(g), and explain the direction of the shift using Appendix A.1.

A.3 A variant: the spherical perceptron

The replica method is easiest to push all the way to closed form in a variant of the problem with *continuous* weights, which is also the classic case in which the annealed and quenched learning curves can be compared analytically. Keep everything in Section 2.6 the same, but let the weights be real, $\theta, \theta^* \in \mathbb{R}^N$. Since $f_\theta = \mathrm{sign}(\theta \cdot x)$ is invariant under $\theta \to \lambda\theta$ with $\lambda > 0$, remove the redundant scale by putting the weights on a sphere,

$$|\theta|^2 = |\theta^*|^2 = N\,, \tag{A.6}$$

normalized so that individual weights stay of order one, and let the prior be the uniform measure $\pi(\theta)\, d\theta$ on the sphere. This is the *spherical perceptron*. The wedge formula (2.41) for the generalization error is unchanged, since its derivation used only that

$(\theta \cdot x, \theta^* \cdot x)/\sqrt{N}$ is a Gaussian pair with correlation R, which holds for any θ with $|\theta|^2 = N$. What changes is the entropy: the number of students at overlap R becomes the *volume* of the slice of the sphere at overlap R. Decompose $\theta = R\,\theta^* + \theta_\perp$ with $\theta_\perp \perp \theta^*$. The constraint (A.6) gives $|\theta_\perp|^2 = N(1-R^2)$, so the slice is a sphere of radius $\sqrt{N(1-R^2)}$ in the $(N-1)$-dimensional orthogonal complement, of volume $\propto \big(N(1-R^2)\big)^{(N-2)/2}$, and

$$\frac{1}{N}\log \int_{\theta:\, R(\theta)=R} d\theta\, \pi(\theta) \;=\; h(R) = \tfrac{1}{2}\log(1-R^2) + \text{const} + O\big(\tfrac{1}{N}\log N\big)\,. \tag{A.7}$$

Repeating the annealed calculation of Exercise 2.3(d) with this entropy gives

$$\begin{aligned} \mathbb{E}_{\text{data}} Z &\approx \int_{-1}^{1} dR\; e^{-N\, S_{\text{ann}}(R)}\,, \\ S_{\text{ann}}(R) &:= \underbrace{-\tfrac{1}{2}\log(1-R^2)}_{\text{entropy}} \; \underbrace{-\,\alpha \log\big(1-\varepsilon(R)\big)}_{\text{energy}}\,, \end{aligned} \tag{A.8}$$

and the annealed free-energy density is

$$\Phi_{\text{ann}}(\alpha) := \min_R S_{\text{ann}}(R)\,. \tag{A.9}$$

Figure 9 shows S_{ann} for several loads. The entropy–energy competition is the same as for the Ising perceptron, but the outcome is different: the entropy $-\frac{1}{2}\log(1-R^2)$ diverges as $R \to 1$, so the perfectly aligned state is infinitely penalized, the minimum always sits in the interior, and it slides continuously from $R = 0$ toward $R = 1$ as α grows. There is no phase transition. This is the answer to the closing question of Exercise 2.3(h): the binary weights of the Ising perceptron give the aligned state finite entropy, $h(1) = 0$, and that is what allows it to compete as a separate basin.

The learning curve. The large-α behavior follows by expanding near $R = 1$. Writing $R = \cos(\pi\varepsilon)$, so that ε is the generalization error itself, $1 - R^2 \approx (\pi\varepsilon)^2$ and

$$S_{\text{ann}} \approx -\log(\pi\varepsilon) + \alpha\,\varepsilon + \text{const}\,, \tag{A.10}$$

which is minimized at

$$\varepsilon_{\text{ann}}(\alpha) \simeq \frac{1}{\alpha}\,. \tag{A.11}$$

The generalization error decays as a *power law* in the amount of data, with exponent one: the simplest instance of the scaling laws of Section 7. The exact, quenched treatment [77, 26] runs through the replica steps of Exercise A.1 with the volume entropy (A.7) in place of the count (here replica symmetry can be justified by the convexity of the region of the sphere consistent with the data), and yields

$$\varepsilon_{\text{Gibbs}}(\alpha) \simeq \frac{0.625}{\alpha}\;: \tag{A.12}$$

the same exponent, with a different constant. As in Appendix A.1, the annealed average overweights poorly aligned students and overestimates the error, $1/\alpha > 0.625/\alpha$; and since the model is realizable, the exponent survives the approximation even though the constant does not.

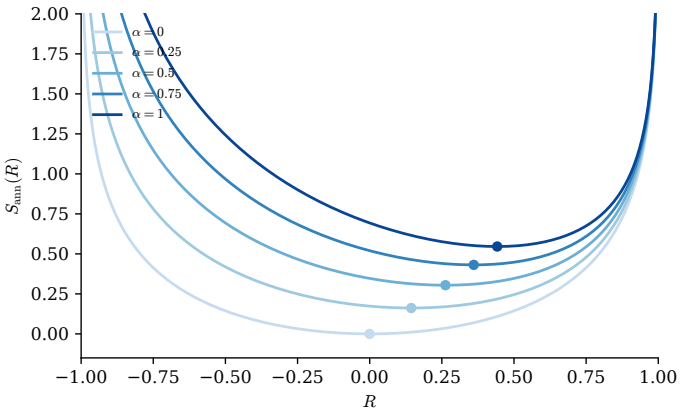


Figure 9: The annealed action $S_{\mathrm{ann}}(R)$ of the spherical perceptron, (A.8), for $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$ (bottom to top at $R = 0$), with the minimizer marked on each curve. The entropy pins the minimum at $R = 0$ when $\alpha = 0$; increasing the load slides it smoothly toward larger overlap, and the entropic wall at $R = 1$ keeps it in the interior. One basin at every α, moving continuously: no phase transition.

Exercise A.2 (The annealed learning curve of the spherical perceptron).

(a) Derive the slice entropy (A.7), and assemble the annealed action (A.8) by repeating the steps of Exercise 2.3(d).

(b) Derive the saddle-point condition for (A.9), and show that $R = 1$ is never a minimum, in contrast with Exercise 2.3(f).

(c) Verify the large-α learning curve (A.11), and show that at small α the annealed theory predicts $R \propto \alpha$. Sketch the full learning curve $\varepsilon(\alpha)$.

(d) Which steps of Exercise A.1 change when the count of binary students is replaced by the volume (A.7), and which do not?

B From linear regression to kernel regression

This appendix is a self-contained review of models that are *linear in their parameters*: linear regression, regression on fixed nonlinear features, kernel regression, and kernel *ridge* regression. It ends with the observation that a neural network linearized around its initialization — the ansatz (4.1) of Section 4 — is exactly such a model, so that training it amounts to kernel ridge regression. Nothing here requires large width, gradient-flow asymptotics, or moving features; those are the subjects of Sections 3 to 5, and this appendix supplies the linear algebra they rest on. Readers who found the kernel formulas of Sections 4.1 and 4.6 appearing out of thin air may want to read it

first.[6]

We use the notation of Section 3.1: training data $(x^\mu, y^\mu)_{\mu=1}^n$ with $x^\mu \in \mathbb{R}^d$ and $y^\mu \in \mathbb{R}$; a model $f_\theta : \mathbb{R}^d \to \mathbb{R}$ with parameters $\theta \in \mathbb{R}^P$; and $f^\mu := f_\theta(x^\mu)$ for its values on the training inputs. It is convenient to stack the data into matrices,

$$X := \begin{pmatrix} (x^1)^T \\ \vdots \\ (x^n)^T \end{pmatrix} \in \mathbb{R}^{n\times d}, \qquad Y := \begin{pmatrix} y^1 \\ \vdots \\ y^n \end{pmatrix} \in \mathbb{R}^n, \qquad f := \begin{pmatrix} f^1 \\ \vdots \\ f^n \end{pmatrix} \in \mathbb{R}^n, \tag{B.1}$$

so that $X^\mu{}_j = x^\mu_j$ is the j-th coordinate of the μ-th input, and the inputs and their labels are stacked in the same way. Throughout, the loss is the MSE of Section 4.1.1,

$$\mathcal{L}_\theta = \frac{1}{2}\sum_{\mu=1}^{n}(f^\mu - y^\mu)^2 = \frac{1}{2}\,|f - Y|^2 = \frac{1}{2}\,\delta_{\mu\nu}\Delta^\mu\Delta^\nu, \qquad \Delta^\mu := f^\mu - y^\mu, \tag{B.2}$$

with $\Delta \in \mathbb{R}^n$ the residual. Many references (including the notebook this appendix is drawn from) normalize the loss as $\frac{1}{2n}|f-Y|^2$ instead; this rescales the ridge parameter λ below by a factor of n and the learning rate η by a factor of $1/n$, and changes nothing else.

B.1 Linear regression

The simplest model is linear in the inputs,

$$f_\theta(x) = \theta \cdot x = \sum_{j=1}^{d} \theta^j x_j, \qquad \theta \in \mathbb{R}^d, \tag{B.3}$$

so that $P = d$ and the vector of training predictions is $f = X\theta$. (An intercept can be included by appending a constant coordinate $x_{d+1} = 1$ to every input, as in Exercise B.1 below.) The loss $\mathcal{L}_\theta = \frac{1}{2}|X\theta - Y|^2$ is a convex quadratic function of θ, with gradient

$$\nabla_\theta \mathcal{L} = X^T(X\theta - Y) = X^T\Delta. \tag{B.4}$$

This has the shape that every gradient in these notes has: the transpose of the Jacobian $\partial f^\mu/\partial\theta^j$ — here the constant matrix X — acting on the residual, *cf.* (4.10). Setting the gradient to zero gives the *normal equations*

$$X^T X\,\theta = X^T Y. \tag{B.5}$$

Since the loss is convex, every solution of (B.5) is a global minimizer, and there are no other critical points. What the solutions look like depends on how $P = d$ compares with n.

[6] This appendix adapts a lecture and an exercise notebook prepared by Charles Renshaw-Whitman for the Iliad Intensive, translated into the notation of Section 3.1.

Fewer parameters than data, $P < n$. If the columns of X are linearly independent — the generic situation when $d \leq n$ — then $X^T X$ is positive definite, since $v^T X^T X v = |Xv|^2 > 0$ for $v \neq 0$. There is a unique minimizer,

$$\theta_* = (X^T X)^{-1} X^T Y \,. \tag{B.6}$$

The normal equations say that the residual $\Delta_* = X\theta_* - Y$ is orthogonal to every column of X. Equivalently, the prediction $X\theta_*$ is the *orthogonal projection* of Y onto the column space $\mathrm{col}(X) \subset \mathbb{R}^n$, the set of all prediction vectors the model can produce (Figure 10). Fitting is projecting. The projection $X\theta_*$ is unique even in cases where θ_* is not.

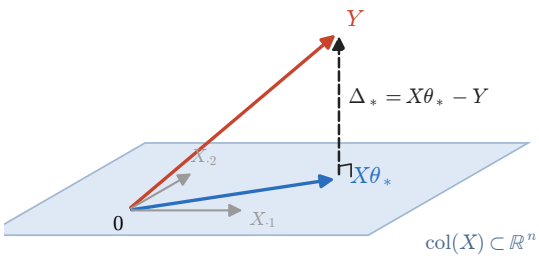


Figure 10: Least squares, geometrically. The normal equations $X^T \Delta_* = 0$ say that the residual is orthogonal to the column space of X, so the fitted prediction vector $X\theta_*$ is the orthogonal projection of Y onto $\mathrm{col}(X)$. "Orthogonal" refers to the Euclidean metric $\delta_{\mu\nu}$ on the space $\mathbb{R}^n$ of function values on the training set.

It is worth pausing on the word "orthogonal." Least squares minimizes the Euclidean (L^2) distance between $X\theta$ and Y, and the projection is orthogonal with respect to the Euclidean metric $\delta_{\mu\nu}$ on $\mathbb{R}^n$ — the same metric on the space of function values that was made explicit in Section 4.1.1. This metric is a *choice*, hidden inside the phrase "mean squared error." It declares that an error of a given size counts the same at every data point, and that errors at different data points are independent. Any other positive-definite metric $g_{\mu\nu}$ on $\mathbb{R}^n$ defines a different loss $\mathcal{L}^g = \frac{1}{2} g_{\mu\nu} \Delta^\mu \Delta^\nu$, with a different minimizer

$$\theta_*^g = (X^T g X)^{-1} X^T g Y \,, \tag{B.7}$$

which is the g-orthogonal projection of Y onto $\mathrm{col}(X)$ ("weighted least squares"). Unless Y already lies in $\mathrm{col}(X)$, different metrics give genuinely different fitted functions; see Exercise B.1(d). Note that $g_{\mu\nu}$ is a metric on the space of function values *indexed by* the data points, not a metric on the input space $\mathbb{R}^d$.

More parameters than data, $P > n$. If $d > n$, or more generally if the columns of X are linearly dependent, then $X^T X$ is singular. The normal equations still have solutions (Exercise B.1(b)), but now infinitely many: an affine subspace of $\mathbb{R}^P$ of dimension $P - \mathrm{rank}(X)$. If moreover X has full row rank n, then $Y \in \mathrm{col}(X)$ and *every*

solution interpolates the data exactly, $X\theta = Y$, with zero loss. The loss has no way to prefer one of them. Something else must make the choice; that is the subject of Appendix B.3, and it is where kernels come from. Far from being an edge case, $P > n$ is the normal situation for neural networks.

Exercise B.1 (Least squares by hand).

(a) Differentiate $\mathcal{L}_\theta = \frac{1}{2}|X\theta - Y|^2$ with respect to a single coordinate θ^j, assemble the components into (B.4), and derive the normal equations (B.5). Explain geometrically why they say that the residual is orthogonal to every column of X.

(b) Show that if X has full column rank then X^TX is positive definite, so that (B.6) is the unique minimizer. Show that the normal equations are always consistent (*i.e.* $X^TY \in \mathrm{col}(X^TX)$), and that if $P > n$ they have infinitely many solutions.

(c) Fit the affine function $f(x) = \theta^1 + \theta^2 x$ to the three points $x \in \{-1, 0, 1\}$ with labels $y = x^2$; that is, take

$$X = \begin{pmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}, \qquad Y = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}.$$

Find θ_*, the predictions, the residual, and the loss. Is the remaining error a failure of optimization or of representation?

(d) Repeat (c) with the weighted loss (B.7) for $g = \mathrm{diag}(1, 4, 1)$, which trusts the middle data point four times as much as the others. Check that the fitted function changes.

B.2 Linear regression with nonlinear features

"Linear model" is a statement about how the parameters enter, not about the shape of the fitted function. Choose, *before looking at any labels*, a *feature map*

$$\psi : \mathbb{R}^d \to \mathbb{R}^P, \qquad \psi(x) = \big(\psi_1(x), \dots, \psi_P(x)\big), \tag{B.8}$$

and consider the model

$$f_\theta(x) = \theta \cdot \psi(x) = \sum_{i=1}^{P} \theta^i \, \psi_i(x), \qquad \theta \in \mathbb{R}^P. \tag{B.9}$$

This is a weighted sum of P fixed shapes ψ_i. It can be as nonlinear in x as the shapes are, but it is linear in θ. Standard choices include monomials in the coordinates of x up to some degree m (giving $P = \binom{d+m}{m}$), Fourier modes, Gaussian bumps $e^{-|x-c_k|^2/2\sigma^2}$ centered at chosen points c_k, and *random features* $\psi_i(x) = \phi(a_i \cdot x)$ with the a_i drawn

at random and then frozen — the last being a one-hidden-layer network whose first layer is never trained.

Stacking the feature vectors of the training inputs into the $n \times P$ *feature matrix*

$$\Psi^{\mu}{}_{i} := \psi_i(x^{\mu}) \,, \tag{B.10}$$

the training predictions are $f = \Psi\theta$. Note that $\Psi^{\mu}{}_{i} = \partial f^{\mu}/\partial\theta^{i}$: the feature matrix is precisely the Jacobian J of Section 4.2, which for a linear model is a constant. Linear regression is the special case $\psi(x) = x$, $\Psi = X$, and *everything* in Appendix B.1 goes through verbatim with X replaced by Ψ: the normal equations are $\Psi^T\Psi\,\theta = \Psi^T Y$; if $P \le n$ and Ψ has full column rank, the unique minimizer is $\theta_* = (\Psi^T\Psi)^{-1}\Psi^T Y$; and if $P > n$ there are infinitely many interpolants.

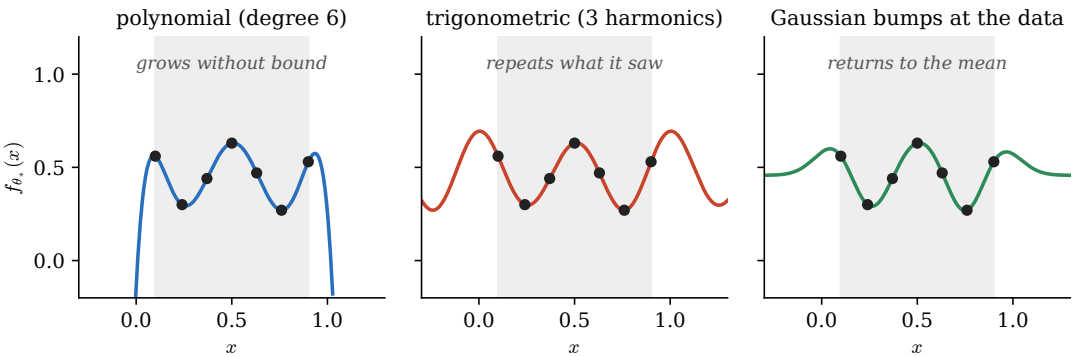


Figure 11: Choosing ψ is choosing the model. The same seven data points (black), interpolated exactly by three different fixed feature maps with $P = n = 7$: polynomials of degree six, trigonometric functions of period one with three harmonics, and Gaussian bumps centered at the data. All three drive the training loss to zero; they disagree everywhere outside the training range (shaded), and that disagreement was decided before the labels were read.

Replacing X by Ψ changes the algebra not at all, and changes the model completely. Figure 11 shows three feature maps, each with $P = n$, fitted to the same seven points. Each interpolates the data exactly, and they disagree everywhere else. Which is right? The data cannot say: the behavior away from the training set was fixed by the choice of ψ, which was made before the labels were seen. The feature map is the model's *inductive bias*, made explicit.

Why would one ever want a very large P? Figure 12 gives the classic answer. Two classes of points on concentric rings cannot be separated by a line in the plane. Append one fixed feature, $\psi(u, v) = (u,\, v,\, c\,(u^2 + v^2))$, and the classes become separable by a plane in $\mathbb{R}^3$: a model that is linear in the lifted coordinates is a circle in the original ones. The price is that P grows fast. All monomials of degree 3 in $d = 1000$ input coordinates already give $P = \binom{1002}{3} \approx 1.7 \times 10^8$, and Gaussian bumps centered at every point of a continuum give $P = \infty$. Yet there are only n data points, and the fit is determined by n numbers. Nothing about the problem should require a $P \times P$ inverse

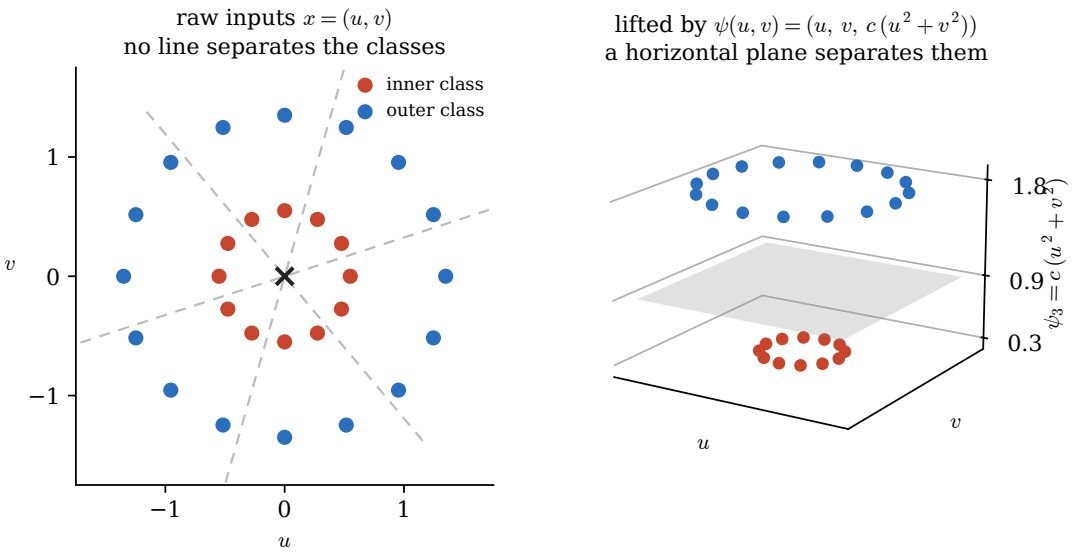


Figure 12: One fixed extra feature can make a hard problem linear. Left: two classes of points in the plane, on concentric rings. No line separates them, since the convex hull of each class contains the origin. Right: after appending the single feature $\psi_3 = c\,(u^2+v^2)$, the rings sit at two different heights, and a horizontal plane $\psi_3 = c\,r_0^2$ separates them. Nothing has been learned: we supplied the radial coordinate.

— so where does P actually enter? Answering that question honestly is the content of the next two sections.

Exercise B.2 (A three-point feature calculation). Take the same three points $x \in \{-1, 0, 1\}$ with labels $y = x^2$ as in Exercise B.1(c), but now the feature map $\psi(x) = (1,\ \sqrt{2}\,x,\ x^2)$.

(a) Form the feature matrix Ψ and find coefficients θ that interpolate all three labels. The representational failure of Exercise B.1(c) is gone: what changed?

(b) Explain precisely how the predictor can be nonlinear in x while the regression problem remains linear.

(c) If the last feature is replaced by $c\,x^2$ for some $c \neq 0$, what changes: the fitted function, its coefficient, or both?

(d) Write the inner product $K(x, z) := \psi(x) \cdot \psi(z)$ in closed form, without vector notation. This is a preview of Appendix B.3.

B.3 Kernel regression

Return to the overparametrized case $P > n$, and assume that Ψ has full row rank n (the generic situation), so that there is a $(P-n)$-dimensional affine family of exact interpolants $\Psi\theta = Y$. To choose one, we need a criterion that the loss does not provide. The standard choice is the interpolant of *minimum norm*,

$$\theta_* := \arg\min_{\Psi\theta = Y} |\theta|^2 \,. \tag{B.11}$$

This introduces a *second* implicit Euclidean metric — now δ_{ij} on parameter space $\mathbb{R}^P$, the metric that also defines gradient descent in Section 4.1.1. (Appendix B.1 used $\delta_{\mu\nu}$ on the space $\mathbb{R}^n$ of function values.) The two metrics play dual roles: the first decides how to fit when the data cannot be matched, the second decides which fit to choose when it can.

To solve (B.11), introduce Lagrange multipliers $\alpha \in \mathbb{R}^n$ for the n constraints and extremize $\frac{1}{2}|\theta|^2 - \alpha \cdot (\Psi\theta - Y)$. Stationarity in θ gives

$$\theta_* = \Psi^T \alpha = \sum_{\mu=1}^{n} \alpha_\mu \, \psi(x^\mu) \,. \tag{B.12}$$

This is the key structural fact: however large P is, the minimum-norm solution lies in the n-dimensional subspace of $\mathbb{R}^P$ spanned by the feature vectors of the training inputs, and is described by n numbers. (In the language of Section 4.2, θ_* lies in the row space of the Jacobian.) The constraint $\Psi\theta_* = Y$ then becomes $\Psi\Psi^T\alpha = Y$, an $n \times n$ linear system. Define the *kernel* and its *Gram matrix* on the training set,

$$\begin{aligned} K(x, x') &:= \psi(x) \cdot \psi(x') = \sum_{i=1}^{P} \psi_i(x)\psi_i(x') \,, \\ \hat{K}^{\mu\nu} &:= K(x^\mu, x^\nu) = (\Psi\Psi^T)^{\mu\nu} \,. \end{aligned} \tag{B.13}$$

The kernel is an inner product of feature vectors: a notion of *similarity* between inputs, fixed once ψ is fixed, and involving no labels. The Gram matrix is symmetric and positive semidefinite, since $c^T\hat{K}c = |\Psi^T c|^2 \geq 0$; it is positive definite exactly when Ψ has full row rank, which we assumed. Then $\alpha = \hat{K}^{-1}Y$, and the prediction at an arbitrary input x is

$$f_*(x) = \theta_* \cdot \psi(x) = \sum_{\mu=1}^{n} \alpha_\mu K(x, x^\mu) = \sum_{\mu,\nu=1}^{n} K(x, x^\mu)\, \hat{K}^{-1}_{\mu\nu}\, y^\nu \,. \tag{B.14}$$

This is *kernel regression* (more precisely, kernel interpolation). The feature map has disappeared from both the fit and the prediction. To fit, one needs K on pairs of training points; to predict, one needs K between the test point and the training points; no feature vector is ever constructed. The prediction is a weighted sum of similarities to the training points, with weights α that depend on the labels — but the labels only reweight the training points, they never change which inputs K considers similar.

Compare (B.14) with the late-time mean (4.38) of a lazily trained network: they are the same formula, with $\hat{K}$ in place of the NTK. This is no coincidence. For any model linear in its parameters the Jacobian is $J = \Psi$, so the NTK (4.10) is

$$\Theta^{\mu\nu} = (JJ^T)^{\mu\nu} = \hat{K}^{\mu\nu}\,, \tag{B.15}$$

constant in time. Moreover, gradient flow finds the minimum-norm solution: $\dot{\theta} = -\eta\,\Psi^T\Delta$ always points along the row space of Ψ, so $\theta(t)-\theta_0$ stays in that n-dimensional subspace and converges to the interpolant nearest to θ_0 — for $\theta_0 = 0$, to θ_* (Exercise B.3(c)). This is the simplest instance of the *implicit bias* of gradient descent, and it is the Euclidean metric on parameter space entering through the back door: the flow is defined by δ^{ij}, and so is the norm that it ends up minimizing.

The kernel trick. There are two readings of (B.14). Read from the feature side, it rewrites a $P \times P$ problem as an $n \times n$ one. Read from the kernel side, it says the feature map is dispensable: any function $K(x, x')$ that arises as an inner product of features will do, whether or not one can write the features down. For $x, z \in \mathbb{R}^2$ and $\psi(x) = (x_1^2,\, \sqrt{2}\,x_1x_2,\, x_2^2)$, one checks that $\psi(x)\cdot\psi(z) = (x\cdot z)^2$: a single dot product, squared, stands in for three features. In d dimensions, $K(x,z) = (x\cdot z)^m$ costs $O(d)$ to evaluate and stands in for all $\binom{d+m-1}{m}$ monomials of degree m. The Gaussian kernel $K(x,z) = e^{-|x-z|^2/2\sigma^2}$ corresponds to an infinite-dimensional feature map that no one ever writes down. Conversely, any symmetric K all of whose Gram matrices are positive semidefinite arises from some feature map (Mercer's theorem): the eigenfunctions of K, weighted by the square roots of its eigenvalues, furnish one (*cf.* Sections 3.3 and 4.6). This is a trade, not a free lunch. The feature route scales with P and the kernel route with n; with many examples and few features, the explicit route is the cheap one.

Exercise B.3 (Minimum-norm interpolation and the kernel).

(a) Carry out the Lagrange-multiplier computation leading to (B.12)–(B.14). Alternatively, use the singular value decomposition $\Psi = USV^T$ of Section 4.2 to show that the minimum-norm interpolant is $\theta_* = \Psi^T(\Psi\Psi^T)^{-1}Y$, and write it in terms of U, S, V.

(b) Show that $c^T\hat{K}c = |\Psi^T c|^2 \geq 0$ for all $c \in \mathbb{R}^n$. Interpret a zero eigenvalue of $\hat{K}$ as a linear combination of training examples that vanishes in feature space. What does it take for $\hat{K}$ to be invertible?

(c) Show that under gradient flow $\dot{\theta} = -\eta\,\Psi^T(\Psi\theta - Y)$ the displacement $\theta(t) - \theta_0$ stays in the row space of Ψ for all t; derive $\Delta(t) = e^{-\eta t\hat{K}}\Delta(0)$; and conclude that, for invertible $\hat{K}$, the flow converges to the interpolant closest to θ_0. Derive (4.36) for this model, with $\Theta = \hat{K}$.

(d) Verify that $\psi(x)\cdot\psi(z) = (x\cdot z)^2$ for $\psi(x) = (x_1^2, \sqrt{2}\,x_1x_2, x_2^2)$, and explain why the $\sqrt{2}$ is not decoration. Which feature map on $\mathbb{R}$ produces the kernel

$(1+xz)^2$ of Exercise B.2(d)? How many features does $(x\cdot z)^m$ on $\mathbb{R}^d$ stand in for?

B.4 Kernel ridge regression

Rather than imposing the minimum-norm criterion as a hard constraint, one can add it to the loss as a penalty. For $\lambda > 0$, *ridge regression* minimizes

$$\mathcal{L}_\lambda(\theta) := \frac{1}{2}|\Psi\theta - Y|^2 + \frac{\lambda}{2}|\theta|^2 . \tag{B.16}$$

The two terms measure distance to the data with $\delta_{\mu\nu}$ and distance to the origin with δ_{ij}: ridge regression is a weighted compromise between the two metrics of Appendices B.1 and B.3, with λ setting the exchange rate. (In the training of neural networks the penalty is called *weight decay*.) The penalty does two jobs at once. When $P > n$ it selects one solution out of infinitely many; and for any P it makes the problem well posed even when the features are redundant or the labels are noisy. Note that the penalty is not invariant under rescaling features: it decides what counts as a "large" coefficient in the coordinates ψ that were supplied.

The gradient is $\Psi^T(\Psi\theta - Y) + \lambda\theta$, so the stationarity condition is

$$(\Psi^T\Psi + \lambda I_P)\,\theta = \Psi^T Y . \tag{B.17}$$

The matrix on the left is positive definite for every $\lambda > 0$, whatever the rank of Ψ, since $v^T(\Psi^T\Psi + \lambda I_P)v = |\Psi v|^2 + \lambda|v|^2 > 0$. So there is always a unique minimizer,

$$\theta_\lambda = (\Psi^T\Psi + \lambda I_P)^{-1}\Psi^T Y , \tag{B.18}$$

at the price of a $P \times P$ inverse. To move the inverse to the $n \times n$ side, expand and regroup the product

$$\Psi^T(\Psi\Psi^T + \lambda I_n) = \Psi^T\Psi\Psi^T + \lambda\Psi^T = (\Psi^T\Psi + \lambda I_P)\Psi^T . \tag{B.19}$$

Both bracketed matrices are positive definite, so multiplying by their inverses on the appropriate sides gives the *push-through identity*

$$(\Psi^T\Psi + \lambda I_P)^{-1}\Psi^T = \Psi^T(\Psi\Psi^T + \lambda I_n)^{-1} . \tag{B.20}$$

Nothing was cancelled — Ψ^T is rectangular and has no inverse — and that is precisely why the identity holds. Substituting into (B.18), the ridge solution again lies in the span of the training feature vectors, $\theta_\lambda = \Psi^T\alpha_\lambda$, now with $\alpha_\lambda = (\hat{K} + \lambda I_n)^{-1}Y$, and the prediction at a new input is

$$f_\lambda(x) = \sum_{\mu,\nu=1}^{n} K(x, x^\mu)\left(\hat{K} + \lambda I_n\right)^{-1}_{\mu\nu} y^\nu . \tag{B.21}$$

This is *kernel ridge regression*, the central formula of this appendix. As $\lambda \to 0$ it reduces to the minimum-norm interpolant (B.14) when $\hat{K}$ is invertible, and in general to the least-squares solution of minimum norm (Exercise B.4(c)). As $\lambda \to \infty$, $\theta_\lambda \to 0$.

Formula (B.21) is exactly the posterior mean of Exercise 4.5, with ridge parameter $\lambda = \beta^{-1}$ — there derived for the infinite-width Gaussian process, here for a finite linear model. The Bayesian reading is worth making explicit. With a Gaussian likelihood $\propto e^{-\frac{\beta}{2}|\Psi\theta - Y|^2}$ and a Gaussian prior $\theta \sim \mathcal{N}(0, \sigma^2 I_P)$, the loss (B.16) is $-\beta^{-1}$ log of the posterior density, up to a constant, with $\lambda = 1/(\beta\sigma^2)$. Ridge regression is thus maximum a posteriori estimation. The prior pushed forward to $f_\theta(x) = \theta \cdot \psi(x)$ is a Gaussian process with kernel $\sigma^2 K(x, x')$, a finite-P instance of Section 3.3, and the posterior mean of Exercise 4.5 for this process reproduces (B.21).

Spectral view. With the singular value decomposition $\Psi = USV^T$ of Section 4.2, the Gram matrix is $\hat{K} = US^2U^T$, with eigenvalues s_a^2 and orthonormal eigenvectors $u_a \in \mathbb{R}^n$: the "features" of Section 4.2, *i.e.* the natural modes in the space of function values. The ridge predictions on the training set are $f_\lambda = \Psi\theta_\lambda = \hat{K}(\hat{K} + \lambda I_n)^{-1}Y$, or, mode by mode,

$$f_\lambda = \sum_a \frac{s_a^2}{s_a^2 + \lambda} \, (u_a \cdot Y) \, u_a \, . \tag{B.22}$$

Ridge regression is a *spectral filter*: modes of the data with $s_a^2 \gg \lambda$ are fit, modes with $s_a^2 \ll \lambda$ are suppressed, and the crossover is at $s_a^2 \approx \lambda$. Compare gradient flow from $\theta_0 = 0$ on the unpenalized loss, whose training predictions are, by (4.35),

$$f(t) = \sum_a \left(1 - e^{-\eta t s_a^2}\right) (u_a \cdot Y) \, u_a \, . \tag{B.23}$$

This is also a spectral filter, with $1/\eta t$ playing the role of λ: *early stopping* regularizes much as ridge does. But the two filters have different shapes (Figure 13), and no choice of λ reproduces a given stopping time on every mode at once (Exercise B.5). This is the precise content of the remark closing Section 4.6, that the inverse temperature of Bayesian learning and the training time of gradient descent play analogous roles.

Exercise B.4 (Ridge regression, two ways).

(a) Derive (B.17) and (B.18), and prove that $\Psi^T\Psi + \lambda I_P$ is invertible even when Ψ is rank deficient.

(b) Prove the push-through identity (B.20) and deduce (B.21). Explain why "cancelling Ψ^T" is not a valid step, and why the identity is nevertheless true. (It is a special case of $A(BA + I)^{-1} = (AB + I)^{-1}A$.)

(c) What happens as $\lambda \to \infty$? Using the SVD, show that θ_λ applies the scalar filter $s/(s^2 + \lambda)$ to each singular value s of Ψ, and hence that as $\lambda \to 0$ it converges to the least-squares solution of minimum norm. When does this limit interpolate the labels exactly?

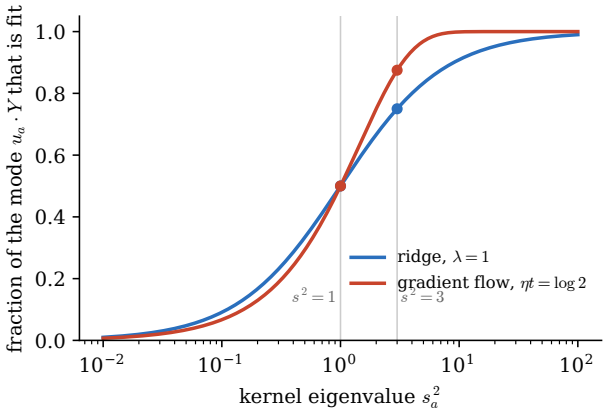


Figure 13: Ridge regression and early-stopped gradient flow as spectral filters: the fraction of a data mode $u_a \cdot Y$ that is fit, as a function of the mode's kernel eigenvalue s_a^2. The two are tuned to agree on the mode with $s^2 = 1$, and disagree elsewhere; the dots mark the two-mode example of Exercise B.5.

(d) Return to $x \in \{-1, 0, 1\}$, $y = x^2$ and the kernel $K(x, z) = (1 + xz)^2$ of Exercise B.2. With $\lambda = 1$, verify that

$$\hat{K} = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 4 \end{pmatrix}, \qquad \alpha_\lambda = \frac{1}{4} \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix},$$

and predict at $z = 2$ using only kernel evaluations. Compare with the interpolant of Exercise B.2(a), which predicts $z^2 = 4$ there, and explain the direction of the discrepancy.

(e) The feature route solves a $P \times P$ system and the kernel route an $n \times n$ one. When is each attractive?

Exercise B.5 (Early stopping vs. ridge, on two data points). Take $n = 2$ training points with Gram matrix and labels

$$\hat{K} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad Y = \begin{pmatrix} 2 \\ 0 \end{pmatrix},$$

and initial predictions $f(0) = 0$.

(a) Verify the eigenpairs $u_\pm = \frac{1}{\sqrt{2}}(1, \pm 1)^T$ with $s_\pm^2 = 3, 1$, and decompose Y into these modes.

(b) Using (B.23), find the training predictions $f(t)$ under gradient flow, and evaluate them at $\eta t = \log 2$. Why do both the eigenvalue s_a^2 and the initial projection $u_a \cdot Y$ matter for what is visible during training?

(c) Using (B.22), find the ridge predictions for $\lambda = 1$. The two filters agree on the mode with $s^2 = 1$; show that the ridge parameter matching gradient flow at time t on a mode with eigenvalue s^2 is $\lambda_{\text{eff}}(s^2, t) = s^2/(e^{\eta t s^2} - 1)$, which depends on the mode — so early stopping is not ridge regression.

(d) State precisely what "spectral bias" means for this finite, fixed-kernel calculation. What additional structure would be needed to turn it into a claim about Fourier frequencies, about robustness to label noise, or about test error?

B.5 Linearized networks are kernel ridge regressors

Now let f_θ be a neural network, with $\theta \in \mathbb{R}^P$ the vector of *all* its weights and biases. It is not linear in θ. But its linearization around the initialization θ_0, the ansatz (4.1),

$$f_\theta^{\text{lin}}(x) = f_{\theta_0}(x) + (\theta - \theta_0) \cdot \nabla_\theta f_{\theta_0}(x)\,, \tag{B.24}$$

is a model of the form (B.9). Compare term by term. The parameters are the displacement $\delta\theta := \theta - \theta_0 \in \mathbb{R}^P$. The feature map is

$$\psi(x) = \nabla_\theta f_{\theta_0}(x) \in \mathbb{R}^P\,, \tag{B.25}$$

the *tangent features*: one number per parameter, the sensitivity of the output at x to that parameter, evaluated at initialization and then frozen. And the constant offset $f_{\theta_0}(x)$ is absorbed by shifting the labels to

$$\tilde{y}^\mu := y^\mu - f_{\theta_0}(x^\mu) = -\Delta^\mu(0)\,. \tag{B.26}$$

The offset matters: the linearized model is fit to the *residuals at initialization*, not to the raw labels. The feature matrix (B.10) is the Jacobian at initialization, $\Psi = J_0$, and the Gram matrix (B.13) is

$$\hat{K} = J_0 J_0^T = \Theta_0\,, \qquad K(x, x') = \Theta(x, x') = \nabla_\theta f_{\theta_0}(x) \cdot \nabla_\theta f_{\theta_0}(x')\,, \tag{B.27}$$

the neural tangent kernel (4.10) at initialization, extended to arbitrary pairs of inputs as in Exercise 4.3. Everything in Appendices B.3 and B.4 now applies verbatim. In particular, ridge regression on the linearized network with penalty $\frac{\lambda}{2}|\theta - \theta_0|^2$ — weight decay toward the initialization — gives, by (B.21),

$$f_\lambda^{\text{lin}}(x) = f_{\theta_0}(x) - \sum_{\mu,\nu=1}^{n} \Theta(x, x^\mu) \left(\Theta_0 + \lambda I_n\right)^{-1}_{\mu\nu} \Delta^\nu(0)\,. \tag{B.28}$$

At $\lambda \to 0$ this is the $t \to \infty$ limit of (4.36), the minimum-norm interpolant of the residuals; and the finite-time solution (4.36) is its early-stopped cousin, in the sense of (B.23). The kernel trick is not a convenience here but a necessity: P is the parameter count of the network, often 10^7 or more, while Θ_0 is $n \times n$.

Two remarks on what is and is not being claimed. First, for the linearized model f^{lin} everything above is exact, and its kernel is constant by construction. For the actual network, (B.24) is an ansatz. Section 4.3 shows that it becomes exact in the large-width limit at fixed depth and data; whether and when it is a good approximation otherwise — and what happens when the tangent features move — is the subject of Sections 5 to 6, not of this appendix. Second, the initialization θ_0 is random, so f_{θ_0}, the tangent features, and the kernel Θ_0 are all random too, and (B.28) is the trained function for one draw. Averaging over draws, at large width where Θ_0 becomes deterministic, gives the Gaussian process (4.38).

Exercise B.6 (Tangent features of a one-hidden-layer network). Consider $f_\theta(x) = \sum_{i=1}^{N} c_i\, \phi(a_i \cdot x)$ with $\theta = (c_i, a_i)_{i=1}^{N}$, so that $P = N(d+1)$.

(a) Compute the tangent features $\nabla_\theta f_{\theta_0}(x)$, and show that the tangent kernel is

$$\Theta(x, x') = \sum_{i=1}^{N} \phi(a_i \cdot x)\, \phi(a_i \cdot x') + \sum_{i=1}^{N} c_i^2\, \phi'(a_i \cdot x)\, \phi'(a_i \cdot x')\, (x \cdot x')\,.$$

(b) Suppose only the output weights c are trained, with the a_i frozen at their random initial values. Show that this is exactly a fixed-feature model (B.9) with random features, that its linearization is exact, and that its kernel is the first term above. Which term do you get by training only the a_i?

(c) Show that gradient flow with weight decay toward the initialization, $\dot\theta = -\eta\big[\nabla_\theta \mathcal{L} + \lambda(\theta - \theta_0)\big]$, applied to the linearized model f^{lin}, converges to the ridge solution (B.28) for every $\lambda > 0$ and from any starting point.

(d) Check that (B.28) at $\lambda \to 0$ coincides with (4.36) at $t \to \infty$.

C Solutions to exercises

Worked solutions to the exercises, in the order in which the exercises appear. (Instructors: uncommenting `\solutionsfalse` in the preamble removes this appendix's contents, and all other solution boxes, from the compiled PDF.)

References

[1] Elisabeth Agoritsas, Giulio Biroli, Pierfrancesco Urbani, and Francesco Zamponi. Out-of-equilibrium dynamical mean-field equations for the perceptron model. *Journal of Physics A: Mathematical and Theoretical*, 51(8):085002, 2018. arXiv:1710.04894.

[2] Alexander Altland and Ben D. Simons. *Condensed Matter Field Theory*. Cambridge University Press, Cambridge, 2 edition, 2010.

[3] Daniel J. Amit, Hanoch Gutfreund, and H. Sompolinsky. Spin-glass models of neural networks. *Physical Review A*, 32(2):1007–1018, August 1985.

[4] Alexander Atanasov, Blake Bordelon, and Cengiz Pehlevan. Neural networks as kernel learners: the silent alignment effect. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2111.00034.

[5] Yasaman Bahri, Ethan Dyer, Jared Kaplan, Jaehoon Lee, and Utkarsh Sharma. Explaining neural scaling laws. *Proceedings of the National Academy of Sciences*, 121(27):e2311878121, 2024. arXiv:2102.06701.

[6] Yasaman Bahri, Jonathan Kadmon, Jeffrey Pennington, Samuel S. Schoenholz, Jascha Sohl-Dickstein, and Surya Ganguli. Statistical mechanics of deep learning. *Annual Review of Condensed Matter Physics*, 11:501–528, 2020.

[7] Jürgen Berges. Introduction to Nonequilibrium Quantum Field Theory. *AIP Conference Proceedings*, 739(1):3–62, December 2004.

[8] Emmy Blumenthal. Building Intuition for Dynamical Mean-Field Theory: A Simple Model and the Cavity Method, July 2025.

[9] Blake Bordelon, Alexander Atanasov, and Cengiz Pehlevan. A dynamical model of neural scaling laws. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.01092.

[10] Blake Bordelon, Abdulkadir Canatar, and Cengiz Pehlevan. Spectrum dependent learning curves in kernel regression and wide neural networks. In *International Conference on Machine Learning (ICML)*, 2020. arXiv:2002.02561.

[11] Blake Bordelon, Lorenzo Noci, Mufan Bill Li, Boris Hanin, and Cengiz Pehlevan. Depthwise Hyperparameter Transfer in Residual Networks: Dynamics and Scaling Limit, December 2023.

[12] Blake Bordelon and Cengiz Pehlevan. Self-consistent dynamical field theory of kernel evolution in wide neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.09653.

[13] Blake Bordelon and Cengiz Pehlevan. Dynamics of Finite Width Kernel and Prediction Fluctuations in Mean Field Neural Networks, November 2023.

[14] Ari Brill. Neural scaling laws rooted in the data distribution, 2024. arXiv:2412.07942.

[15] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: scaling inference compute with repeated sampling, 2024. arXiv:2407.21787.

[16] Abdulkadir Canatar, Blake Bordelon, and Cengiz Pehlevan. Spectral bias and task-model alignment explain generalization in kernel regression and infinitely wide neural networks. *Nature Communications*, 12:2914, 2021.

[17] Michael Celentano, Chen Cheng, and Andrea Montanari. The high-dimensional asymptotics of first order methods with random data, 2021. arXiv:2112.07572.

[18] Zhongtian Chen, Edmund Lau, Jake Mendel, Susan Wei, and Daniel Murfet. Dynamical versus bayesian phase transitions in a toy model of superposition. *arXiv preprint arXiv:2310.06301*, 2023.

[19] Lénaïc Chizat and Francis Bach. On the global convergence of gradient descent for over-parameterized models using optimal transport. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. arXiv:1805.09545.

[20] Lénaïc Chizat, Edouard Oyallon, and Francis Bach. On lazy training in differentiable programming. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1812.07956.

[21] Youngmin Cho and Lawrence K. Saul. Kernel methods for deep learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2009.

[22] Carson C. Chow and Michael A. Buice. Path Integral Methods for Stochastic Differential Equations, October 2012.

[23] Leticia F. Cugliandolo. Recent Applications of Dynamical Mean-Field Methods, May 2023.

[24] Simon S. Du, Xiyu Zhai, Barnabás Póczos, and Aarti Singh. Gradient descent provably optimizes over-parameterized neural networks. In *International Conference on Learning Representations (ICLR)*, 2019.

[25] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022.

[26] Andreas Engel and Christian Van den Broeck. *Statistical Mechanics of Learning*. Cambridge University Press, 2001.

[27] Zhou Fan, Justin Ko, Bruno Loureiro, Yue M. Lu, and Yandi Shen. Dynamical mean-field analysis of adaptive Langevin diffusions: propagation-of-chaos and convergence of the linear response, 2025. arXiv:2504.15556.

[28] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2210.10760.

[29] Crispin W. Gardiner. *Handbook of Stochastic Methods: for Physics, Chemistry and the Natural Sciences*. Springer, 3 edition, 2004.

[30] Mario Geiger, Stefano Spigler, Arthur Jacot, and Matthieu Wyart. Disentangling feature and lazy training in deep neural networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2020(11):113301, 2020. arXiv:1906.08034.

[31] Antoine Georges, Gabriel Kotliar, Werner Krauth, and Marcelo J. Rozenberg. Dynamical mean-field theory of strongly correlated fermion systems and the limit of infinite dimensions. *Reviews of Modern Physics*, 68(1):13–125, January 1996.

[32] Cédric Gerbelot, Emanuele Troiani, Francesca Mignacco, Florent Krzakala, and Lenka Zdeborová. Rigorous dynamical mean-field theory for stochastic gradient descent methods. *SIAM Journal on Mathematics of Data Science*, 6(2), 2024. arXiv:2210.06591.

[33] Nigel Goldenfeld. *Lectures On Phase Transitions And The Renormalization Group*. CRC Press, Boca Raton, March 2018.

[34] Andrey Gromov. Grokking modular arithmetic, 2023. arXiv:2301.02679.

[35] Géza Györgyi. First-order transition to perfect generalization in a neural network with binary synapses. *Physical Review A*, 41(12):7097–7100, 1990.

[36] Boris Hanin. Random neural networks in the infinite width limit as gaussian processes. *The Annals of Applied Probability*, 33(6A):4798–4819, 2023.

[37] Boris Hanin. Special lectures on the mathematics of deep learning. Five-lecture series, Department of Mathematics, University of Chicago, 6–9 April 2026. Video recordings, YouTube, 2026. https://www.youtube.com/@uchicagomath.

[38] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: surpassing human-level performance on ImageNet classification. In *International Conference on Computer Vision (ICCV)*, 2015. arXiv:1502.01852.

[39] Tom Henighan, Jared Kaplan, Mor Katz, Mark Chen, Christopher Hesse, Jacob Jackson, Heewoo Jun, Tom B. Brown, Prafulla Dhariwal, Scott Gray, et al. Scaling laws for autoregressive generative modeling, 2020. arXiv:2010.14701.

[40] Danny Hernandez, Jared Kaplan, Tom Henighan, and Sam McCandlish. Scaling laws for transfer, 2021. arXiv:2102.01293.

[41] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2203.15556.

[42] Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. arXiv:1806.07572.

[43] Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. Large language models struggle to learn long-tail knowledge. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.08411.

[44] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[45] Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S. Dhillon, David Brandfonbrener, and Rishabh Agarwal. The art of scaling reinforcement learning compute for LLMs, 2025. arXiv:2510.13786.

[46] G. Kotliar, S. Y. Savrasov, K. Haule, V. S. Oudovenko, O. Parcollet, and C. A. Marianetti. Electronic structure calculations with dynamical mean-field theory. *Rev. Mod. Phys.*, 78:865–951, Aug 2006.

[47] R. Kubo. The fluctuation-dissipation theorem. *Reports on Progress in Physics*, 29(1):255, January 1966.

[48] Ryogo Kubo. Statistical-Mechanical Theory of Irreversible Processes. I. General Theory and Simple Applications to Magnetic and Conduction Problems. *Journal of the Physical Society of Japan*, 12(6):570–586, June 1957.

[49] Tanishq Kumar, Blake Bordelon, Samuel J. Gershman, and Cengiz Pehlevan. Grokking as the transition from lazy to rich training dynamics, 2024. arXiv:2310.06110.

[50] T. Lancaster and S. Blundell. *Quantum Field Theory for the Gifted Amateur*. OUP Oxford, 2014.

[51] Yann LeCun, Léon Bottou, Genevieve B. Orr, and Klaus-Robert Müller. Efficient backprop. In *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*. Springer, 1998.

[52] Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein. Deep neural networks as Gaussian processes. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1711.00165.

[53] Chan Li and Nigel Goldenfeld. Broken Ergodicity and the Violation of the Fluctuation-Dissipation Theorem Lead to Generalization Beyond Overfitting in Machine Learning, July 2026.

[54] J. Lin. Feature identification via the empirical NTK, 2025. arXiv:2510.00468.

[55] Alexander Maloney, Daniel A. Roberts, and James Sully. A solvable model of neural scaling laws. *arXiv preprint arXiv:2210.16859*, 2022.

[56] Song Mei, Andrea Montanari, and Phan-Minh Nguyen. A mean field view of the landscape of two-layer neural networks. *Proceedings of the National Academy of Sciences*, 115(33):E7665–E7671, 2018.

[57] Eric J. Michaud, Ziming Liu, Uzay Girit, and Max Tegmark. The quantization model of neural scaling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.13506.

[58] Francesca Mignacco, Florent Krzakala, Pierfrancesco Urbani, and Lenka Zdeborová. Dynamical mean-field theory for stochastic gradient descent in Gaussian mixture classification*. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12):124008, December 2021.

[59] Francesca Mignacco and Pierfrancesco Urbani. The effective noise of Stochastic Gradient Descent. *Journal of Statistical Mechanics: Theory and Experiment*, 2022(8):083405, August 2022.

[60] Andrea Montanari and Pierfrancesco Urbani. Dynamical Decoupling of Generalization and Overfitting in Large Two-Layer Networks, October 2025.

[61] Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.16264.

[62] Radford M. Neal. *Bayesian Learning for Neural Networks*, volume 118 of *Lecture Notes in Statistics*. Springer, 1996.

[63] Lars Onsager and Stefan Machlup. Fluctuations and irreversible processes. *Physical Review*, 91(6):1505–1512, 1953.

[64] R. K. Pathria and Paul D. Beale. *Statistical Mechanics*. Elsevier/Academic Press, Amsterdam ; Boston, 3rd ed edition, 2011.

[65] Ben Poole, Subhaneil Lahiri, Maithra Raghu, Jascha Sohl-Dickstein, and Surya Ganguli. Exponential expressivity in deep neural networks through transient chaos. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1606.05340.

[66] Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. Resolving discrepancies in compute-optimal scaling of language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.19146.

[67] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: generalization beyond overfitting on small algorithmic datasets, 2022. arXiv:2201.02177.

[68] Charles Renshaw-Whitman. Iliad physics learning: lectures and exercise notebooks for the august 2026 iliad intensive. https://github.com/CharlesR-W/IliadPhysicsLearning, 2026.

[69] Zohar Ringel, Noa Rubin, Edo Mor, Moritz Helias, and Inbar Seroussi. *Applications of Statistical Field Theory in Deep Learning*. Foundations and Trends in Machine Learning. Now Publishers, 2025. arXiv:2502.18553.

[70] Hannes Risken. *The Fokker–Planck Equation: Methods of Solution and Applications*. Springer, 2 edition, 1996.

[71] Daniel A. Roberts, Sho Yaida, and Boris Hanin. *The Principles of Deep Learning Theory: An Effective Theory Approach to Understanding Neural Networks*. Cambridge University Press, 2022. arXiv:2106.10165.

[72] Grant M. Rotskoff and Eric Vanden-Eijnden. Trainability and accuracy of artificial neural networks: an interacting particle system approach. *Communications on Pure and Applied Mathematics*, 75(9):1889–1935, 2022. arXiv:1805.00915.

[73] Noa Rubin, Inbar Seroussi, and Zohar Ringel. Grokking as a first order phase transition in two layer networks. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[74] Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2304.15004.

[75] Samuel S. Schoenholz, Justin Gilmer, Surya Ganguli, and Jascha Sohl-Dickstein. Deep information propagation. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1611.01232.

[76] Inbar Seroussi, Gadi Naveh, and Zohar Ringel. Separation of scales and a thermodynamic description of feature learning in some cnns. *Nature Communications*, 14:908, 2023.

[77] H. Sebastian Seung, Haim Sompolinsky, and Naftali Tishby. Statistical mechanics of learning from examples. *Physical Review A*, 45(8):6056–6091, 1992.

[78] Utkarsh Sharma and Jared Kaplan. A neural scaling law from the dimension of the data manifold, 2020. arXiv:2004.10802.

[79] James B. Simon, Madeline Dickens, Dhruva Karkada, and Michael R. DeWeese. The eigenlearning framework: a conservation law perspective on kernel ridge regression and wide neural networks. *Transactions on Machine Learning Research (TMLR)*, 2023. arXiv:2110.03922.

[80] Jamie Simon, Daniel Kunin, Alexander Atanasov, Jeremy Cohen, Arthur Jacot, Eric J. Michaud, Enric Boix-Adserà, Nikhil Ghosh, Mason Kamb, Berkan Ottlik,

Blake Bordelon, Florentin Guth, Dhruva Karkada, and Joseph Turnbull. There will be a scientific theory of deep learning. *arXiv preprint arXiv:2604.21691*, 2026.

[81] Justin Sirignano and Konstantinos Spiliopoulos. Mean field analysis of neural networks: a law of large numbers. *SIAM Journal on Applied Mathematics*, 80(2):725–752, 2020.

[82] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. arXiv:2408.03314.

[83] H. Sompolinsky, A. Crisanti, and H. J. Sommers. Chaos in Random Neural Networks. *Physical Review Letters*, 61(3):259–262, July 1988.

[84] Konstantinos Spiliopoulos, Richard B. Sowers, and Justin Sirignano. *Mathematical Foundations of Deep Learning Models and Algorithms*, volume 252 of *Graduate Studies in Mathematics*. American Mathematical Society, 2025.

[85] Jonas Stapmanns, Tobias Kühn, David Dahmen, Thomas Luu, Carsten Honerkamp, and Moritz Helias. Self-consistent formulations for stochastic nonlinear neuronal dynamics. *Physical Review E*, 101(4):042124, April 2020.

[86] Alain-Sol Sznitman. Topics in propagation of chaos. In *École d'Été de Probabilités de Saint-Flour XIX — 1989*, volume 1464 of *Lecture Notes in Mathematics*, pages 165–251. Springer, 1991.

[87] David Tong. Lectures on quantum field theory. University of Cambridge, 2006. Lecture notes accessed for Part III of the Mathematical Tripos.

[88] David Tong. Lectures on kinetic theory. Lecture notes, 2012.

[89] David Tong. Statistical field theory. Lecture notes available at https://davidtong.org/teaching/statistical-field-theory/, 2012. Lectures presented for Part III of the Mathematical Tripos.

[90] David Tong. Statistical physics. Lecture notes available online at https://davidtong.org/teaching/statistical-physics/, 2012. Aimed at final year undergraduates.

[91] Nicolaas G. van Kampen. *Stochastic Processes in Physics and Chemistry*. North-Holland, 3 edition, 2007.

[92] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. arXiv:2206.07682.

[93] Christopher K. I. Williams. Computing with infinite networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1997.

[94] Lechao Xiao, Yasaman Bahri, Jascha Sohl-Dickstein, Samuel S. Schoenholz, and Jeffrey Pennington. Dynamical isometry and a mean field theory of CNNs: how to train 10,000-layer vanilla convolutional neural networks. In *International Conference on Machine Learning (ICML)*, 2018. arXiv:1806.05393.

[95] Greg Yang. Tensor programs I: Wide feedforward or recurrent neural networks of any architecture are gaussian processes. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019.

[96] Greg Yang and Edward J. Hu. Tensor programs IV: feature learning in infinite-width neural networks. In *International Conference on Machine Learning (ICML)*, 2021. arXiv:2011.14522.

[97] Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs V: tuning large neural networks via zero-shot hyperparameter transfer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2203.03466.

[98] Lenka Zdeborová and Florent Krzakala. Statistical physics of inference: thresholds and algorithms. *Advances in Physics*, 65(5):453–552, 2016.

[99] J. Zinn-Justin. *Quantum Field Theory and Critical Phenomena*. International Series of Monographs on Physics. Clarendon Press, 2002.